Prague University of Economics and Business

Faculty of Informatics and Statistics

THE COMPARISON OF THE VIDEO GAME DEVELOPMENT PROCESS WITH AND WITHOUT THE USE OF ARTIFICIAL INTELLIGENCE.

BACHELOR THESIS

Study programme: Applied Informatics

Author: Anton Ivanichenko

Bachelor Thesis Supervisor: Ing. Zuzana Šedivá, Ph. D

Prague, May 2025

Acknowledgement

I want to greatly thank Ing. Zuzana Šedivá, Ph. D for her support, guidance, and patience during the process of creation of this bachelor's thesis. Additionally, I want to thank the survey respondents and playtesters.

Abstract

The main goal of this bachelor thesis is to compare the processes of video game development with and without the use of artificial intelligence. The output of this work is a comparative analysis of two game prototypes: manually developed and artificially developed. The analysis focuses on three factors: time costs, financial costs, and overall quality of the prototypes.

This work also inspects several other topics, such as the definition of a video game, roles in the game development team, and components of a video game.

Keywords

Game, Game Development, Artificial Intelligence, Godot.

Abstrakt

Hlavním cílem této bakalářské práce je porovnat procesy vývoje videoher s využitím a bez využití umělé inteligence. Výstupem práce je srovnávací analýza dvou herních prototypů: jednoho vytvořeného manuálně a druhého vytvořeného pomocí umělé inteligence. Analýza se zaměřuje na tři faktory: časové náklady, finanční náklady a celkovou kvalitu prototypů.

Práce se také zabývá několika dalšími tématy, jako je definice videohry, role ve vývojářském týmu a komponenty videohry.

Klíčová slova

Hra, Vývoj videoher, Umělá inteligence, Godot.

Contents

A List of Figures

A List of Tables

Introduction

Video games are one of the most creative and complex forms of digital media. What makes them especially interesting is how all these parts come together in synergy. I was motivated by a desire to explore how games are made, how different roles contribute to the process, and how overall development can be better understood and improved.

The Goal of the Work

The main goal of the work is to compare the **manual** and **artificial** approaches to developing a video game.

For both approaches are defined characteristics, rules, and procedures. To achieve more accurate results, the rules for **manual** and **artificial** approaches must be followed throughout the entire development process. Deviation or non-compliance with procedures is recorded as **intervention**. This ensures consistency of results and allows for the precise identification of differences between approaches.

The result of the work are **two** artifacts in the form of game prototypes and a comparative analysis of development approaches.

Research Questions

Q1: What are the key differences between manual and artificial development approaches?

Q2: Are players able to differentiate between manually and artificially developed games?

Methodology

Throughout this work, the following methodological approaches were used:

- Definition analysis.
- Information analysis.
- Market analysis.
- Comparison.
- Creation of prototypes.
- Survey.
- A/B testing.
- Supervised interviews.

Criteria for evaluating prototypes:

1) Time costs.
2) Financial costs.
3) Overall quality (based on player reviews).

Structure of the Work

This work is divided into two main parts: theoretical and practical.

The theoretical part introduces the core concepts and themes that form the basis for the practical part. The theoretical part covers several areas, including an overview of the video game industry, the roles within a game development team, the exploration of game components, and the application of artificial intelligence in video game development.

The practical part focuses on the design, implementation, and comparison of two game prototypes. The practical part includes a definition of requirements for prototypes, a market analysis of similar games, and a collection and evaluation of player feedback.

Possible Limitations of the Work

The market for AI tools and services is rapidly evolving and transforming. As a result, the AI services, which are used in the practical part, may become outdated or obsolete in a relatively short amount of time. Therefore, there is a risk that the results of the work may not fully reflect the capabilities of newer versions of AI services.

Possible Benefits of the Work

This work provides an overview of video games, the gaming industry, the game development process, and the potential applications of AI tools in this field. It may be particularly valuable for programmers, game developers, and individuals interested in exploring the interaction between games and AI.

Annotated Literature

Throughout the development of this thesis, multiple information sources were used, including web articles, books, research papers, reports, and videos. Below are listed the most notable contributions that significantly influenced this work.

The book "The Art of Game Design: A Book of Lenses" by Jesse Schell (2019) served as a pivoting point for several topics in my work: the insights on the definition and characteristics of the game were quite useful; the "Element Tetrad" framework that was proposed by Jesse Schell was used as a basis for my categorization of game components. The book has also provided practical information and advice on aspects of game design.

The article "Real-Time Strategy: Evolutionary Game Development" by Martin Walfisz, Peter Zackariasson, and Timothy Wilson (2006) provided helpful information regarding development team composition and the game development process. The article also presented a model of project organization, which I adopted into my own model of development team composition.

The reports "Global Games Market Report 2023" and "Global Games Market Report 2024" by the Newzoo team (2024; 2024) were mainly used to describe the current state of the video game market and its trends.

The research paper "A Short and Simple Definition of What a Videogame Is" by Nicolas Esposito (2005) helped with defining the term "video game"; it also provided information about the term "gameplay" and explored the perspectives of other authors on how to approach the definition problem.

The books "Dive into Design Patterns" by Alexander Shvets (2021) and "Game Programming Patterns" by Robert Nystrom (2014) were used as supplemental material. The books explore and explain software architecture and design patterns. They also provide valuable insights on how to structure, optimize, and maintain the programming code.

1 The Video Game Industry

1.1 A Definition of the Term "Video Game"

Before describing the video game industry and its aspects, it is important to first define the term "video game". This term has been discussed across various publications, ranging from online posts and blogs to professional works and articles. Unfortunately, there is no uniform definition, and many sources define this term differently. Therefore, I have decided to provide my own definition for the term "video game".

I have examined definitions from **seven** sources. It should be noted that some of these sources define the term "game" instead of "video game". Nevertheless, I have included them, because video games are a subcategory of games; therefore, the definition of a video game should reflect the general principles and characteristics of a game. However, this approach has a slight risk of overgeneralization. To minimize this risk, I have only included definitions of the term "game" that come from video game industry specialists.

For each source, I inspected the definition's text and interpreted it into key themes, which the given definition includes. The definitions with their respective sources and themes can be found on the table below.

Table 1: Analysis of the definitions; Source: Author.

Source	Definition	Key themes
(VIDEO GAME | English Meaning - Cambridge Dictionary, n.d.)	*"A game in which the player controls moving pictures on a screen by pressing buttons."*	I/O device, Software
(Definition of Video Game, n.d.)	*"An interactive software that is used for entertainment, role playing and simulation."*	Interactive, Software, Entertainment
(Schell, 2019)	*"A problem-solving activity, approached with a playful attitude."*	Activity, Objectives, Entertainment
(Esposito, 2005)	*"A game which we play thanks to an audiovisual apparatus, and which can be based on a story."*	Media, Narrative
(Mullich, 2016)	*"A playful activity with rules and goals."*	Activity, Objectives, Rules
(Salen & Zimmerman, 2004)	*"A system in which players engage in an artificial conflict, defined by rules, that results in a quantifiable outcome."*	System, Objectives, Rules, Conflict

Source	Definition	Key themes
(Saleem, 2013)	*"An electronic game that has an interface designed for human interaction on a video device."*	I/O device, Software, Interactive

To strengthen my understanding, I overviewed the ten main characteristics of a game, which Jesse Schell describes in his book (Schell, 2019). Those characteristics are:

1) Games are entered willfully
2) Games have goals
3) Games have conflict
4) Games have rules
5) Games can be won and lost
6) Games are interactive
7) Games have challenges
8) Games can create internal value
9) Games engage players
10) Games are closed, formal systems

Additionally, in his research paper, Nicolas Esposito examines another closely related term called "gameplay", which is described as feature of a video game that could not be found in any other form of art (Esposito, 2005). For this reason, I have also decided to provide my explanation for the "gameplay" term.

In summary, I have formulated my own definitions by synthesizing information from the created table and the ten characteristics of games. My aim was to provide a basic understanding of this topic and create a context for the following chapters.

A **video game** is a type of interactive software and a form of digital media. The main purpose of a game is to entertain the player. Interaction occurs through designated input and output devices such as a keyboard, mouse, controller, monitor, and headset. The primary interaction between the player and the game is called **gameplay**.

Gameplay is a system of in-game components and their dynamic interactions with each other. This system usually defines the game limitations, rules, objectives, and conflicts. Gameplay should create meaningful value for the player.

1.2 Brief Overview of the Video Game Industry

The video game industry is one of the economic sectors of the entertainment industry that specializes in the development, marketing, distribution, and monetization of video games across various platforms, such as consoles, personal computers, and mobile devices. The video game industry is considered one of the most important and innovative sectors, due to its cultural, social and technological influence (Maria et al., 2022).

The industry presents thousands of career opportunities in multiple disciplines across the globe. Moreover, it advances innovation in computer hardware and software, often pushing and expanding the limits of technology. The cutting-edge solutions, which were introduced in games, are also often later appropriated in different fields, such as virtual reality, autonomous vehicles, medical imaging and many others (Thau, 2024). The industry also plays a big role in education, culture and social interaction (Shroyer, 2023).

The video game industry generates significantly higher revenue than the movie, recorded music, and book publishing industries. To illustrate this difference, I have provided the following data on the revenues for each industry in the year 2022:

- The video game industry generated $184.4 billion in revenue (Arora, 2023).
- The movie industry generated $26 billion in revenue (box office) (Arora, 2023).
- The recorded music industry generated $26.2 billion in revenue (Arora, 2023).
- The book publishing industry generated $137.12 billion in revenue (Straits Research, 2024).

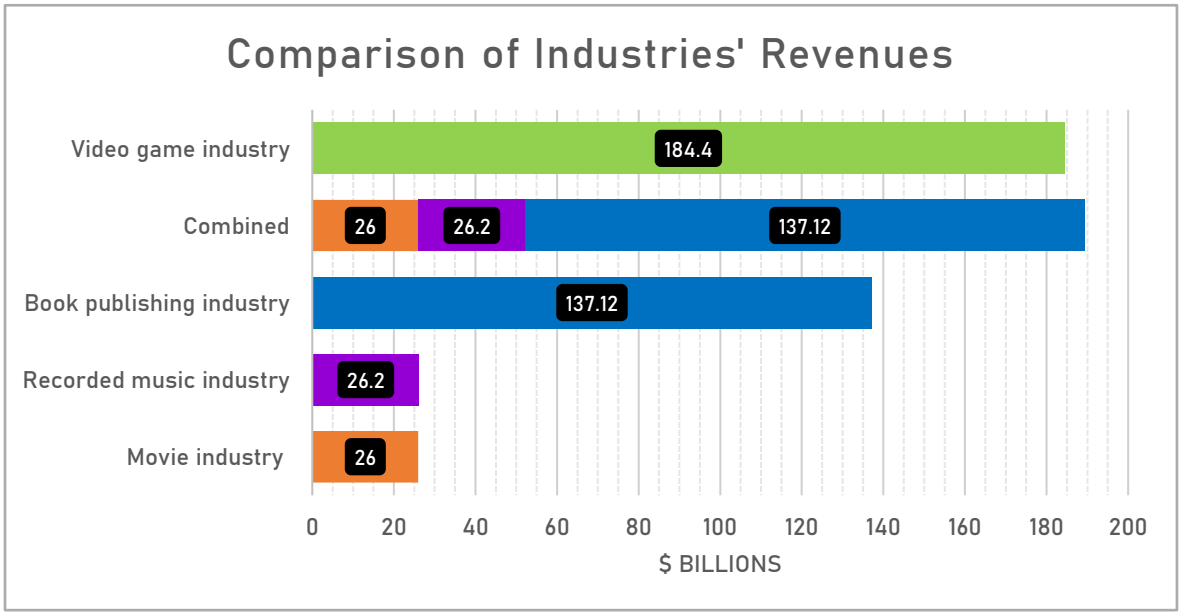


Figure 1: Comparison of industries' revenues (in the year 2022); Source: Author.

Overall, video games are a vast and dynamic sector of entertainment industry, with a continuously growing influence on technology, economy and culture-social aspects.

1.3 The Current State of Video Game Market

The video game market is slowly but consistently expanding. Forecasts expect the market to grow even further in the coming years (Wijman, 2024). The marker's growth is driven by wider access to the high-speed internet and greater online connectivity, new innovations in hardware and software such as more powerful soundcards, GPUs and CPUs, improved visual capabilities and rendering techniques, and more advanced AR/VR devices, all of which are enhancing the gaming experience (Maria et al., 2022; Precedence Research, 2025).

A rise in video game popularity resulted in a higher demand for games. Consequently, the number of video games is also steadily increasing year after year. This presents both opportunities and challenges for industry.

The oversaturation of the market has made discoverability more difficult, particularly for smaller companies and independent developers, as thousands of new titles are released each year. Additionally, rushed game development cycles (UNI Global Union, 2022), which are often driven by market competition, have resulted in unfinished, buggy, and low-quality releases, resulting in higher dissatisfaction and distrust of the players. The visibility itself is currently a major concern due to market oversaturation.

For example, on Steam (digital game distribution service) alone, the number of games released reached 14 285 in 2023 and 18 942 in 2024, with a 15.5% and 32.6% year-over-year increase, respectively.

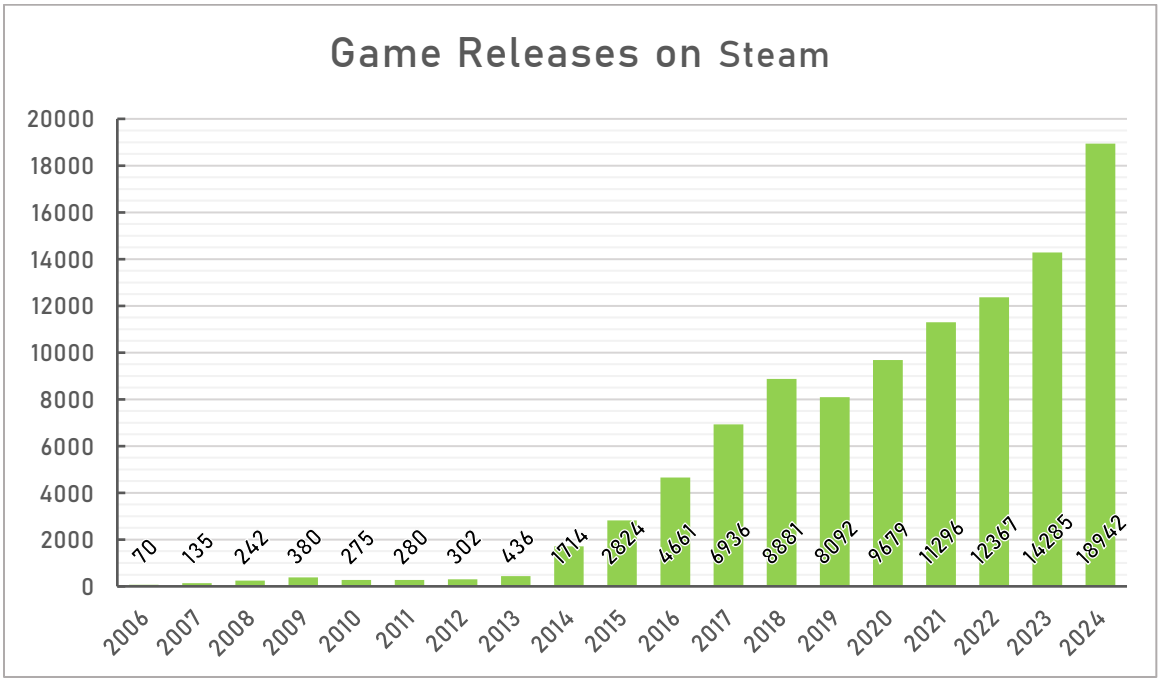


Figure 2: Game Releases on Steam; Source: (SteamDB, 2025).

1.3.1 Market Segmentation: Revenue by Region

The revenue distribution varies significantly by region due to macroeconomic factors, such as economic stability and wealth, cultural aspects, governmental policies, technological advancements, and player preferences. The market is commonly segmented by the following regions:

- Asia-Pacific, which is dominated by China, Japan, and South Korea.
- Europe, where major players are Germany, the UK, and France.
- Latin America, where major players are Mexico, Brazil, and Argentina.
- Middle East and Africa, which is represented by Turkey, the UAE, and Egypt.
- North America, which consists only of the USA and Canada.

Table 2: Market revenue comparison by region (2023 and 2024); Source: Author.

YEAR	**2023**			**2024**		
REGION	REVENUE ($B)	CHANGE (%YoY)	SHARE (%)	REVENUE ($B)	CHANGE (%YoY)	SHARE (%)
Asia-Pacific	$ 84.1B	-0.8%	46%	$ 85.9B	+1.5%	46%
North America	$ 50.6B	+1.7%	27%	$ 50.2B	-0.6%	27%
Europe	$ 33.6B	+0.8%	18%	$ 34.8B	+3.2%	18%
Latin America	$ 8.7B	+3.8%	5%	$ 9.1B	+6.2%	5%
Middle East and Africa	$ 7.1B	+4.7%	4%	$ 7.7B	+8.7%	4%
TOTAL	**$ 184.0B**	**+0.6%**	**100%**	**$ 187.7B**	**+2.1%**	**100%**

Reports estimate that the global video games market has generated $184.0 billion in 2023 and $187.7 billion in 2024. In both years, the biggest contributions to revenue came from the USA ($47.3 billion in 2023 and $47.0 billion in 2024) and China ($43.6 billion in 2023 and $45.0 billion in 2024). The combined revenue from the USA and China makes up approximately 50% of the global market. The market's growth is expected to continue, with forecasts projecting it to reach a value of $213.3 billion in 2027 (Buijsman, 2024; Wijman, 2024). The graphical representation of the revenue data can be found below.

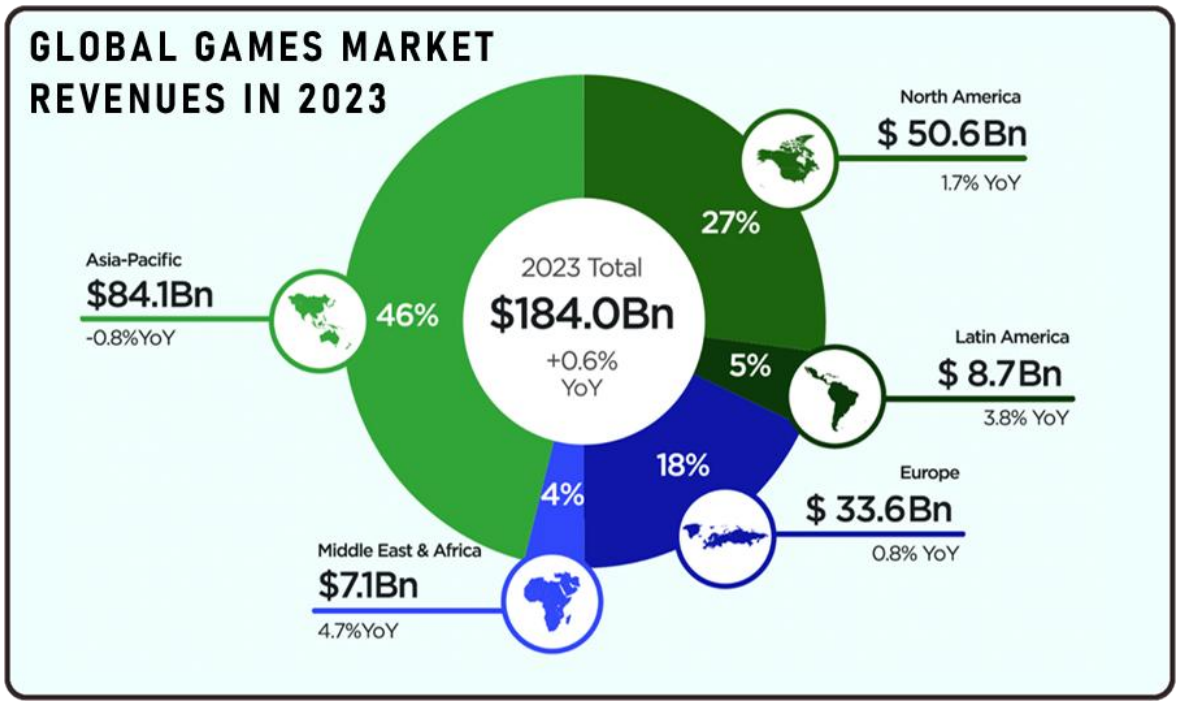


Figure 3: Market revenue by region (2023); Source: (Wijman, 2024).

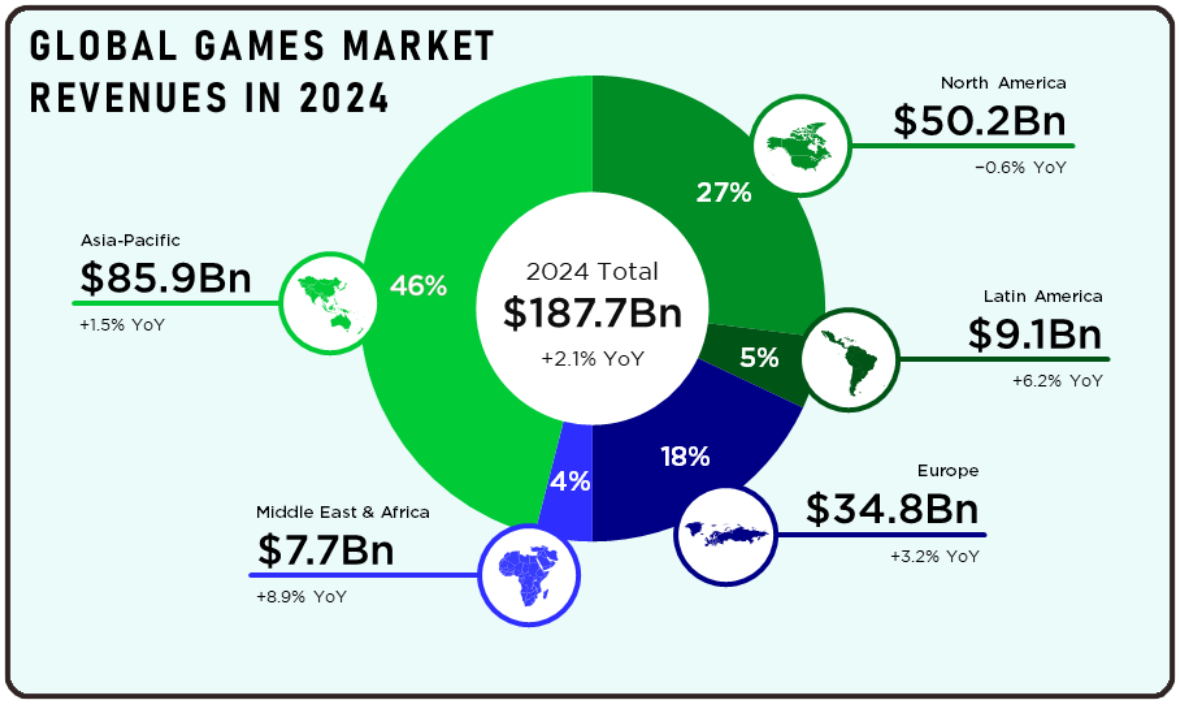


Figure 4: Market revenue by region (2023); Source: (Buijsman, 2024).

1.3.2 Market Segmentation: Platforms

Platform Categorization

The video game industry typically splits into three main platform groups: **Mobile**, **Console**, and **PC**.

A **mobile game** is a video game designed to be played on **mobile devices** such as smartphones and tablets. Mobile games typically have the following features:

- Mobile games are affordable: they are either completely free or have generally lower prices than console and PC games, because they use different monetization models, such as "freemium" or advertising (Peñaranda, 2022).
- Most mobile games fall into the casual genre: those games have simple gameplay; therefore, they appeal to a wider audience (*Mobile Gaming*, n.d.).
- Mobile games have much lower system requirements than their counterparts (*Mobile Gaming*, n.d.).

Due to its accessibility and convenience, mobile gaming is considered the most popular form of gaming in the world (*Mobile Gaming*, n.d.; Peñaranda, 2022).

A **console** is a specialized computer that is primarily designed and optimized for playing video games. These kinds of games are called **console games**. Consoles typically come with controllers for playing and usually require a connection to a display device, such as a monitor or TV screen. Unique features of consoles include exclusive games, user-friendly interfaces, support for streaming services, and high performance. There are different types of consoles:

- Home consoles - larger, more powerful consoles that are placed in a room and serve as a hub for entertainment.
- Handheld consoles - smaller, more portable consoles that come equipped with their own display unit and in-built controller functions.
- Hybrid consoles - consoles that combine elements of both home and handheld consoles.

Video game consoles have software and hardware that are different from desktop computers. Both software and hardware are under the control of their respective manufacturers (*Definition of Video Game Console*, n.d.; *Video Game Consoles | Dimensions.Com*, n.d.).

PC gaming involves playing video games on **desktops** or **laptops**, typically on high-end Windows machines. Key features include access to a vast game library, support for various controllers and high-resolution displays, superior graphics and performance, and the ability to customize or upgrade both hardware and software. PC gaming also allows modding: use of user-created content that modifies games (Tait, 2024).

Revenue by Platform

Table 3: Market revenue comparison by platforms (2023 and 2024); Source: Author

YEAR	**2023**			**2024**		
PLATFORM	REVENUE ($B)	CHANGE (%YoY)	SHARE (%)	REVENUE ($B)	CHANGE (%YoY)	SHARE (%)
Mobile games	$ 90.5B	-1.4%	49%	$ 92.6B	+3.0%	49%
Console games	$ 53.1B	+1.7%	29%	$ 51.9B	-1.0%	28%
PC games	$ 38.4B	+5.3%	21%	$ 43.2B	+4.0%	23%
Browser PC games	$ 1.9B	-16.9%	1%	-	-	-
TOTAL	$ 184.0B	+0.6%	100%	$ 187.7B	+2.1%	100%

Reports demonstrate that in both years, mobile games maintained a stable dominance, generating $90.5 billion in 2023 and $92.6 billion in 2024, accounting for almost 50% of the total market revenue. Console games showed a slight decline from $53.1 billion to $51.9 billion. Meanwhile, PC games continue to show growth, rising from $38.4 billion in 2023 to $43.2 billion in 2024, increasing their market share by 4%. Revenue from browser PC games, however, continued its decline and was no longer separately reported in 2024 (Buijsman, 2024; Wijman, 2024). The graphical representation of the revenue data can be found below.

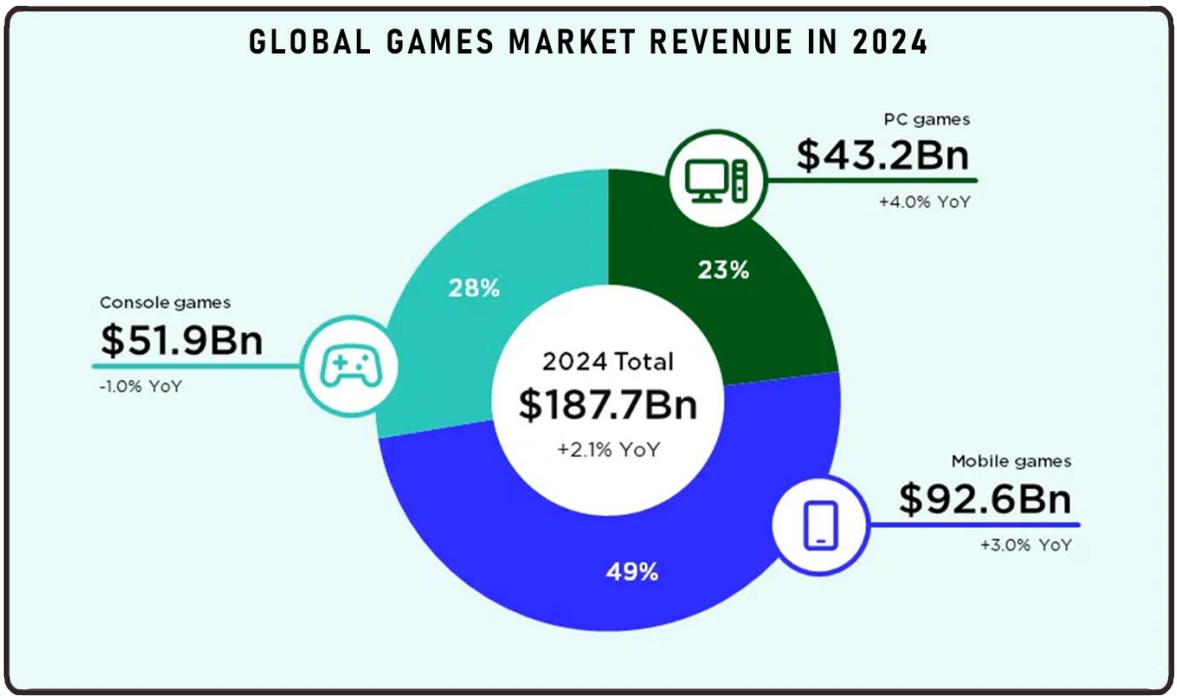


Figure 5: Market revenue by platforms (2023); Source: (Wijman, 2024).

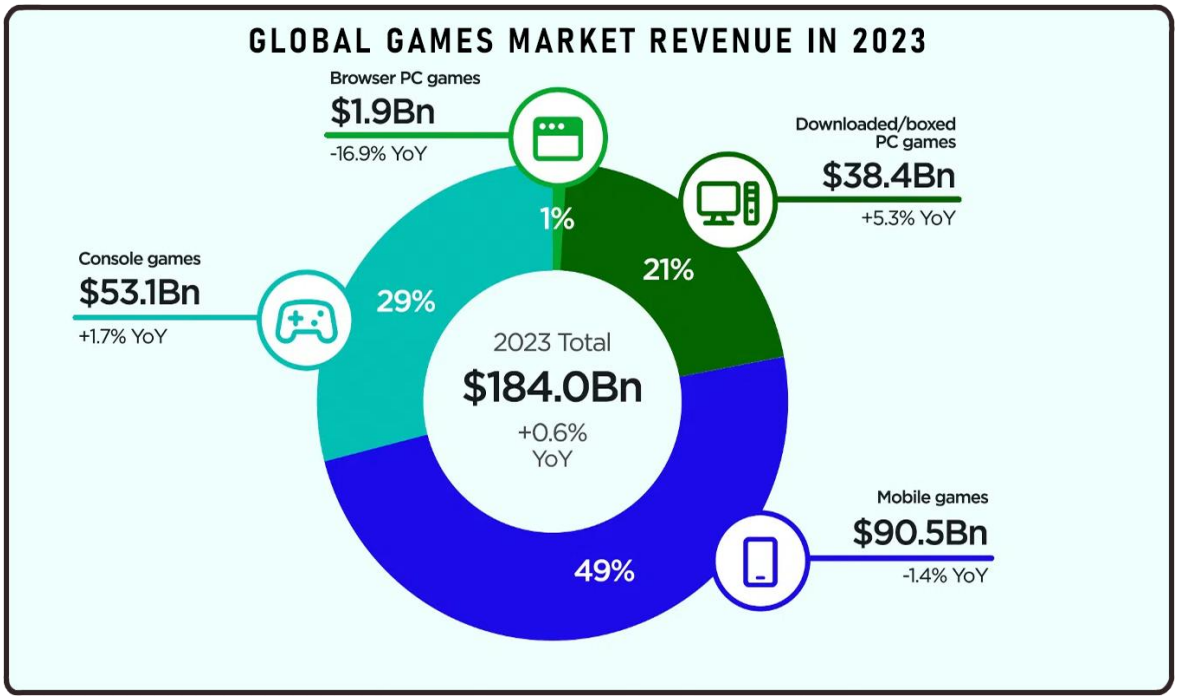


Figure 6: Market revenue by platforms (2024); Source: (Buijsman, 2024).

1.3.3 Market Segmentation: Games by Budget

Game Categorization

A widely used method of categorizing video games relies on the scale of their development budgets and team sizes. This classification divides games into three categories: **Indie**, **AA**, and **AAA**. Each of these categories reflect not only the level of financial investment but also differences in development scope and duration, marketing and publishing strategies, and the degree of creative freedom.

Indie games (Independent games) – games that are developed by small teams with limited budgets, often prioritizing creativity and innovations in gameplay. Examples: Celeste, Hollow Knight, Potion Craft: Alchemist Simulator.

AA games (Double-A games) – mid-tier productions with higher financial constraints and production values. Examples: It Takes Two, Kingdom Come: Deliverance, Stray.

AAA games (Triple-A games) – large-scale productions backed by major publishers with high development and marketing budgets. Examples: Red Dead Redemption 2, God of War Ragnarök, the Witcher 3.

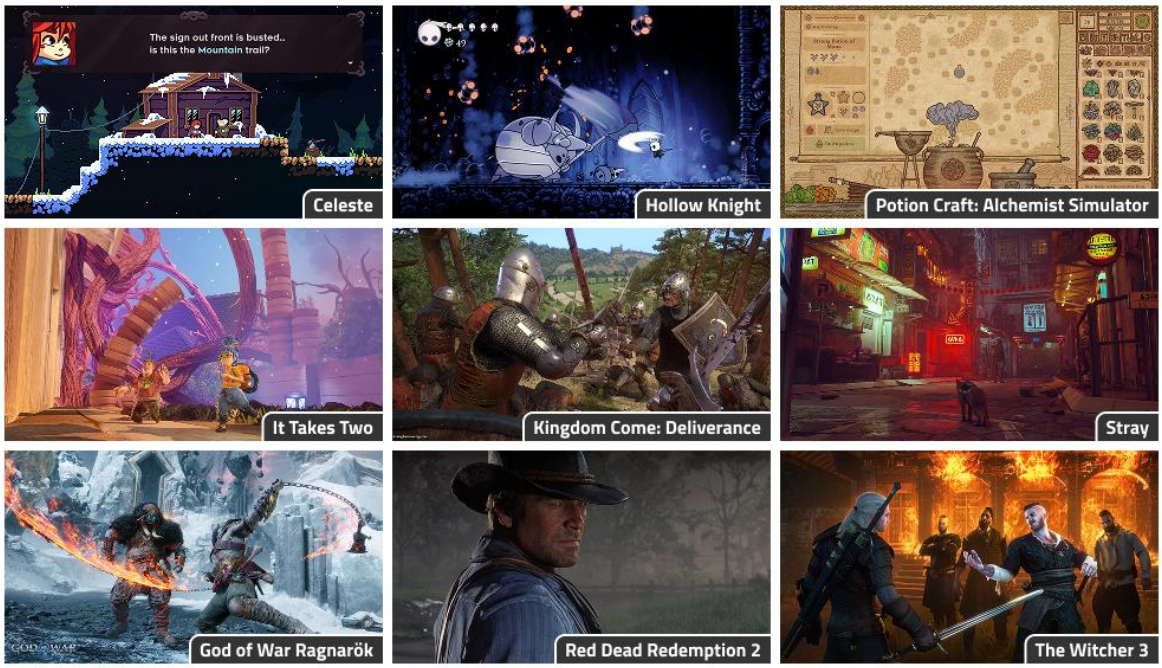


Figure 7: Examples of games; Source: Author.

It is important to note that this classification is informal and is not supported by any strict industry standard or norm. Instead, it represents a consensus among developers, publishers, and players, based on common observed patterns in the development process. The following table summarizes different sources and outlines the main characteristics typically associated with each budget category.

Table 4: Characteristics of Indie, AA, and AAA games; Source: (INLINGO, 2024; Porokh, 2023; Rocket Brush Studio, 2023; Zolotarenko, 2024).

Category	Indie	AA	AAA
Development Budget	Up to $1,000,000.	Between $1,000,000 and $50,000,000.	Over $50,000,000.
Development Team Size	Small teams, ranging from a single developer to around 10 people.	Mid-sized studios with 30 to 50 developers.	Large teams, typically ranging from 50 to 100 full-time developers, with some projects involving more than 1,000 members.
Development Time	Varies widely: usually between a few months and 3 years.	Generally, it ranges between 1 and 3 years.	Typically takes 3 to 7 years, with some projects extending beyond this timeframe.
Graphics, Technology and Creative Freedom	Developed with limited technical resources. The focus is on innovation, unique artistic vision and new gameplay mechanics. Often explore unconventional themes with high creative freedom.	High-quality graphics and technology. Balance between creative freedom and commercial appeal. Feature varied genres with unique elements.	Developed with cutting-edge technology and graphics. High production values. Inclusion of famous voice actors. Aim for mass-market appeal, often incorporating "tried-and-true" gameplay mechanics and genres.
Publisher	Often self-published or lack a dedicated publisher.	Supported by publishers, which allows greater creative freedom compared to AAA games.	Typically backed by established publishers.
Marketing	Due to limited budget, games rely on word-of-mouth and social media.	Marketing efforts are modest, focusing on digital advertising.	High budget provides extensive marketing campaigns, including trailers, teasers, and promotional events, generating significant hype (awareness) before release.

Category	Indie	AA	AAA
Distribution	Distributed through digital platforms like Itch.io, Steam, and mobile app stores.	Primarily distributed digitally, with occasional limited physical releases, especially for collector's editions.	Distributed globally across multiple platforms, combining both physical and digital releases.
Franchising	Sometimes it may lead to sequels if a game is successful.	A game is likely to develop into franchise upon achieving success.	A game is often designed with franchising in mind, with expectations of multiple sequels

The "Indie" Categorization Nuances

It should be noted that "indie" categorization may sometimes be misleading. While most games that are recognized and labelled as indie generally have lower budgets, small teams, and limited marketing, certain titles break these criteria. The notable examples are the games Neon White, The Witness, and Dave the Diver.

Neon White and The Witness had core development teams of **fifteen** (*MobyGames | Neon White Credits*, 2022) and **six** members (Webber, 2016), respectively, fitting in indie constraints. However, their budgets have significantly surpassed the usual budget constraints: Neon White had a **$2 million** budget (McZeal, 2023), while The Witness had a **$6 million** budget (McZeal, 2023); therefore, both fall within the range of a typical AA budget game.

Meanwhile, the game Dave the Diver adds another layer of complexity. Despite being widely recognized as an indie game, it was developed by Mintrocket, a subsidiary of Nexon, one of the largest game publishers in South Korea. While it features an indie-style aesthetic and creative gameplay, its development was backed by a major corporation, raising questions about whether substantial publisher involvement should disqualify a game from the "indie" category (Zack Daniels, 2023).

Additionally, the term "indie" is commonly used to describe self-published (independent) games. While this classification is correct, it comes with a few exceptions. For example, a game by Larian Studios, called Baldur's Gate 3, is a large-scale project with a high production budget, a big development team, and an extensive marketing campaign. It is a highly polished product that surpasses many AAA titles. However, it is also a self-published game, which technically makes it an indie game (Steam Publisher, 2025). This further demonstrates that categorization should always be based on several factors.

To address those issues, some specialists and developers have suggested introducing a new category called "III" (Triple-I): an independently funded game that offers high production values, quality, and scope on par with AA games or even AAA projects (Carter, 2024; Lemne, 2016; Zack Daniels, 2023).

Correct classification is important to prevent misinformation and false advertising. Firstly, this ensures that consumers receive accurate information about a game before buying it or supporting its developers. Secondly, this allows smaller games and studios to get the recognition they deserve, because mislabeling games as "indie" makes it harder to gain visibility and stand out for genuine independent games.

2 Roles in a Game Company

Creating a video game is a complex task that requires a diverse set of skills. Modern game development involves a wide range of roles: some projects have generalist roles, while others feature highly specialized positions. The inclusion of a specific role depends on the game's genre, setting, gameplay, platform, and budget. This chapter explores and examines the roles and teams in game development.

2.1 Expert Outlooks on Roles in a Game Company

An analysis of roles within a game development team requires information from game industry professionals. For better understanding, I have selected insights from **five** specialists. Each of them provided a list of roles along with explanations based on their professional experience. Some specialists also mentioned more specialized variants of roles (sub-roles). The selected experts are:

- **Thaddeus Sasser,** a video game designer, has worked at three of the largest game publishers: Activision, Ubisoft, and Electronic Arts; he has participated in the development of the Call of Duty and Battlefield franchises.
- **David Mullich**, a video game producer and designer with a long history in interactive entertainment; one of the key developers of games Heroes of Might and Magic III and Vampire: The Masquerade – Bloodlines.
- **Masahiro Sakurai**, a video game director and concept designer, is best known as the creator of both the Kirby and the Super Smash Bros. series on Nintendo platforms.
- **Timothy Cain** (Tim Cain), a video game developer, best known as one of the main creators of Fallout and Fallout 2, has also worked on Vampire: The Masquerade – Bloodlines, Tyranny, and The Outer Worlds.
- **Elizabeth England** (Liz England), a game designer specializing in systems design, has taken part in the development of Scribblenauts game series and Sunset Overdrive.

The primary objective of this analysis was to provide information for description and categorization of key roles that contribute to the process of developing a video game. To present these roles in a consistent structure, I used the following format:

1) **Role Name** (alternative or commonly used aliases, if applicable).
   a) Sub-role Name, if applicable.

It should be noted that the main body of the work includes role definitions only from **Thaddeus Sasser** and **David Mullich**. Explanations provided by the other experts are included in the appendices.

2.1.1 Thaddeus Sasser on Game Development Roles

In “Game Development Roles" article Thaddeus Sasser (2023) gives a very simplified explanation of the roles without any deeper details. He defines **four** main roles in game development:

1) **Artist** - a person, who creates a visual look of the game.
2) **Designer** - a person, who makes the game interesting and entertaining.
3) **Producer** - a person, who controls that the game will be completed on time, on budget, and at the expected quality level.
4) **Programmer** (Engineer) - a person, who writes the code and makes the game work.

Although, information, which Thaddeus Sasser provides is brief, it is still enough to make a surface-level understanding of these roles.

2.1.2 David Mullich on Game Development Roles

In his blog, David Mullich (2015) describes typical roles on a game development team. He outlines **six** roles in total:

1) **Project Manager** (Development Director, Producer) - a person, who provides the team with direction and support, while also balancing the time and budget constraints.
2) **Game Designer** - a person, who takes the idea of the game and expands it into a detailed description of how the game should work, which includes goals, rules, and story.
3) **Artist** - a person, who is responsible for creating the game’s art assets, such as game concepts, models, textures, entity skeletons (rigs), and animations.
4) **Audio Specialist** - a person, who is responsible for creating the music and sound effects.
5) **Programmer** - a person, who writes a code which integrates game design, art and audio assets together. May also be responsible for creating additional development tools.
6) **QA Specialist** - a person, who tests different versions of the game and ensures that it is complete and works properly.

David Mullich also outlines effective team meeting practices and notes that successful teams and games are built on **teamwork**, **communication**, and **trust**.

2.1.3 Masahiro Sakurai on Game Development Roles

In the video "Jobs in Game Development [Team Management]" Masahiro Sakurai provides a thorough explanation of game development roles (Sakurai, 2022). His explanation offers a structured overview of **fourteen** the most common roles in game development:

1) **Director**
2) **Planner** (Game Designer)
3) **Programmer**
4) **Modeler** (3D Graphics Artist)
5) **Artist** (2D Graphics Artist)
6) **Animator**
    a. Character Rigger
7) **UI Designer**
8) **Visual Effect Designer**
9) **Sound Designer**
    a. Music Composer
    b. Sound Effect Designer
    c. Voice-Over Supervisor
10) **Technical Support**
11) **Manager**
12) **Game Tester** (Play Tester)
13) **Bug Tester**
14) **Footage Specialist**

Masahiro Sakurai mentions that the game industry also includes **PR** (Public Relations), **Sales**, and **Production** roles; however, these roles are not typically considered **"developer"** roles. He additionally comments that although these roles have distinct responsibilities and objectives, all of them need to work together in tandem to achieve mutual understanding and move the entire team forward (Sakurai, 2022).

2.1.4 Timothy Cain on Game Development Roles

In his video, Timothy Cain gives detailed insights into roles in game development (Cain, 2024). He discusses their structure, responsibilities and importance within a development team. Timothy Cain defines **seven** main roles and several sub-roles:

1) **Game Director** (Project Leader)
    a. Design Director
    b. Programming Director
    c. Art Director
2) **Producer**
    a. Executive Producer
    b. Lead Producer
    c. Assistant Producer
3) **Programmer**
    a. Gameplay Programmer
    b. AI Programmer
    c. Graphics Programmer
    d. Networking Specialist
4) **Audio Specialist**
    a. Music Specialist
    b. Sound Effects Specialist
5) **Art Specialist**
    a. 3D Artist (3D Modeler)
    b. 2D Artist
    c. UI Artist
    d. Texture Artist
    e. Animator
    f. Rigger
    g. Concept Artists
    h. Cinematics Artist
    i. Motion Capture Artist
6) **Game Designer**
    a. Systems Designer
    b. UX Designer
    c. Narrative Designer
    d. Level Designer
7) **QA Specialist**

Similarly to Masahiro Sakurai, Timothy Cain also mentions roles outside the core development group, namely **HR**, **Marketing**, **Localization** departments and different administrative roles (Cain, 2024).

2.1.5 Elizabeth England on Game Development Roles

Elizabeth England has rather a unique and creative method of defining roles in a game company. To help people better understand the breakdown of roles, she uses an analogy called **"The Door Problem"** - a problem that initially may seem trivial and simple, after a more thorough observation, becomes multi-dimensional and complex.

The creation of a game also can be described as a such problem, as it often requires a wide range of skills and knowledge. In her blog Elizabeth England (2014) compares how people with different specialties approach the problem and briefly explains what each of them does to solve it. She has listed **thirty-five** specific roles:

1) **Creative Director**
2) **Project Manager**
3) **Designer**
4) **Concept Artist**
5) **Art Director**
6) **Environmental Artist**
7) **Animator**
8) **Sound Designer**
9) **Audio Engineer**
10) **Composer**
11) **Visual Effects Artist**
12) **Writer**
13) **Lighter**
14) **Legal**
15) **Character Artist**
16) **Gameplay Programmer**
17) **AI Programmer**
18) **Network Programmer**
19) **Release Engineer**
20) **Core Engine Programmer**
21) **Tools Programmer**
22) **Level Designer**
23) **Combat Designer**
24) **Systems Designer**
25) **Monetization Designer**
26) **QA Tester**
27) **Localization Specialist**
28) **Producer**
29) **Publisher**
30) **CEO**
31) **PR**
32) **Community Manager**
33) **Customer Support**
34) **UI Designer**
35) **UX Specialist** (Usability Researcher)

Elizabeth England's breakdown includes by far the highest number of roles within a development team. Particularly valuable for my analysis was her inclusion of **supporting roles**, most of which were not mentioned in other classifications, such as the localization specialist, legal advisor, community manager, and several others.

2.2 The Composition of a Development Team

Martin Walfisz, Peter Zackariasson, and Timothy Wilson (2006) describe the game's development as "a group effort", because with time, the game has become more sophisticated and time-pressuring. They state that the typical foundation of each project within a game development is four main teams: design team, programming team, art team, and audio team. These teams are interconnected with each other, coordinated by a project manager, who works primarily through team leaders (leads). The project manager has two key responsibilities:

- To ensure the game is fun and engaging.
- To ensure the project is delivered on time and within budget constraints.

To achieve this, the project manager interacts with:

- An internal producer, who controls the progress and state of the game.
- An external producer, who oversees funding and publishing of the game.

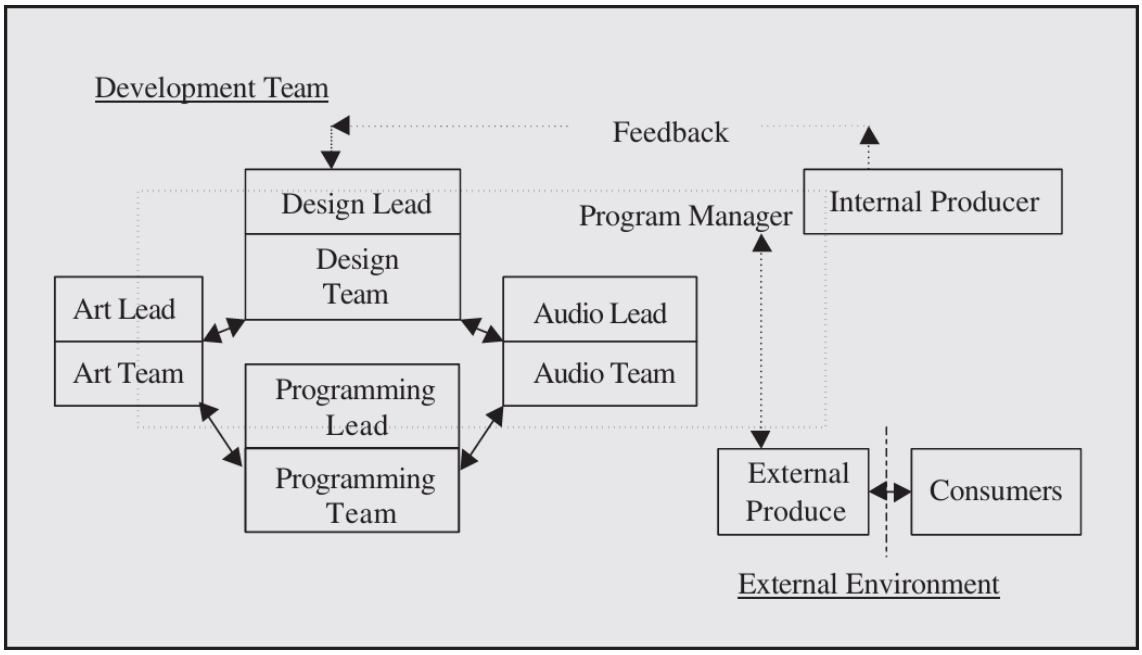


Figure 8: Structure of development team; Source: (Walfisz et al., 2006).

The collective of authors also describes two environments: teams, project manager, and internal producer are members of an **internal involvement** (a company or a studio), while external producer and consumers are members of an **external involvement**. The collective also notes that this structure is replicated for each individual project in a multi-project firm (Walfisz et al., 2006).

Unfortunately, this model does not include several roles, which appeared in the analysis of information from the experts, thus it only a partial view of the overall team structure, focusing on core development team. Additionally, the model does not clearly explain the main responsibilities of teams. For these reasons, I have decided to improve the model by creating an **extended framework**.

2.3 Extended Model of Development Team Composition

In my model, I have introduced **three** new teams and additionally have changed the naming of the existing teams, so now they are able to represent a slightly larger spectrum of roles. The explanations for each team's responsibilities and functions are listed below.

2.3.1 Responsibilities of Teams

Production and Management

The "Production and Management" team oversees the entire development process. Producers and managers coordinate and direct all other teams. The typical responsibilities of this team include planning and controlling tasks and goals; allocating and managing timelines, budget and personnel; establishing a smooth workflow and communication between teams; and identifying and resolving development problems (Kovtun, 2024; Minaiev, 2025; Sakurai, 2022).

It should be noted that some positions in a game development team are deeply integrated into other teams. For example, sub roles of the director, such as creative director, programming director, and art director, provide a substantial contribution to the output of a respective team due to their high level of skill and experience (Cain, 2024). Therefore, it is quite difficult to fully separate and assign them to the "Production and Management" team.

Game Design and Creative Direction

The "Game Design and Creative Direction" team members are responsible for shaping the creative vision and engaging experience of the game. They have diverse responsibilities and are involved in almost all aspects of a game. This team is typically represented by a variety of game designers, who develop gameplay mechanics and rules; build levels and maps; create story concepts and narrative elements; prototype and iterate ideas. Their work also involves creating design documents, which play the main role in collaboration between other teams (Kovtun, 2024; Minaiev, 2025; Sakurai, 2022).

Programming and Engineering

The "Programming and Engineering" team's primary responsibility is to write the code that ensures the game functions correctly. This team interprets design documents and translates them into executable commands for the computer (Sakurai, 2022). Generalist programmers are typically tasked with implementing gameplay mechanics, such as controls, object interactions, and in-game systems (Cain, 2024). Additionally, this team includes many specialized roles focusing on network infrastructure, AI behavior, physics simulation, rendering and graphics, the game engine, and additional development tools. Overall, programmers form the backbone of game development, integrating game design, art, and audio together (Kovtun, 2024; Minaiev, 2025; Sakurai, 2022).

Visuals and Animation

The "Visuals and Animation" specialists are responsible for creating the aesthetic visual look and feel of the game. Their main task is to maintain a consistent artistic style across the game. The overall visual style consists of different components such as 2D illustrations, 3D models, animations, visual effects (VFX), textures, in-game footage, and user interface (UI) elements. In addition, their role sometimes extends to optimizing these graphical components and creating custom art tools and scripts (Kovtun, 2024; Minaiev, 2025; Sakurai, 2022).

Sound and Music

The "Sound and Music" specialists are responsible for defining the audio style of the game. There are **three** categories in which their work falls (Sakurai, 2022):

- **Sound** - various sound effects (SFX), ambient sound, and audio feedback elements.
- **Music** - musical pieces, songs, and compositions.
- **Voice** - recordings of voice actors: narrations, replicas, and dialogs.

Similarly to the "Visuals and Animation" workflow, a big effort is also put on optimization: sound editing, mixing, and mastering - to maintain clarity and balance across all audio assets. The "Sound and Music" team enhances the gameplay, artistic style, and narrative of the game (Kovtun, 2024; Minaiev, 2025).

Quality Assurance and Testing

The "Quality Assurance and Testing" specialists ensure that a game functions as intended and meets quality standards before and after release. There are **five** main areas of their work:

- **Functionality** - focus on gameplay features, mechanics and general functions.
- **Performance** - focus on frame rates, load times, and memory usage.
- **Usability** - focus on the player experience.
- **Compliance** - focus on platform-specific requirements and standards.
- **Compatibility** - focus on testing the game across various devices, operating systems, and configurations.

Their work also includes manual playtesting; automated testing; gameplay balancing; collecting feedback from users; documenting problems and reporting them back to the development team (Kovtun, 2024; Minaiev, 2025; Sakurai, 2022).

Marketing and Public Relations

The "Marketing and Public Relations" team plays a crucial role in generating awareness and increasing visibility for a game. They implement various digital marketing strategies, tactics, and campaigns before, during, and after a game's release. However, their role extends beyond simply promoting a product; they also focus on building long-term relationships and partnerships.

One of their main responsibilities is reaching out to journalists and influencers, such as streamers, content creators, and other social media personalities. Additionally, their work involves collaborating with other companies.

Another major responsibility is establishing a connection between developers and gamers, often through community-building. This includes choosing the right media channels for promotion, managing player expectations, and maintaining open, honest communication with fans throughout development. In return, the fanbase provides interest, loyalty, and feedback.

Overall, the "Marketing and Public Relations" team's efforts help the game to achieve a significantly higher level of success and to have a more positive impact on reputation (Fisher, 2023; Mizina, 2025; Torossian, 2024).

Support and Operations

The "Support and Operations" team combines a broad range of duties that ensure the smooth functioning of a game company, which includes:

- **Legal** - mostly consists of regulatory work, which includes managing IP (intellectual property) rights, drafting EULA (end-user license agreement), negotiating publishing agreements, and solving other law-related issues (Davies, 2024).
- **Analytics** - an important and complex discipline, which is focused on data analysis and interpretation, for example identification of user patterns and preferences, improvement of acquisition and retention, detection of fraudulent and suspicious activities, and optimization of game development processes (Whitehead, 2024).
- **Localization** - a vital part of any game, which allows to increase market reach and enhance player experience; it consists of translation and adaptation of game content. Localization specialists must consider cultural norms, nuances, and preferences to avoid negative impact (Shroyer, 2023).

These represent only a part of their overall responsibilities; other functions include finance, office administration, security, technical support, maintenance, and many other behind-the-scenes operations. The "Support and Operations" team serves as a solid foundation for all other teams.

2.3.2 Relations Between the Teams

I have outlined two main groups: **core teams** and **support teams**. The main goal of the core teams is to create direct value for the game, while the support teams assist them in achieving this objective. Additionally, I consider "Production and Management" to be an independent team, as its main responsibility is to coordinate and direct the work of all other teams.

Table 5: Development groups; Source: Author.

Core Teams	**Support Teams**
Game Design and Creative Direction	Quality Assurance and Testing
Programming and Engineering	Marketing and Public Relations
Visuals and Animation	Support and Operations
Sound and Music	
Independent Team	
Production and Management	

Each team is interconnected with other teams in the same group. The "Production and Management" team connects the groups with each other. Additionally, each team in the **core group** is indirectly linked with the "Support and Operations" team, due to this team having a wide variety of personnel and auxiliary tasks. All teams are located in an internal environment (a company or a studio); however, "Production and Management" and teams of the **support group** can directly interact with an external environment (clients, media, and other companies).

The graphical representation of this framework, including teams, their connections with each other, and examples of team members can be found below.

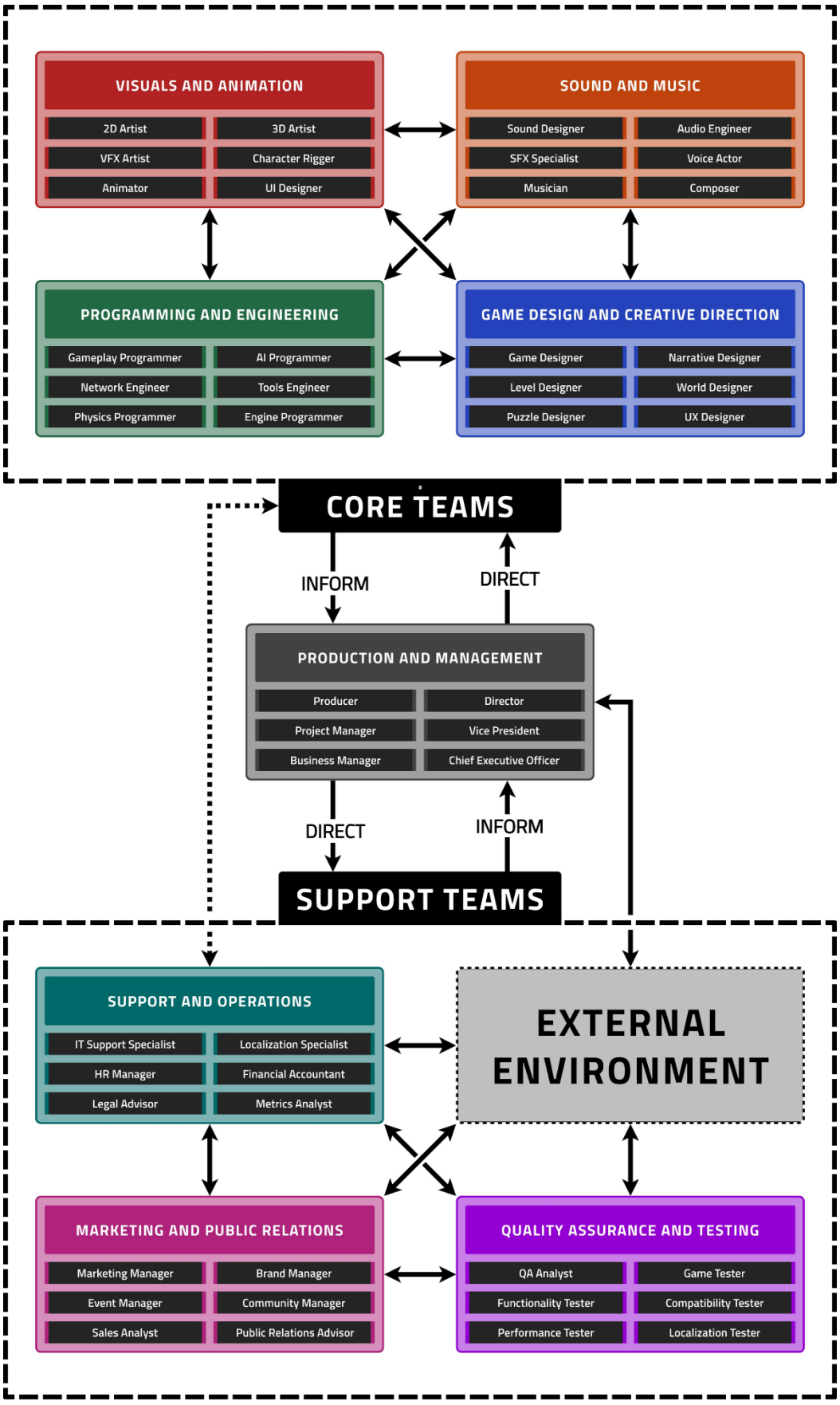


Figure 9: Expanded structure of development teams; Source: Author.

3 Game Components

The development of a video game is a complex process that involves turning ideas and concepts into defined content. This chapter describes what elements make such content, how these elements can be classified, what their relationships and functions are, and what factors limit their implementation.

3.1 Abstract Components

Games vary significantly in their themes, genres, and gameplay. Therefore, a certain level of generalization and abstraction is necessary to properly describe the components from which a game is made. Such abstraction is provided by Jesse Schell in the form of the "Elemental Tetrad" model (Schell, 2019).

3.1.1 The "Elemental Tetrad" Model

The "Elemental Tetrad" is an abstract model, which is designed to provide an understanding of a game by breaking it down into four main elements, which form the game: **aesthetics**, **mechanics**, **story**, and **technology** (Schell, 2019).

Elements of the Tetrad

Aesthetics consists of several aspects: sensory communication (typically visual and auditory; sometimes tactile; very rarely olfactory and gustatory), emotional sensations (joy, fear, anger, sadness, disgust), and more complex feelings (excitement, loneliness, nostalgia, ...). Aesthetics have the most direct influence on a player's experience.

Mechanics are rules, limitations, and objectives within the game. A combination of mechanics and their interactions with each other make the gameplay. Mechanics should be consistent and balanced: changing a single mechanic may have cascading effects on others.

Story can be described as a group of narrative elements, which provide a context for a game, and a sequence of events, which occur and unfold within the game.

Technology combines any tool, material, software, and hardware that is used in the process of the game's creation and allows it to function.

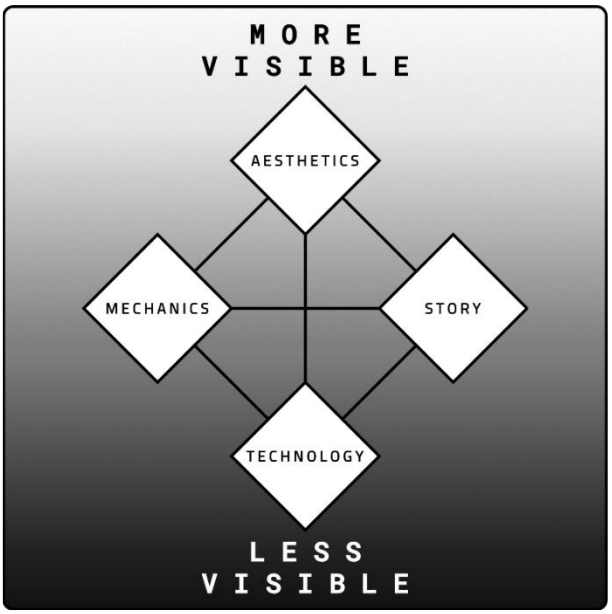


Figure 10: "Elemental Tetrad"; Source: (Schell, 2019)

Jesse Schell also explains that all elements have the same level of importance and that the diamond shape of the model represents a visibility gradient: aesthetics are the most visible element to the player, technology is the least visible and often hidden, while mechanics and story are positioned somewhere in between (Schell, 2019).

Relationships Between Elements

This model also illustrates relationships between aesthetics, mechanics, story, and technology:

- Aesthetics reinforce the narrative and ideas of the story.
- Aesthetics bring clarity and understanding to the mechanics.
- Mechanics move and progress the story.
- Mechanics strengthen immersion in the aesthetic and story.
- Story gives the mechanics meaning and sense.
- Story influences the pacing of aesthetics.
- Technology is a medium, where aesthetics, mechanics, and story exist.
- Technology imposes direct constraints onto the other elements.
- Aesthetics, mechanics, story, and technology make the game more entertaining, memorable, and impactful.

I interpret the main idea behind the "Elemental Tetrad" as an encouragement to use a holistic game design approach:

- Elements are interconnected and interdependent.
- Elements are equally important.
- Elements should exist in harmony and balance.
- A change in one element affects the others.
- A synergy of elements improves overall experience.

3.1.2 Definition of Abstract Components

Below I have provided a definition for abstract components based on my understanding and analysis of Jesse Schell's "Elemental Tetrad" model.

Abstract components can be described as the conceptual design foundation of a game. They are represented by **aesthetics**, **mechanics**, **story**, and **technology**. These elements shape the experience of a player and have a bidirectional relationship with the theme, genre, and gameplay. Theme, genre and gameplay define abstract components, while abstract components reinforce theme, genre and gameplay.

3.2 Solid Components

In order to convert design into realization, abstract components must be transformed into actual, interactable elements - **solid components**. Solid components are the way in which abstract components are manifested. Unfortunately, the relationship between abstract and solid components is rarely one-to-one, because, a single solid component is typically influenced by multiple abstract components simultaneously, due to their interconnected nature. This influence often depends on the game's theme, genre, and gameplay. Because of this complexity, strict categorizations, such as "UI equals aesthetics" or "combat system equals mechanics", tend to oversimplify the structure of game. To further illustrate these nuances, I have provided an example with one of the most frequently used game components - the button.

3.2.1 The "Button" Example

In this example, two button implementations are examined. Before comparison it is important to provide some context: each button is used in hypothetical adventure game set in a mysterious underground laboratory complex, from which the player must escape; the primary function of each button is to allow the player to return to the main menu; the game is developed by using a game engine, which provides built-in tools for creating basic UI (user interface) elements.

The "Return" Button

The first button represents a basic implementation. It uses a simple rectangular shape with a neutral grey background and displays the label "RETURN TO THE MAIN MENU".

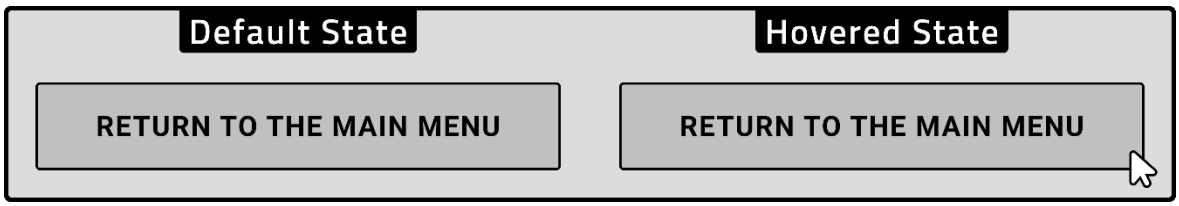


Figure 11: The "Return" button states; Source: Author.

The button is primarily defined by aesthetics: its minimalistic design offers no strong thematic indicators. However, even in this basic implementation, technology is present in the form of the UI toolkit of the engine, and mechanics are also involved: the click interaction results in action, which can be considered rule-based **behavior**.

The "Escape" Button

The second button represents more complex implementation. It features an icon, shadowing, and non-standard shape.

Figure 12: The "Escape" button states; Source: Author.

This version shows the influence of multiple abstract components:

- **Aesthetics** - more sophisticated visual design.
- **Mechanics** - unlike the first button, it additionally includes a hover interaction.
- **Technology** - requires additional graphical assets and scripting.
- **Story** - the word "ESCAPE" and door icon support the narrative context.

Conclusion

This example shows how even a very simple component can be shaped by multiple abstract influences simultaneously. Even though both buttons serve the same mechanical purpose, their overall contribution to the game is different.

In summary, **solid components** are the actual elements of the game that emerge from **abstract components**. The design and implementation of solid components are influenced by four factors: aesthetics, mechanics, story, and technology.

4 Artificial Intelligence

4.1 The Key Terms

4.1.1 Artificial Intelligence

Artificial intelligence (AI) can be described as a collection of tools, systems, methods, and technologies designed to perform complex tasks, which are typically done by humans. Artificial intelligence is developed to **"think"** and **"act"** rationally by imitating and simulating human cognitive processes such as reasoning, planning, communication, and learning (Stryker & Kavlakoglu, 2024; What Is Artificial Intelligence? | NASA, 2024).

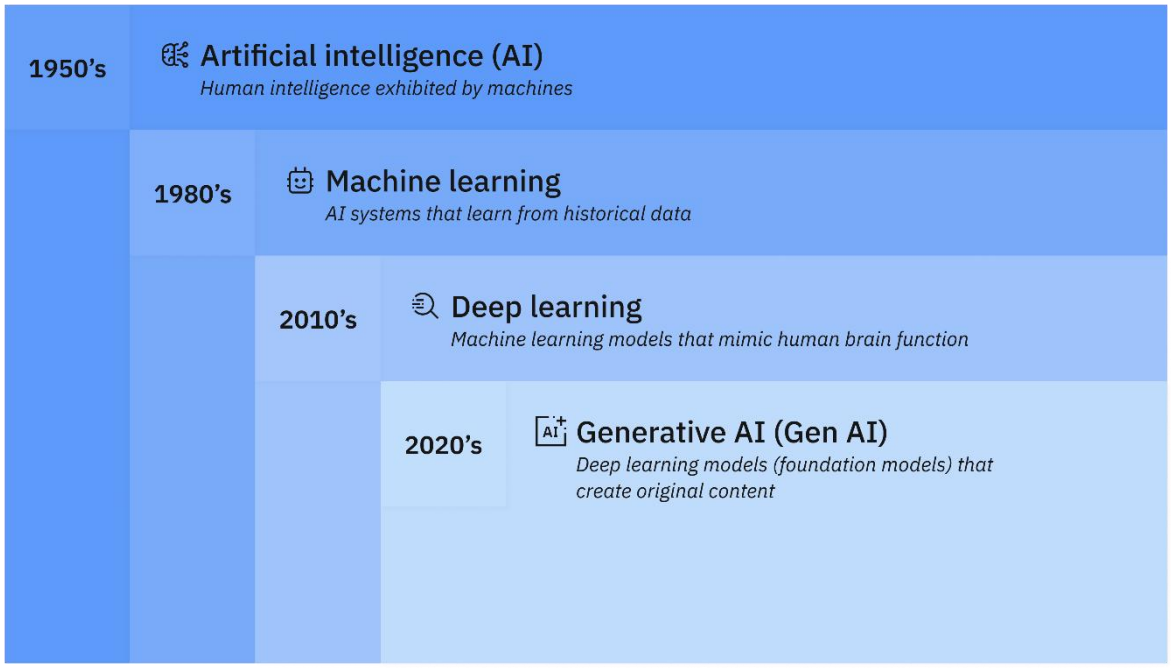


Figure 13: Relation between AI fields; Source: (Stryker & Kavlakoglu, 2024).

4.1.2 Symbolic AI

Symbolic AI (also known as "Classic AI" or "Good Old-Fashioned AI") is a subset of a**rtificial intelligence** that relies on explicitly defined symbols and rules to solve complex problems (GeeksforGeeks | What Is Symbolic AI?, 2024).

Typical examples of symbols include logical predicates such as **AND, OR, XOR, NOT,** and others. By combining these symbols, rules are created (often in the form of an "**IF-THEN-ELSE**" pattern), which are then used in algorithms for decision-making.

Thanks to its explicitly defined rules and symbols, symbolic AI often produces more transparent and explainable outputs. Therefore, it is easier for people to understand, inspect, and validate the results (Culbreath, 2024).

Unfortunately, symbolic AI has some limitations in scalability and adaptability, which have led to the development of other AI fields. However, symbolic AI still serves as a foundation for modern artificial intelligence by providing structured approaches to represent knowledge and perform clear logical reasoning (GeeksforGeeks | What Is Symbolic AI?, 2024).

4.1.3 Machine Learning

Machine learning (ML) is a subset of artificial intelligence that includes a wide range of techniques and algorithms that enable the creation of models by training computers to make classifications, generate predictions, or discover similarities across large datasets without being explicitly programmed for specific tasks (Stryker & Kavlakoglu, 2024; What Is Artificial Intelligence? | NASA, 2024).

There are many types of machine learning techniques and algorithms (Stryker & Kavlakoglu, 2024):

- Logistic regression.
- Linear regression.
- Decision trees.
- Random forest
- K-nearest neighbor.
- Clustering.
- And others.

There are also three main forms of machine learning:

- **Supervised learning** involves the use of labelled data. The goal is to train a model to classify data or predict outcomes accurately based on known inputs and outputs, which were defined by a human supervisor (Stryker & Kavlakoglu, 2024).
- **Unsupervised learning** involves the use of unlabeled and unstructured data. The goal is for models to make their own predictions about data without human intervention (Stryker & Kavlakoglu, 2024).
- **Semi-supervised learning** combines supervised and unsupervised learning by using both labelled and unlabeled data (Stryker & Kavlakoglu, 2024).

4.1.4 Neural Networks

Neural network – a machine learning technique to train models to process and analyze complex data. A neural network consists of interconnected layers of nodes. These layers and nodes work together to find patterns and relationships, modify data via series of transformations, and make predictions (Layers in ANN | GeeksforGeeks, 2025; Stryker & Kavlakoglu, 2024).

There are three primary types of layers (Layers in ANN | GeeksforGeeks, 2025):

- **Input layer** - receives raw data and passes it to the hidden layers.
- **Hidden layers** - the intermediate layers; perform most of the computations.
- **Output layer** - produces output predictions.

The structure of neural network is inspired by the structure of brain neurons (What Is Artificial Intelligence? | NASA, 2024).

4.1.5 Deep Learning

Deep learning is a subset of machine learning that uses multilayered neural networks, typically with hundreds of hidden layers, which are designed to simulate the complex decision-making processes of the human brain more closely (Stryker & Kavlakoglu, 2024).

4.1.6 Generative AI

Generative AI is a subset of deep learning models that can create complex multimedia content, such as text, images, video, or audio, based on a user's request (**prompt**) (Stryker & Kavlakoglu, 2024).

Rise of Generative AI

Generative AI has become a mainstream business tool in 2025, with 71% of companies reporting its use in at least **one** business function. This rapid adoption is largely driven by the rise of AI services such as ChatGPT, Claude, and Perplexity, which enable automation in areas like content creation, customer support, and digital assistants (Cardillo, 2025).

4.2 Artificial Intelligence in Game Development

This chapter explores the various applications of artificial intelligence in video games and game development. The chapter mostly relies on the works of **Tommy Thompson**.

Tommy Thompson is a game developer and consultant. He runs the "AI and Games" YouTube channel and similarly named website, both of which are dedicated to video games, game development and artificial intelligence. One of his main goals, as he describes it, is "to help the world better understand how AI makes better games and games make for better AI" (Thompson, n.d.).

Tommy Thompson offers two dimensions for understanding the use of AI in video games (Thompson, 2025). I have decided to refer to them as **Function** (what is the application of AI in the game) and **Purpose** (why AI is adopted in the project). According to Tommy Thompson (Thompson, 2025), the Function and Purpose views can be divided into more specific aspects:

- **Function**:
  - **AI that Plays** - 'plays' the game in some extent.
  - **AI that Creates** - creates 'content' for the game
  - **AI that Models** - models a property in the game space.
- **Purpose**:
  - **AI for Player Experience.**
  - **AI for Game Production.**

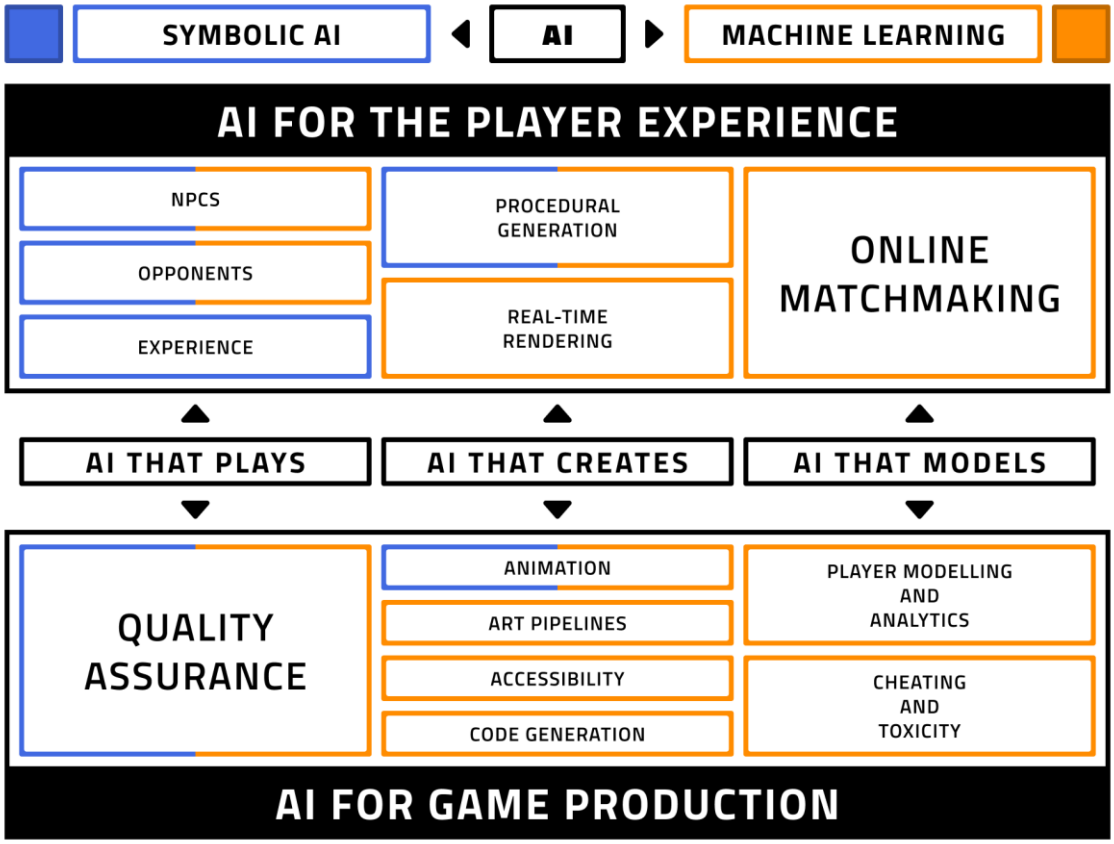


Figure 14: How AI is used in the video game industry; Source: (Thompson, 2025).

AI that Plays for Player Experience

As the name suggests, this category of AI actively participates during gameplay and is used to solve various tasks within the game, such as interacting with the player, controlling entities in the game, and manipulating in-game processes and overall game flow (Thompson, 2025).

Because these tasks require a specific approach, this has led to the appearance of a sub-discipline referred to as game AI, which includes various symbolic AI methods adapted to solve game design problems. This type of AI should be efficient; however, it does not need to be perfect in its execution, as its main task is to entertain the player (Thompson, 2025).

In addition, machine learning techniques are also used as "playing AI," although they are not as heavily adopted as symbolic AI techniques. An example of their use includes modelling and imitating the player's behavior (Thompson, 2025).

AI that Creates for Player Experience

The typical use cases of creative AI for the player experience are **procedural content generation** and **real-time rendering**.

Procedural content generation refers to the creation of content such as levels, characters, stories, and weapons via specialized algorithms. These algorithms intelligently apply predefined rules and processes to craft content during gameplay. The rules are usually implicitly defined within the design of the algorithm. To achieve variation in results, procedural generation often relies on pseudo-randomness (Thompson, 2025).

Real-time rendering focuses on improving the visual quality of a game as it is being played, often supported by AI-enabled hardware. For example, one of the real-time rendering techniques is called **Deep Learning Super Sampling** (DLSS), a process that enhances the overall graphical quality in real time by utilizing AI-driven upscaling (increasing the resolution of an image by adding more pixels) via the GPU (Thompson, 2025).

AI that Models for Player Experience

The usual application of modelling AI for the player experience is in **online matchmaking** systems. The main function of such AI is to model the perceived skill of players so they can be paired with or against players of relatively similar skill levels, helping to create more engaging and fair matches in games (Thompson, 2025).

AI That Plays for Production

AI also plays a significant role in quality assurance, particularly in **testing**. Testing is considered one of the most demanding aspects of game development. Bugs and issues can appear in various ways and are often difficult to replicate, as they may require a specific sequence of actions to trigger. To overcome this, AI is used to speed up and automate the detection of such problems (Thompson, 2025).

AI That Creates for Production

The **"AI That Creates for Production"** today is usually associated with **generative AI**, as it is one of the most trending areas in game development. Generative AI is essentially a form of procedural content generator, but instead of relying on explicitly coded rules or algorithms, it uses large datasets of existing content examples (Thompson, 2025). According to Tommy Thompson and Bernard Marr (Marr, 2024; Thompson, 2025) applications of this AI include:

- Generation of graphical assets.
- Automated modeling.
- Voice synthesis and modification.
- Generative music and sound effects.
- Generation of subtitles.
- Text-to-speech conversion
- Motion-matching techniques
- Texture upscaling.

Generative AI can be considered the least standardized area in game development, as it comes with **integration barriers**, **production challenges**, and **weak legal regulations** (Thompson, 2025).

AI That Models for Production

The first area of such AI application is **analytics**. This type of AI is used to collect, process, and interpret data on player behavior. Its purpose is to help developers understand how their audience interacts with the game, which parts of the game players like, and what problems they are experiencing (Thompson, 2025).

The second area where this AI is applied is in identifying **"bad actors"** - players, who exploit and abuse the game or behave in ways that harm other players and ruin their gaming experience. One frequent example of such behavior is commonly referred to as **toxicity**, which includes harassing other players through text or voice channels or inappropriate actions within the game space (Thompson, 2025).

4.3 The Survey

4.3.1 Purpose of the Survey

The aim of the conducted survey was to collect data about how artificial intelligence affects and interacts with people's lives, including such aspects as **work**, **education**, and **everyday activities**. The additional goals were to determine which AI tools are used more often, gather **opinions on AI-generated conten**t, and understand **concerns about AI** technology.

4.3.2 Structure and Methodology

The survey consisted of **twenty** questions and **five** interconnected sections:

- Personal information (questions 1–3).
- AI use at work (questions 4–7).
- AI use in study (questions 8–11).
- AI use in daily life (questions 12–16).
- Opinions, ethical views, and concerns (questions 17-20).

The survey mostly used quantitative questions with the **additional inclusion** of a few qualitative questions. Most of the questions were closed and semi-closed, except for questions 15 and 20.

The survey was made and distributed via **Google Forms** in three languages: Czech, English, and Russian. This was done to ensure a broader audience reach.

The survey used Likert scales and frequency scales to collect responses.

Likert scales were used to measure participants' opinions and attitudes toward the impact of AI. The scale included **five** ordered choices in the following pattern: very positive, positive, neutral, negative, and very negative.

Frequency scales were used to measure how often participants engaged with AI in work and study activities. Scales provided percentage-based response options.

Furthermore, filter choices (such as "Don't know" or "I can't say") were included as answers for the respondents when they were uncertain, ensuring greater accuracy and reliability of the collected data.

In multi-choice questions each answer was counted as individual "**vote**".

Additionally, a branching system was used to guide respondents to relevant questions based on their primary activities (work, education, or other). This approach was used to ensure greater attention and more honest answers.

This survey is also referred to as the **"AI survey".**

4.3.3 Audience

The total number of participants is **thirty-one**. The survey primarily included answers from individuals aged between 20–24 years. In terms of gender, the participant pool has an approximate 2:1 ratio between male and female respondents.

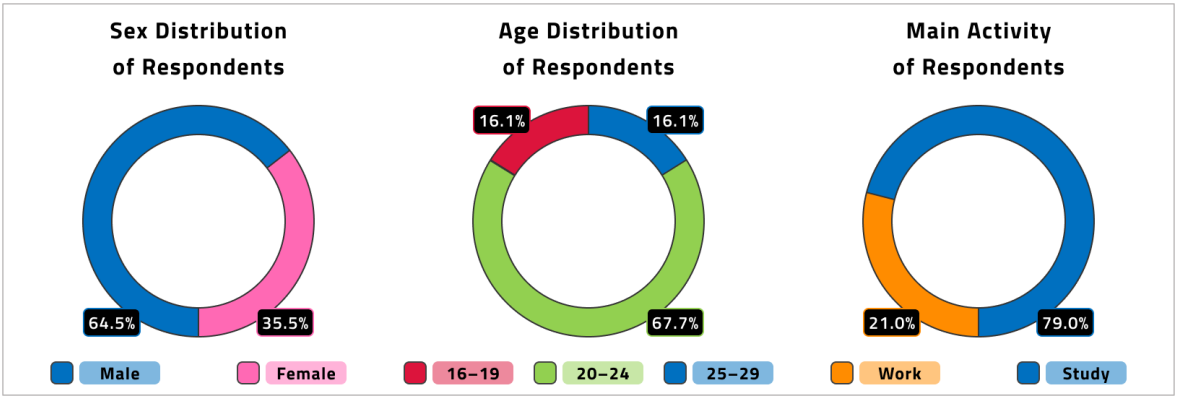


Figure 15: Demographic characteristics of respondents; Source: Author.

The majority of survey respondents reported a main activity related either to work or study, with some combining both to varying degrees. Four categories were identified:

- Only work.
- Only study.
- Mainly work, but also study.
- Mainly study, but also work.

In the following analysis, respondents were classified according to the context: those primarily working (including those who also study) were considered **workers** (29.0%), and those primarily studying (including those who also work) were considered **students** (71.0%). The classification also included a broad range of disciplines to capture the diversity of career and academic fields. The fields of main activity were:

- IT / Programming / Software Development.
- Administrative / Office Work.
- Hospitality / Customer Service.
- Business / Marketing / Sales.
- Law / Politics.
- Healthcare / Medicine.
- Arts / Design / Creative Work.
- Technical Field / Engineering.
- Education / Research.
- Sports.
- And others.

The detailed categorization of disciplines is presented below.

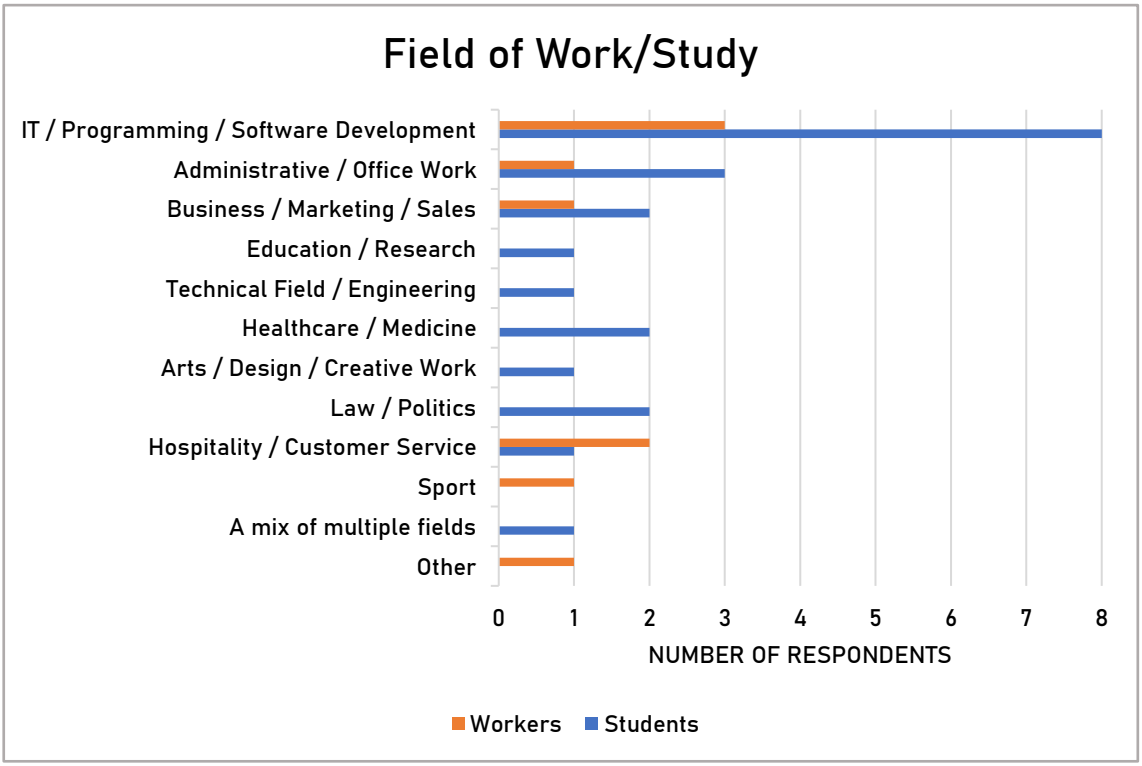


Figure 16: Overview of the respondents' main activity fields; Source: Author.

Quick summary

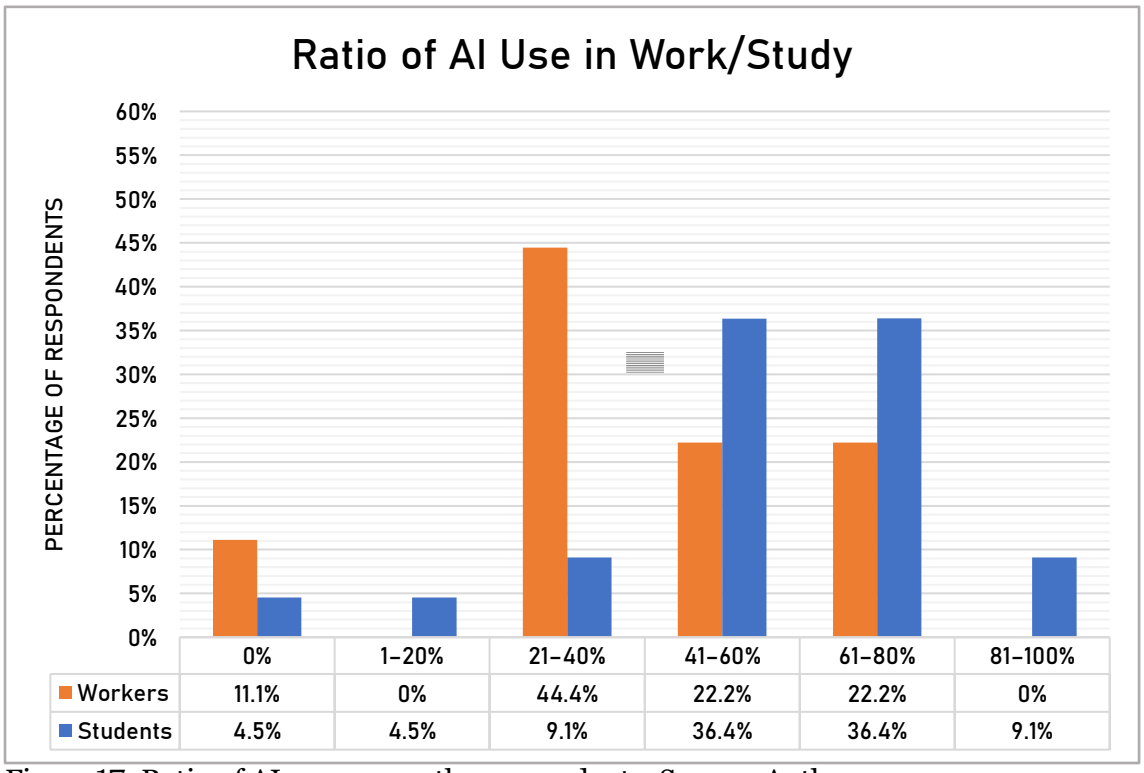
"IT / Programming / Software Development" group has the highest combined count (**eight** students and **three** workers).

"Administrative / Office Work", Business / Marketing / Sales" and "Hospitality / Customer Service" categories have moderate representation.

Aside from the "IT / Programming / Software Development" category, students are relatively evenly spread across various fields.

4.3.4 Collected Insights

In this chapter, I have presented the results gathered from the survey. The data has been organized and displayed to demonstrate findings. I have also provided my interpretations to offer additional context and insights for the responses.



	0%	1–20%	21–40%	41–60%	61–80%	81–100%
Workers	11.1%	0%	44.4%	22.2%	22.2%	0%
Students	4.5%	4.5%	9.1%	36.4%	36.4%	9.1%

Figure 17: Ratio of AI use among the respondents; Source: Author.

Interpretation (ratio of use)

Workers tend to use AI moderately, apart from those in the "IT / Programming / Software Development" group - likely due to the nature of the field.

Unexpectedly, a worker from the "Sport" category occasionally uses AI tools (21–40%). that AI is making its way into less "tech" fields.

Students demonstrate a higher overall usage of AI services due to increasing availability of AI. This raises some concerns about potential **over-reliance** on AI and **reduction of mental capabilities.**

One student from the "Healthcare / Medicine" field reported a high level (61–80%) of AI usage in their studies. This is particularly **concerning** due to the sensitive and rigorous aspects of medical education.

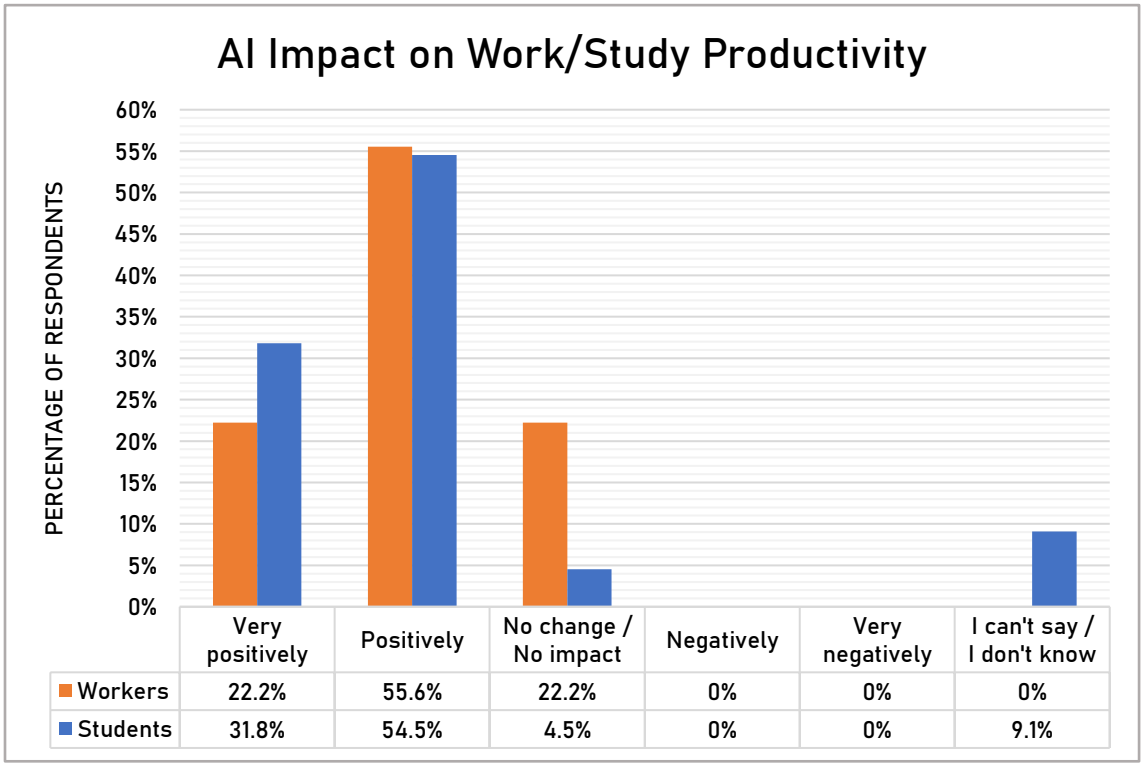


Figure 18: Impact of AI on respondents' productivity; Source: Author.

Interpretation (productivity)

The majority of the respondents have perceived the impact of AI on productivity as **positive** and **beneficial**.

Interestingly enough, none of the respondents (neither workers nor students) reported AI having a negative impact. The lack of negative responses may suggest an **overly optimistic view**; it is important to recognize that productivity gains do not necessarily equal improved quality. AI tools may enable users to complete tasks faster, but those tasks need to be verified and corrected to avoid potential errors, bias, or misinformation.

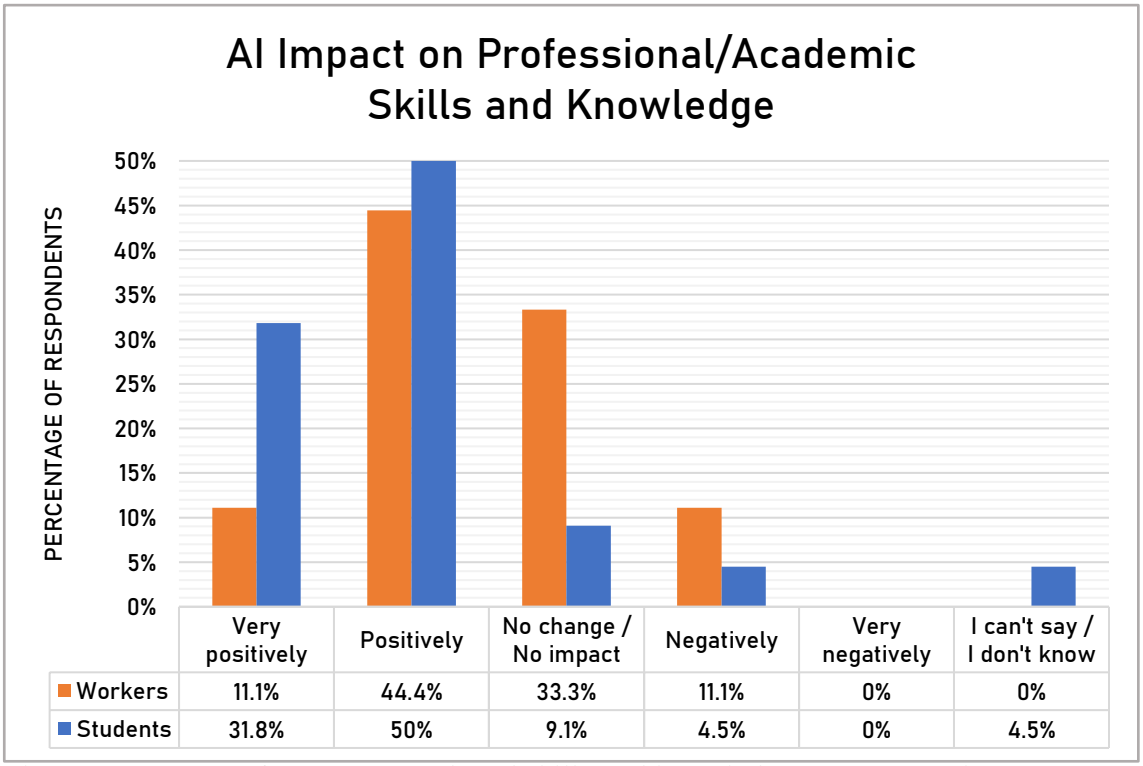


Figure 19: Impact of AI on respondents' skills and knowledge; Source: Author.

Interpretation (professional/academic skills and knowledge)

Most students and workers perceived AI's impact on their **skills** and **knowledge** as **positive**, similarly to the effect on **productivity**. This suggests that AI is more frequently used as a **source of information** and a **learning tool.**

However, some respondents (11.1% of workers and 4.5% of students) have reported a negative impact on their knowledge. Furthermore, these respondents have reported a moderate use of AI and were from tech-oriented fields ("IT / Programming / Software Development" and "Technical Field / Engineering"), where AI appeared first and is currently used the most. This suggests that **overreliance** on AI tools may lead to **skill degradation.**

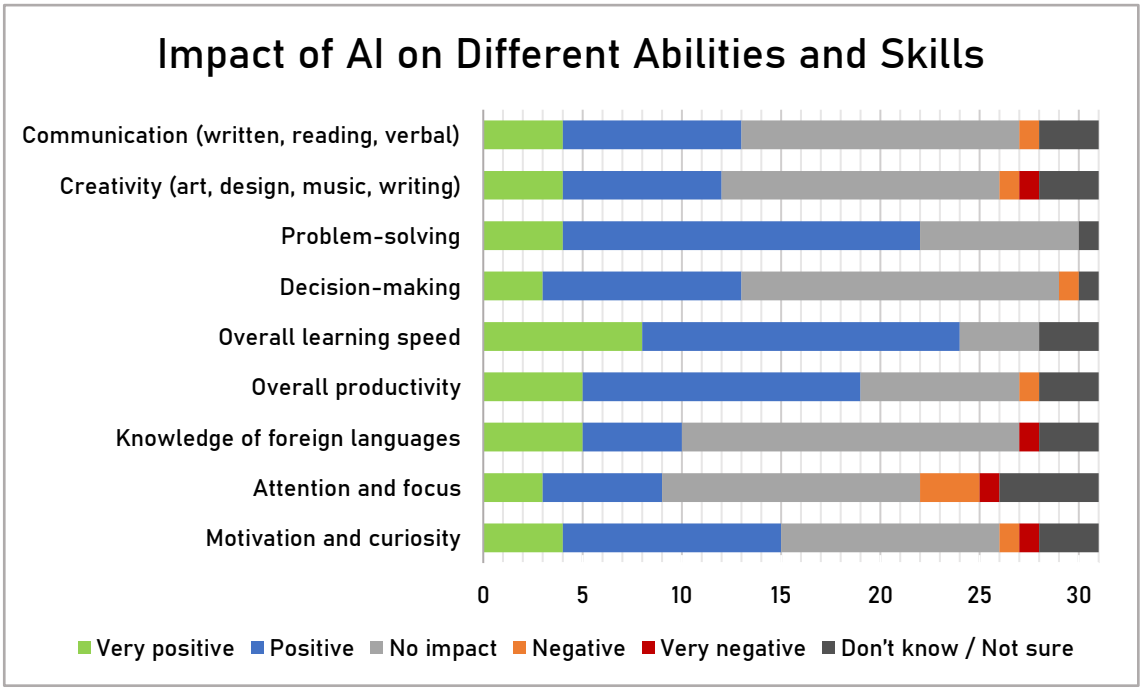


Figure 20: Impact of AI on different abilities and skills.

Interpretation (abilities and skills)

The respondents perceive AI having a **positive** impact on practical skills (namely learning speed and problem-solving), which indicates that AI is widely considered a **supporting tool**.

However, other skills received more mixed responses, indicating a moderate level of **uncertainty**. Additionally, several respondents reported a **negative** impact on **one** or **two** skills

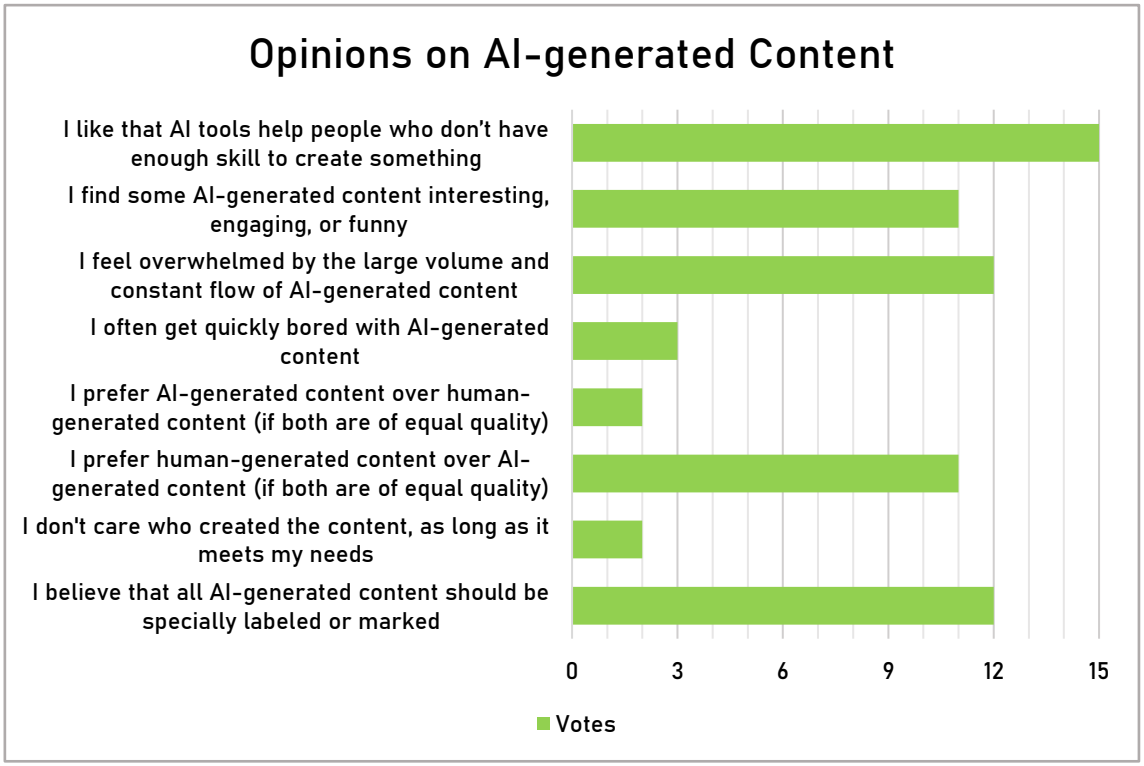


Figure 21: Opinions on AI-generated content; Source: Author.

Interpretation (AI-generated content)

"I like that AI tools help people who don't have enough skill to create something" (14 votes) - suggests strong support for making the AI more **accessible** to a wider audience to support **creativity** and **hobbies**.

"I believe that all AI-generated content should be specially labeled or marked" (12 votes) - suggests a desire for **transparency** and **ethical behavior** between content consumers and producers.

"I find some AI-generated content interesting, engaging, or funny" (10 votes) - showing that many users appreciate AI content as a **method of entertainment.**

"I feel overwhelmed by the large volume and constant flow of AI-generated content" (9 votes) - indicates that the number of AI-generated content is rapidly growing, resulting in **oversaturation**.

Additionally, **ten** respondents preferred human-generated content over AI-generated content when both are of equal quality, while only **two** preferred AI in such cases, suggesting a clear **preference for human-made content.**

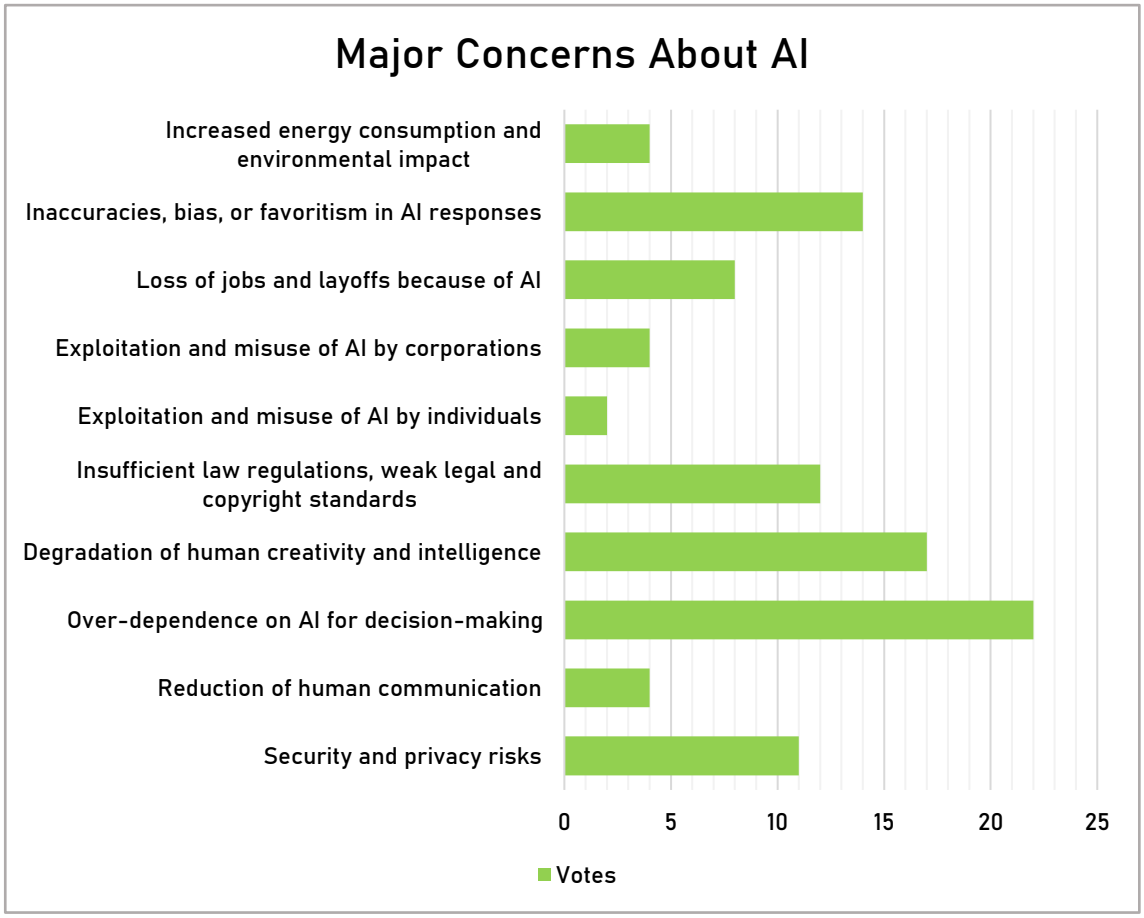


Figure 22: Major concerns about AI; Source: Author.

Interpretation (concerns)

Over-dependence on AI for decision-making is the top concern (22 votes), which suggests widespread fear that excessive reliance could **compromise human autonomy** in decision-making.

"Degradation of human creativity and intelligence" (17 votes) - the second most concerning issue; respondents think that AI may **worsen** instead of improving human **intelligence.**

Respondents are also concerned about **biases** and **favoritism** in AI responses (15 votes) and **insufficient legal regulations** (13 votes). These responses show a need for **accountability** and **fairness** from the companies that provide AI services.

4.3.5 Summary

Respondents had mixed feelings about AI-generated content. On one hand, many liked that AI helps people who aren't skilled enough with personal projects or hobbies. Several people also said that AI-generated content can be fun and interesting.

On the other hand, people expressed concern about when the real negative impact of AI might start to show. I am particularly worried about its possible effects on the entertainment industry. While most people still prefer human-made content, the speed at which AI can produce it far exceeds human capabilities.

A few respondents mentioned feeling overwhelmed by the constant stream of AI-generated content, and probably the situation soon will become worse, as currently there is a lack of clear laws requiring AI-generated content to be labeled. Without proper regulation, AI-generated content can easily flood platforms and displace human-made work, leaving creators without the recognition they deserve.

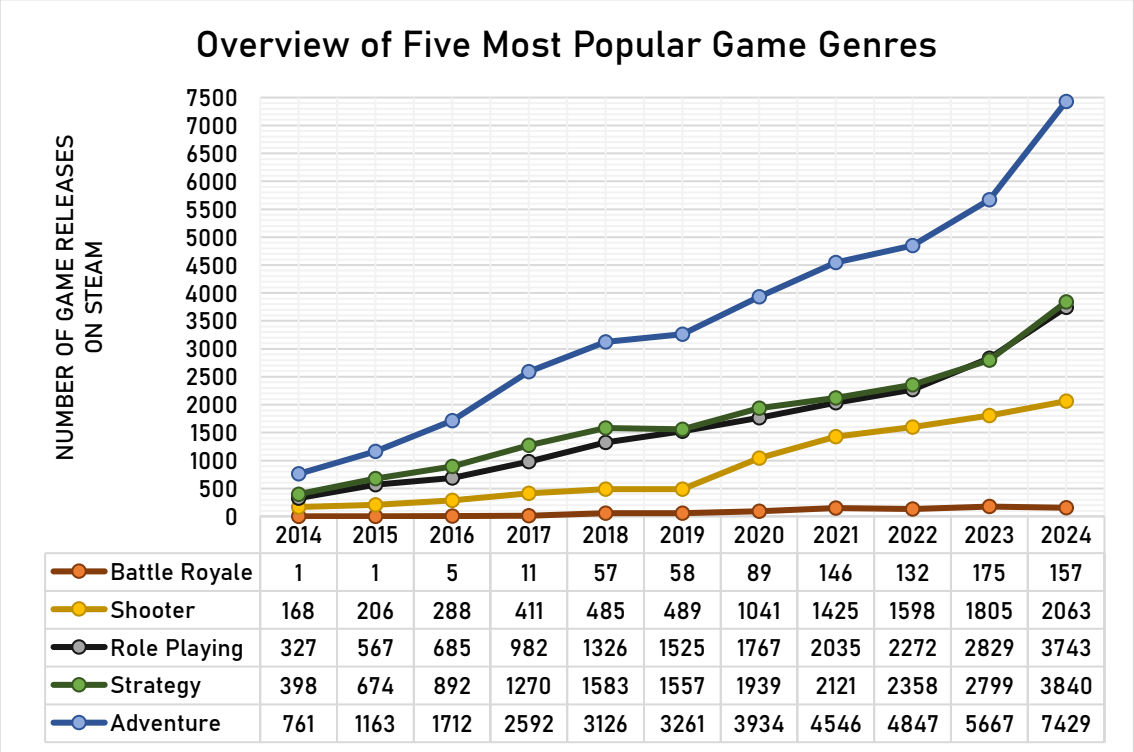
There's also another big concern: we still don't fully understand AI's long-term impact on our skills and intelligence. While it's true that AI can help us complete a wide range of tasks, the long-term effect remains unknown.

5 Market Analysis

5.1 Genre Analysis

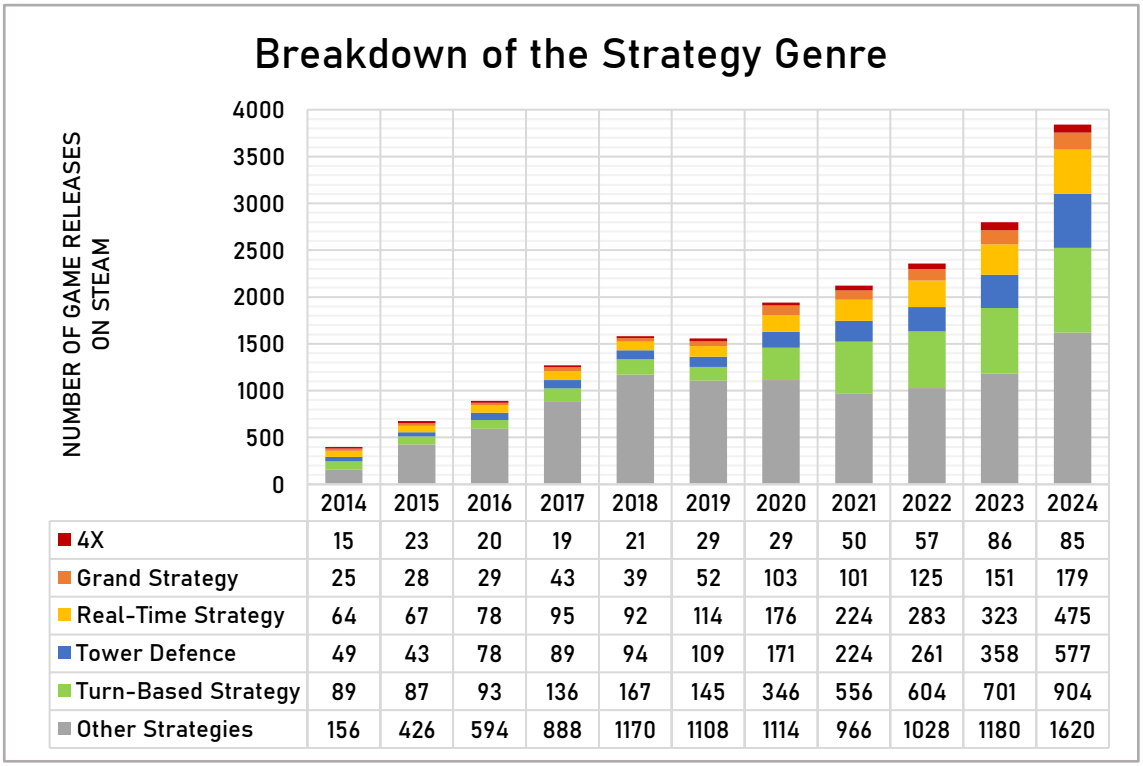
I have decided to choose the strategy genre for my project. According to Newzoo's market report data ("Newzoo's Global Games Market Report 2023", 2024), strategy ranks as the 5th most profitable genre on PC. However, Steam statistics reveal that strategy is also the second most saturated genre among the top-performing categories (along with the role-playing genre), signaling high competition. Therefore, I have decided to focus on the turn-based strategy subgenre, appealing to players who prefer more slow-paced and tactical gameplay.

Additionally, I have decided to incorporate mechanics from the card game genre, which, according to RocketBrush (Top Video Game Genres | RocketBrush, 2024), is also one of the best-performing game genres. This approach aims to make use of the profitability of the strategy genre while targeting a more specific and less saturated player base.



	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024
Battle Royale	1	1	5	11	57	58	89	146	132	175	157
Shooter	168	206	288	411	485	489	1041	1425	1598	1805	2063
Role Playing	327	567	685	982	1326	1525	1767	2035	2272	2829	3743
Strategy	398	674	892	1270	1583	1557	1939	2121	2358	2799	3840
Adventure	761	1163	1712	2592	3126	3261	3934	4546	4847	5667	7429

Figure 23: Number of games by most popular genres; Source: (SteamDB, 2025).



	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024
4X	15	23	20	19	21	29	29	50	57	86	85
Grand Strategy	25	28	29	43	39	52	103	101	125	151	179
Real-Time Strategy	64	67	78	95	92	114	176	224	283	323	475
Tower Defence	49	43	78	89	94	109	171	224	261	358	577
Turn-Based Strategy	89	87	93	136	167	145	346	556	604	701	904
Other Strategies	156	426	594	888	1170	1108	1114	966	1028	1180	1620

Figure 24: Breakdown of the strategy genre; Source: (SteamDB, 2025).

5.2 Game Analysis

To better define the requirements for my game prototype, I have decided to inspect several popular card-based strategy card games. Specifically, I analyzed **Slay the Spire**, **Monster Train**, **Hearthstone**, and **Inscryption**. By examining their core features, gameplay mechanics, visuals, and other aspects, I have identified the common elements. The information presented in this analysis was gathered through a combination of sources: visiting community-maintained wiki websites, watching gameplay videos on YouTube, and personally playing the games.

The table of games' features can be found below.

Table 6: Overview of games' features; Author: (*Hearthstone Wiki*, n.d.; *Inscryption Wiki*, n.d.; *Monster Train Wiki*, n.d.; *Slay the Spire Wiki*, n.d.; *Video Game Database | MobyGames*, n.d.; *Video Game Industry Statistics and Facts | GameStats*, n.d.).

Features	Slay the Spire	Monster Train	Hearthstone	Inscryption
Synopsis	Craft a unique deck, encounter bizarre creatures, discover relics of immense power, and Slay the Spire!	Set on a train to Hell, you will use tactical decision making to defend multiple vertical battlegrounds.	Play as a Warcraft hero using a customized deck to defeat opponents in duels.	Play a creepy card game in a cabin while unravelling a deeper mystery.
Genre	Card Game. Turn-Based Strategy. Dungeon Crawler. Fantasy.	Card Game. Turn-Based Strategy. Deckbuilding. Demons. Roguelike.	Online Multiplayer. Collectible Card Game (CCG). Turn-Based Strategy.	Story Rich. Strategy. Card Game. Horror. Surreal.
Gameplay Mechanics	Turn-based card combat with deckbuilding and strategic planning each run.	Multi-floor combat with deckbuilding, upgrades, and clan-based synergies.	Real-time matches with turn-based card play and hero powers.	Turn-based card combat with card sacrifices, fusion, and evolving mechanics.
Aesthetics	Stylized 2D hand-drawn visuals with abstract design.	Colourful, cartoonish 2.5D visuals.	Cartoonish fantasy style.	Dark folk-horror art with immersive UI and hand-drawn cards.
Key Mechanics	Deckbuilding. Relics. Card Synergies. And others...	Multi-floor battles. Relics. Clans. Card upgrades. And others...	Hero powers. Class-restricted cards.	Sacrifice system. Sigils. Deathcards.
Cost System	Three **energy** per turn. Upgrades present.	Three **embers** per turn. Upgrades present.	**Mana** system. Increases each turn from one to ten. Refreshed each turn.	**Blood** and **Bones** systems. Cards require sacrifices or bones gained from deaths.

Features	Slay the Spire	Monster Train	Hearthstone	Inscryption
Units	No allied units. Player plays cards directly against enemies.	Allied units (**monsters**) present, represented by cards.	Allied units (**minions**) present, represented by cards.	Allied units (**creatures**) present, represented by cards.
Battlefield	Single-lane layout. No positioning.	Vertical multi-floor grid. Four floors. Complex unit positioning system.	Single-lane board with left-to-right unit placement. Simple unit positioning system.	Four-lane grid. Complex unit positioning system.
Story	Light story elements.	Light story elements.	No story elements.	Intriguing horror and mystery story.
Monetization	One-time payment.	One-time payment. Paid DLC (downloadable content) preset.	Free-to-play. **EXTENSIVE** monetization via microtransactions.	One-time payment. Free DLC (downloadable content) present.
Tutorial	Present. Brief.	Present. Interactive.	Present. Interactive. Voiced.	Present. Interactive. Integrated into the story.

6 Preparation for Development

6.1 Measurements

During the development of prototypes, the following units have been used:

- The **man-hour (MH)** - a unit of measurement, which represents the amount of work performed by one person in one hour.
- The **man-day (MD)** - a unit of measurement, which represents the amount of work performed by one person in one working day, which typically consists of eight man-hours.

6.2 Tools

Godot is a free and open-source game engine for making both 2D and 3D games. It's developed by a community of volunteers. Godot lets users build games from scratch without needing to pay for a license. Godot also has extensive documentation, including tutorials and guides (Godot Engine | Godot Docs, n.d.).

Krita is a free, open-source painting program made for digital artists. It's designed to help artists create digital artworks from scratch, with a strong focus on illustration, concept art, matte painting, textures, comics, and animation (Welcome to the Krita | Krita Docs, n.d.).

Figma is a collaborative web application for interface design. The feature set of Figma focuses on user interface and user experience design, with an emphasis on real-time collaboration. Figma features various graphic editing tools (Figma | Wikipedia, 2025).

Google Sheets is a free web-based spreadsheet application. The application allows users to create and edit files online while collaborating with other users in real-time. Users can track their edits and view a revision history. The application is compatible with Microsoft Excel file formats (Google Sheets | Wikipedia, 2025).

ChatGPT is a generative AI chatbot, which is based on large language models and developed by the OpenAI company. It’s. ChatGPT is designed to generate human-like text in response to user prompts and is able to help with tasks like writing, coding, answering questions, and creating multimedia content. Due to its popularity, OpenAI currently operates the service on a freemium model (ChatGPT | Wikipedia, 2025).

Prototypes were developed using the **4.3 version** of Godot engine, the **GPT-4o model** of ChatGPT, and with the **Plus** subscription of **ChatGPT**.

6.3 Development Approaches

This chapter defines two distinct methodologies for game development: the **manual approach** and the **artificial approach**. Each methodology specifies the permitted processes, roles, and rules.

6.3.1 The Manual Approach

Game development is a highly complex process, requiring significant time, effort, a wide range of specialized skills, and a strong collaboration within the development team. As I have explained in previous chapters, development teams are typically divided into **eight** major role categories:

- Game Design and Creative Direction.
- Programming and Engineering.
- Visuals and Animation.
- Sound and Music.
- Production and Management.
- Support and Operations.
- Marketing and Public Relations.
- Quality Assurance and Testing.

Each category contains multiple specialized roles that work together to create a game. This approach focuses on three roles from the core development team:

- 2D Artist (referred to as **artist**).
- Game Designer (referred to as **designer**).
- Gameplay Programmer (referred to as **programmer**).

During the manual approach, I assumed all the listed roles. These roles represent the foundational set of skills in **design**, **code**, and **art**, which allow for a simulation of the game development process in a single-member team. Unfortunately, I was unable to include a role from the "Sound and Music" category due to time limitations and personal skill constraints.

Since the manual approach is focused on human skills and knowledge, the use of AI tools is **strongly discouraged**. Therefore, I should rely on official documentation, web guides, forum discussions, and tutorials if I need any information, guidelines, or help.

Responsibilities of Roles

Artist – responsible for the creation of all visual assets for the game, including wireframes for user interface layout, card artworks, and icons. The artist should try to maintain a consistent art style for all visual components as much as possible.

Designer – in charge of conceptualizing and outlining the gameplay systems, mechanics, and rules. This includes defining the interaction between the player and the game, balancing gameplay, setting win and lose conditions, and creating a tutorial.

Programmer – responsible for the code implementation of game mechanics and functions as defined by the designer. Additionally, the programmer should create the user interface based on the artist's wireframes and integrate other visual assets into a functional prototype.

6.3.2 The Artificial Approach

As I have mentioned in previous parts of my work, applications of AI in game development span a wide range of areas. For this project, I have focused on a specific area – "AI That Creates for Production". During the artificial approach, the AI (ChatGPT) has taken the role of the main game developer, combining the roles of **artist**, **designer,** and **programmer**.

In this approach, I have intentionally decided to reduce human involvement as much as possible to better examine the capabilities of AI. Unfortunately, while AI by itself can significantly accelerate the development process and fulfil a diverse set of tasks, it is not possible to fully eliminate human participation, as somebody still must input commands and instructions; then, after output is generated by AI, process and adapt the result for the game prototype.

This human participant is referred to as the **prompt writer.** The prompt writer interacts with the AI through structured prompts and provides limited oversight and corrections.

When acting as the prompt writer, I followed **general guidelines** provided by OpenAI for communicating with ChatGPT (How to Create a Good Prompt | OpenAI Help Center, n.d.; Best Practices for ChatGPT | OpenAI Help Center, n.d.):

- **Be clear and specific:** provide detailed, unambiguous prompts with enough context to generate accurate responses.
- **Approach the task iteratively**: start with a basic prompt and refine it according to the output; treat it as an ongoing conversation.
- **Use tone descriptors:** guide the model's style using terms like formal, casual, humorous, or professional.
- **Break down complex tasks:** Divide large or layered requests into smaller parts, highlighting key priorities.
- **Focus on goals:** clearly express your intent, even if phrasing isn't perfect.

However, there is one guideline I chose not to follow strictly: limiting ambiguity. I deliberately allowed a **reasonable degree of ambiguity**. My reasoning was this: if I provided overly detailed instructions, such as specifying exact functions and attributes for a class implementation, then both AI and I would be acting more as co-developers. Therefore, I decided to leave some space for interpretation to better observe the AI's creative and technical capabilities.

Additionally, I decided to use **keywords** in my prompts to support the guideline principles and to ease communication with ChatGPT. I introduced a total of **seven** keywords; their descriptions are provided in the table below.

Table 7: Keywords for communication with AI; Source: Author.

KEYWORD	DESCRIPTION
[TASK]	Specifies the objective/problem/question to be completed, solved, or answered.
[STEPS]	Specifies that a step-by-step algorithm or breakdown should be provided in the response.
[REMEMBER]	Specifies that context or information should be saved in memory.
[RULES]	Specifies the general rules or constraints that should be followed.
[NEED]	Specifies elements or approaches that should be included in the response.
[AVOID]	Specifies elements or approaches that should be excluded from the response.
[OPTIONAL]	Specifies elements or approaches that may be included, but are not mandatory.

6.3.3 Interventions

In the context of this work, an **intervention** is any action that disrupts the intended separation between the **artificial** and **manual** development approaches. In the artificial approach, an intervention refers to significant human involvement, particularly when specialized human skills are used. As for the manual approach, an intervention occurs when AI tools or services are introduced into the process, even partially.

The following actions **do not qualify** as interventions during the **artificial approach**:

- **Design**: correcting typographical mistakes.
- **Code**: writing additional comments or documentation.
- **Code**: correcting indentation and formatting.
- **Code**: renaming variables, functions, and other objects.
- **Code**: adding and positioning UI elements based on AI suggestions.
- **Code**: using the Godot editor to reference and assign scripts, nodes, and scenes.
- **Code**: using the Godot editor to manage file structure, assets, nodes and their attributes.

The following actions **do qualify** as interventions during the **artificial approach**:

- **Design**: designing game mechanics from scratch.
- **Design**: creating gameplay content (cards, units, spells) from scratch.
- **Design**: writing any informational text (tutorials, hits, descriptions).
- **Art**: creating graphical assets (artworks, wireframes, and icons).
- **Art**: modifying AI-generated graphical assets (artworks, wireframes, and icons).
- **Code**: adding missing variables, functions, classes and other objects.
- **Code**: correcting GDScript syntax.
- **Code**: correcting UI layout issues (overlaps, clipping, off-screen positioning).
- **Code**: integrating individual scripts and code parts to make the game function.
- **Code**: manually implementing missing functionality (in cases where AI was incapable of generating it).

Any use of AI **does qualify** as intervention during the **manual approach**.

Interventions should be recorded to better describe and evaluate the development process. I have introduced five levels of the **intervention ratio.** The aim of this metric is to help classify to which extent the intended development approach (artificial or manual) was compromised. The intervention ratio is calculated as the number of man-hours (MH) spent on interventions divided by the total number of man-hours required to complete the work.

$$Intervention\ Ratio = \frac{\sum Intervented\ MH}{\sum MH} * 100\%$$

Equation 1: Calculation of intervention ratio; Source: Author.

A table with classifications is shown below.

Table 8: Classifications of intervention ratio; Source: Author.

RATIO	CLASSIFICATION	DESCRIPTION	
		Manual Approach	***Artificial Approach***
0%	**None**	AI tools and services were not utilized. A human developer was fully responsible for all tasks.	There was no human involvement beyond basic prompting. The AI took care of all tasks.
1-10%	**Minimal**	Only a few tasks required assistance from AI. The development process was mostly handled by a human.	The AI-generated work received minimal human corrections or adjustments. The AI remained the main developer.
11-40%	**Minor**	The AI contributed in a limited but noticeable way. Human efforts were still dominant during the work.	Human involvement was occasionally needed to guide, complete, or refine AI output. AI still produced most of the game content.
41-70%	**Major**	The AI played a major supporting role. The human efforts heavily relied on AI assistance.	Human contribution became a significant part of the process. Many components of the game require manual work to complete or fix.
71-100%	**Critical**	The AI handled most of the work, with only minor input from a human.	Most of the work had to be completed, adjusted or redone by a human. The AI was unable to deliver usable results on its own.

6.4 Prototype Requirements

This section lists the key functional and design requirements for the game; it is based on the analysis of games' features. A moderate level of ambiguity was intentionally maintained when defining requirements to test the capabilities of generative AI during the development process.

1. The game should be a **single-player turn-based strategy card game**.
2. The game should be developed in the **Godot Engine** (version 4.3).
3. The game should be developed by using the **GDScript** programming language.
4. The game should be compatible with 64-bit Windows desktop OS.
5. The game should run in **1280x800** resolution mode.
6. The game should include **two** opposing sides: the player and the enemy sides.
7. Each side should have a maximum of **six** units deployed on the battlefield at the same time.
8. The game should have **three** player units.
9. The game should have **three** enemy units.
10. The game should have **five** abilities (such as spells, magic, or skills), which the player is able to use.
11. The units and abilities should be represented by a card.
12. The game should have a cost resource.
13. The card should require the cost resource to be played.
14. The cost resource should function as a limiter for the number of cards that can be played on a single turn.
15. All units must have at least the following attributes:
    a. Health
    b. Damage
16. Additionally, at least **one** unit should have an extra attribute.
17. The game should include a **win** and **lose** conditions.
18. The game should have an aesthetic style that fits the following description:
    a. Dark, gloomy, grim tone.
    b. Minimalist artwork design with no use of complex shading.
    c. Limited color palette.
    d. Charcoal drawing and/or Ink drawing.
    e. Fantasy and/or old fairy tale setting.
19. The following fonts, provided by the Google Fonts service, should be used:
    a. Acme.
    b. Pirata One.
20. The game should have basic information on how to play.
21. The game should include information about used fonts and the game engine.

It should be noted that each prototype was also deployed on **GitHub** as a **browser game** to facilitate more convenient playtesting.

7 Overview of the Development Approaches

7.1 Design

At the beginning, I decided to create a short synopsis for both game prototypes. A game synopsis is a concise summary that outlines the core elements of a game: story, setting, characters, themes, and gameplay (Freed, 2015). The synopsis served as a basis for designing all other game components, which needed to align with it.

Manual Approach Synopsis: Take command of a squad of monster-slayers and repel waves of minions sent by forces of evil.

Artificial Approach Synopsis: In a dying world of shadow and ash, summon your mighty champions and strike down the cursed, before you too are lost to the darkness. This is your last stand. Fight or be forgotten!

For the manual approach, the battlefield has a grid structure with six positions on each side: three frontline and three backline positions. This implies that I have to implement two attack types: melee and range. The AI decided to use a two-row layout, where the enemy units are positioned in the upper row and the player units in the lower row. A more detailed depiction of the layout is presented in the next chapter.

After that, units and spells have been designed. The table of their characteristics can be found below.

Table 9: Main gameplay elements; Source: Author.

Element Name	Game	Side	Type	Description and Additional Notes
Spearman	A	Player	Unit	Frontline. Cost (1). Health (7). Damage (2).
Musketeer	A	Player	Unit	Backline. Cost (2). Health (3). Damage (6).
Knight	A	Player	Unit	Frontline. Armored ("Armor" attribute reduces incoming damage). Cost (4). Health (10). Damage (4). Armor (2).
Skeleton Warrior	A	Enemy	Unit	Frontline, Armored ("Armor" attribute reduces incoming damage). Health (5). Damage (2). Armor (1).
Evil Mage	A	Enemy	Unit	Backline. Health (3). Damage (5).
Demon	A	Enemy	Unit	Backline, Armored ("Armor" attribute reduces incoming damage). Health (13). Damage (6). Armor (1).

Element Name	Game	Side	Type	Description and Additional Notes
Armor Up!	A	Player	Spell	Give unit 2 temporary Armor. Cost (2). Power (2).
Restoration	A	Player	Spell	Heal unit by 5 health. Cost (1). Power (5).
Improvise	A	Player	Spell	Discard hand, draw 3 cards, receive 3 orders. Cost (1). Power (3).
Arrow Rain	A	Player	Spell	Deal 2 damage to each enemy unit. Cost (1). Power (2).
Lightning Bolt	A	Player	Spell	Deal 10 damage. Cost (3). Power (10).
Templar of the Dawn	B	Player	Unit	Cost (4). Health (18). Damage (5).
Sanctum Mage	B	Player	Unit	Cost (3). Health (14). Damage (4).
Restless Sentinel	B	Player	Unit	Unyielding (survives 1 lethal attack). Cost (3). Health (22). Damage (3).
Cultist of Shadows	B	Enemy	Unit	Health (12). Damage (3).
Howling Wraith	B	Enemy	Unit	Health (8). Damage (6).
Cursed Knight	B	Enemy	Unit	Armored (reduces all incoming damage by 1). Health (20). Damage (5).
Ashes to Ashes	B	Player	Ability	Destroy an enemy unit with 3 or less health. Cost (1).
Blood Pact	B	Player	Ability	Sacrifice 1 friendly unit to draw 3 cards. Cost (2).
Lunar Offering	B	Player	Ability	Discard your hand, then draw that many new cards. Cost (2).
Solar Offering	B	Player	Ability	Gain 2 extra mana next turn. Cost (1).
Soulfire Surge	B	Player	Ability	Deal 5 damage to all enemies. Cost (4).

7.2 Art

7.2.1 Wireframes

Before developing the user interface for both prototypes, I have decided to first create wireframes of the main screen and the card for each prototype.

A wireframe is a basic and simplified visual representation of an application. It focuses on blueprinting the structure, layout, and functionality of the application's design. A wireframe usually uses limited colors, minimal typography, and primitive shapes, and it intentionally omits complex and detailed graphical elements (What Is Wireframing?, n.d.).

Manually Created Wireframes

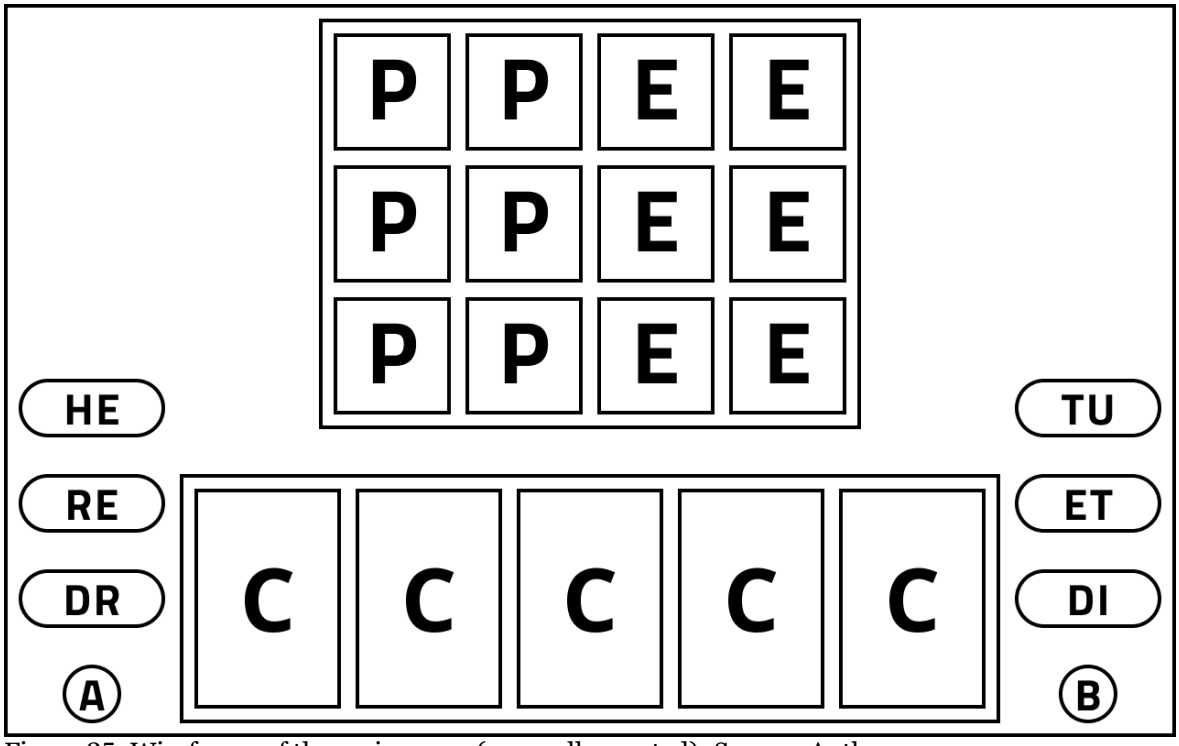


Figure 25: Wireframe of the main scene (manually created); Source: Author.

This wireframe represents the main gameplay scene of prototype A. The scene is divided into four primary sections: the **battlefield**, the **hand**, the **left controls** and the **right controls**. This layout is designed to keep essential gameplay information visible while maintaining a simple and symmetrical user interface.

- Battlefield:
  - **P** – represents player units positioned on the left side of the battlefield (the left Ps are backline units, and the right Ps are the frontline units).
  - **E** – represents enemy units positioned on the right side of the battlefield (the right Es are backline units, and the left Es are the frontline units).
- Hand:
  - **C** - represents cards currently in the player's hand that can be played.

- Left controls:
  - **HE** – Help button, opens the instructions menu.
  - **RE** – Resource indicator, displays the current resource amount (orders).
  - **DR** – Draw button, allows viewing of cards in the draw group.
  - **A** – Draw indicator shows how many cards are left in the draw group.
- Right controls:
  - **TU** – Turn indicator, shows the number of turns passed since the game have started.
  - **ET** – End turn button, ends the current turn.
  - **DI** – Discard button, allows viewing of cards in the discard group.
  - **B** – Draw indicator, shows how many cards are left in the draw group.

The main scene wireframe served as a basis for positioning and programming of the UI; and additionally, was extended during the process of development.

After completing the main scene wireframe, I continued the process by designing a layout for the card component. During this stage, I encountered the following challenge: the card information needed to be displayed in a way that was both clear and easy to understand, without covering too much of the card's artwork. To solve this, I positioned the card's description and name below the artwork and arranged the attribute icons and values in a vertical box layout along the left side of the card, thus keeping instructional clarity while preserving space for the artwork to remain visible. It also was later used as a basis for a layout of the unit component.

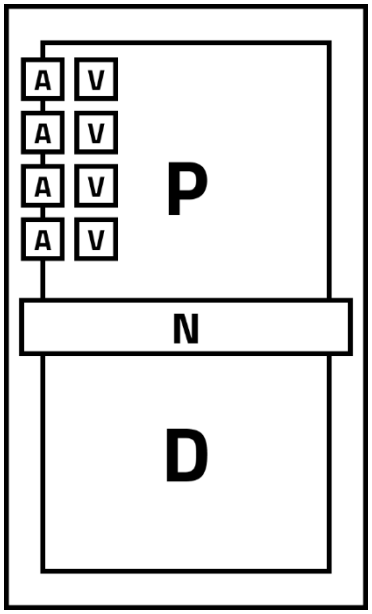


Figure 26: Wireframe of the card (manually created); Source: Author.

Card wireframe structure:

- **P** – Picture (artwork).
- **N** – Name of the card.
- **D** – Description (informational text).
- **A** – Attribute icon (graphical representation of an attribute).
- **V** – Attribute value (numerical representation of an attribute).

Artificially Created Wireframes

During the creation of the wireframe, the AI **struggled significantly**, especially when generating the main scene wireframe. All available iteration attempts were used for generating the main scene layout; however, the AI was unable to produce an acceptable result. The generated wireframes contained several flaws, such as cropping, multiple layout elements missing, and overlapping. Manual intervention was required. However, I decided to give the AI one last chance by asking it to generate a text-based representation of the main scene layout, based on the best graphical iteration. To my surprise, the AI produced a decent text-based wireframe of the main scene.

MAIN SCENE WIREFRAME

```
+-------------------------------------------------------------------------------+
|                          Enemy Spawner: [2 Remaining]                         |
|-------------------------------------------------------------------------------|
|                         Enemy Units (Max 6 Slots)                             |
|    [E1]     [E2]     [E3]     [E4]     [E5]     [E6]                        |
|-------------------------------------------------------------------------------|
|============================== BATTLEFIELD LINE ===============================|
|-------------------------------------------------------------------------------|
|    [P1]     [P2]     [P3]     [P4]     [P5]     [P6]                        |
|                          Player Units (Max 6 Slots)                           |
|-------------------------------------------------------------------------------|
| Help   [Deck: 12]            [Mana: 5 / 10]            [Discard: 3]  End Turn|
|-------------------------------------------------------------------------------|
|                        Player Hand (Max 7 Cards)                              |
|  [Card 1]  [Card 2]  [Card 3]  [Card 4]  [Card 5]  [Card 6]  [Card 7]        |
+-------------------------------------------------------------------------------+
```

Figure 27: Text-based wireframe of the main scene (artificially created); Source: Author.

The text-based version allowed for much faster manual creation of the wireframe, as the UI elements were already positioned reasonably well. I only needed to make several adjustments, rather than building the layout entirely from scratch. This significantly reduced the amount of effort I needed to put in. The illustration of the most successful AI-generated iteration (the **ninth** iteration) is shown below alongside the final manually adjusted wireframe.

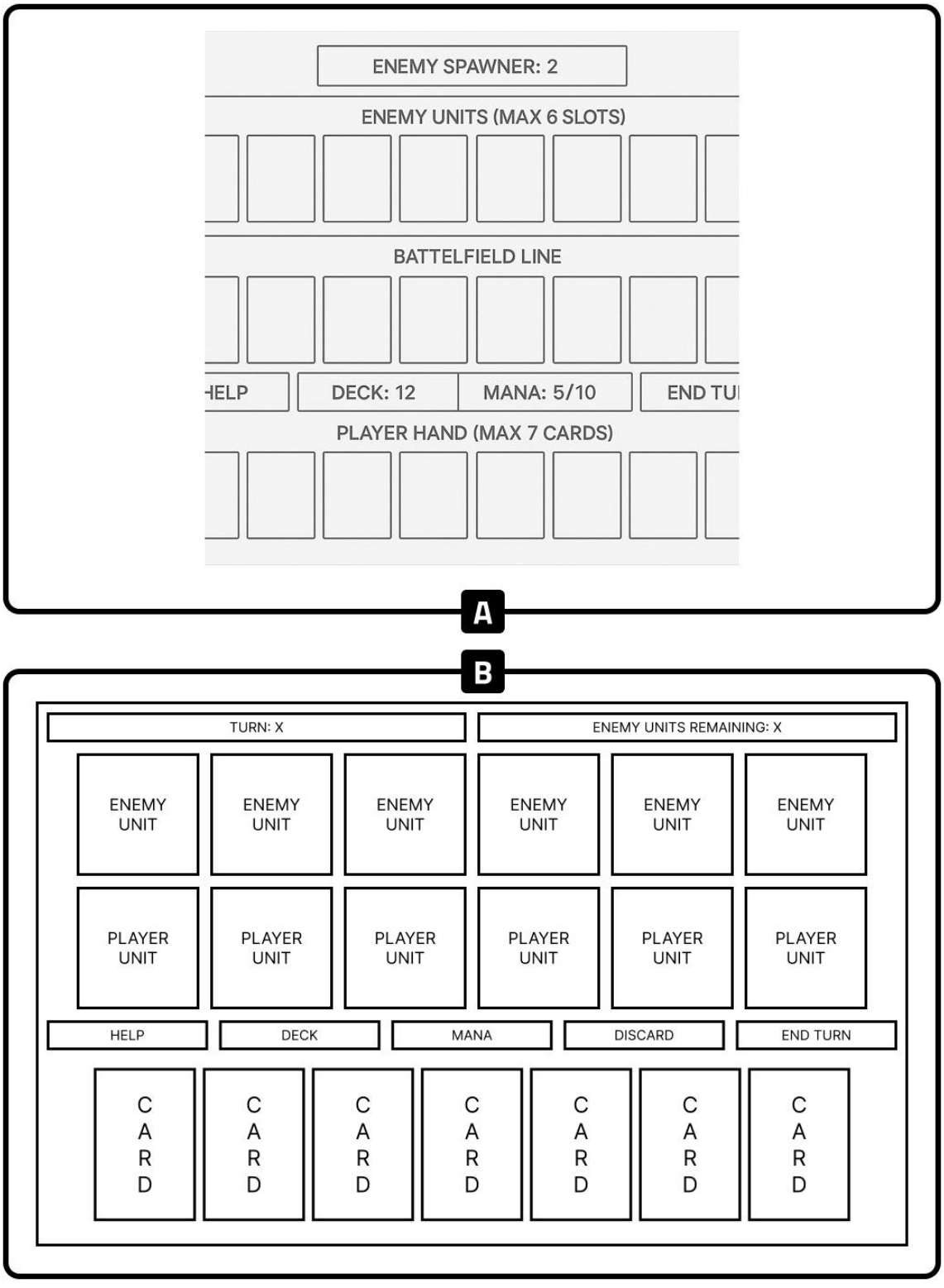


Figure 28: (A) Wireframe of the main scene (artificially created); Source: Author.

Figure 29: (B) Improved version of the artificially created wireframe (manually created); Source: Author.

For the card layout, I decided to generate a text-based version first. Once I generated the clearest text-based wireframe, I asked the AI to create a graphical representation of the text version. With this approach, the AI has achieved relatively good and sufficient results without needing my intervention.

CARD WIREFRAME

```
+-------------------------------------------------+
| COST: [X]                  TYPE: [Unit/Ability] |
|-------------------------------------------------|
| NAME: [Card Name]                               |
|-------------------------------------------------|
|               [Card Art Placeholder]            |
|          (Symbol / ASCII / Icon space)          |
|-------------------------------------------------|
| EFFECTS / STATS:                                |
|  🗡️ Damage: [X]      ❤️ HP: [Y]   (if Unit)     |
|   💥 Effect: [Short effect description] 💥      |
|     (e.g., "Burns enemy for 2 turns.")          |
+-------------------------------------------------+
```

Figure 30: Text-based wireframe of the card (artificially created); Source: Author.

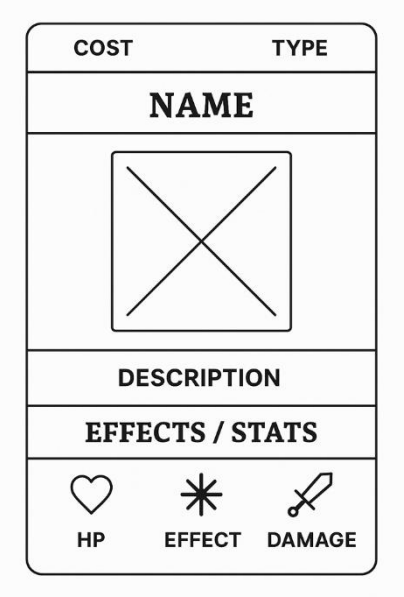


Figure 31: Wireframe of the card scene (artificially created); Source: Author.

7.2.2 Art Style

The Manual Approach

Finding the right references is an important step when developing an art style manually. These references don't need to be perfect matches of the target style, but they should serve as inspiration and guidelines.

I have chosen the minimalistic flat design of standard playing cards as my starting point, then explored for additional references that aligned more closely with the required aesthetics. I decided to take the overall design of the cards from the **Inscryption** game and the look of heroes from the **Darkest Dungeon** game. Using these references as my main source of inspiration, I began creating my own card artwork.

The collected references and the final illustrations are shown below.

Figure 32: (A) Standard playing cards, Source: (*Standard 52-Card Deck | Wikipedia*, 2025).

Figure 33: (B) Creature cards; Source: (*Cards | Inscryption Wiki*, n.d.).

Figure 34: (C) Character sprites; Source: (*Heroes | Darkest Dungeon Wiki*, n.d.).

Figure 35: (D) Manually made card art; Source: Author.

The Artificial Approach

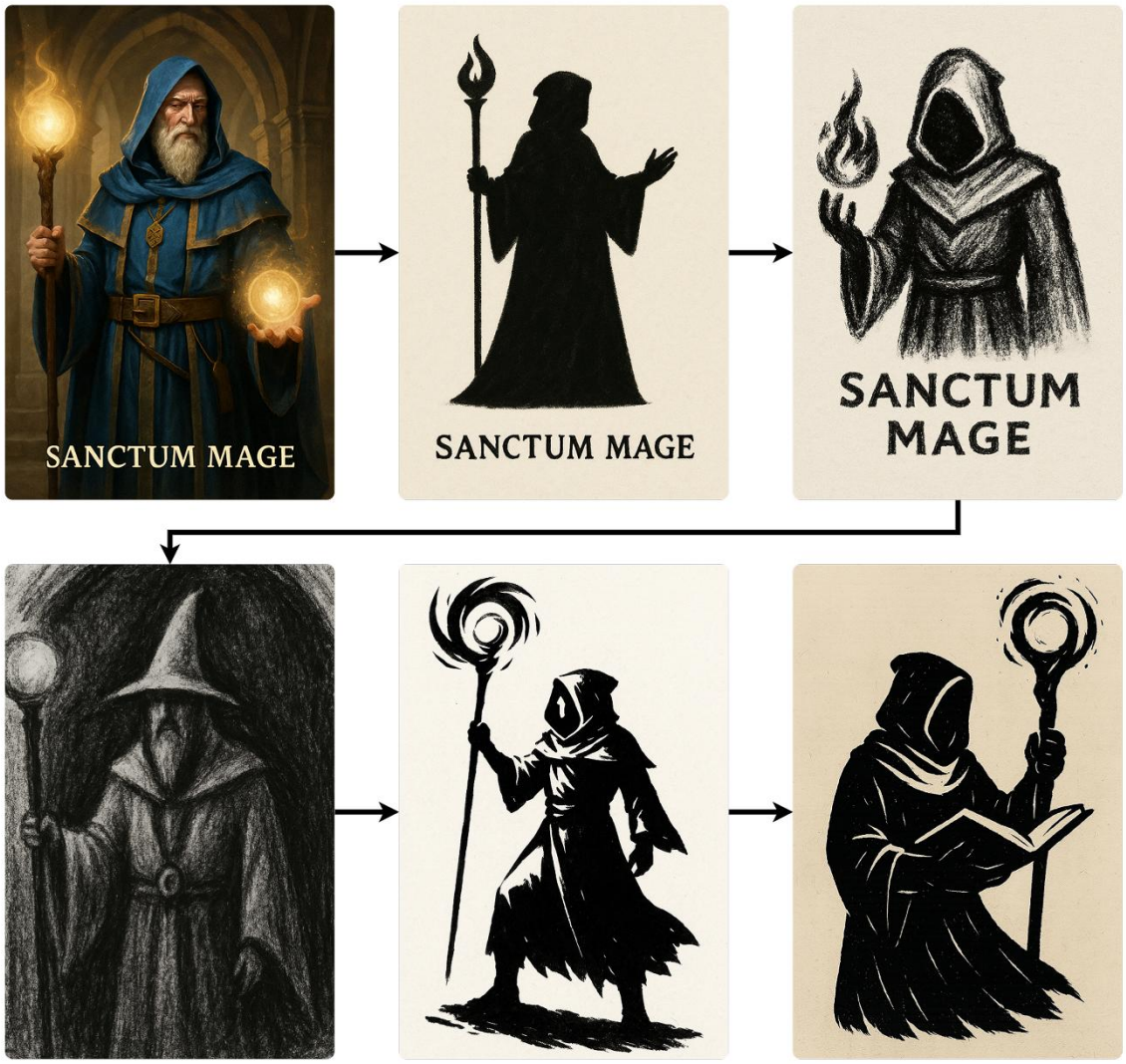


Figure 36: Evolution of the AI art style (artwork); Source: Author.

Creating an artwork with AI is an **iterative process** requiring constant prompt refinement and visual evaluation, as the outputs, which are produced by AI, must correspond to an overall art style of the game.

The "evolution" of the "Sanctum Mage" unit demonstrates how each generation moved the visuals closer to the target style. After multiple iterations, the prompt became more descriptive, which allowed to keep crucial details and remove unwanted parts.

After achieving the targeted aesthetics, the AI what told to remember the art style. During the entire process of developing art assets, the AI **was able to provide consistent results**, although with some **artifacts.** It should also be noted that all the artworks were based on a **synopsis** and **descriptions** that were generated during the **design phase**.

Additionally, I have deliberately decided not to use my art assets as a basis for content generation, as it would be too easy for AI to imitate my skill level and art style. Instead, I tried to make the AI follow such an art style that would be **comparable** to my art style but **not too similar**.

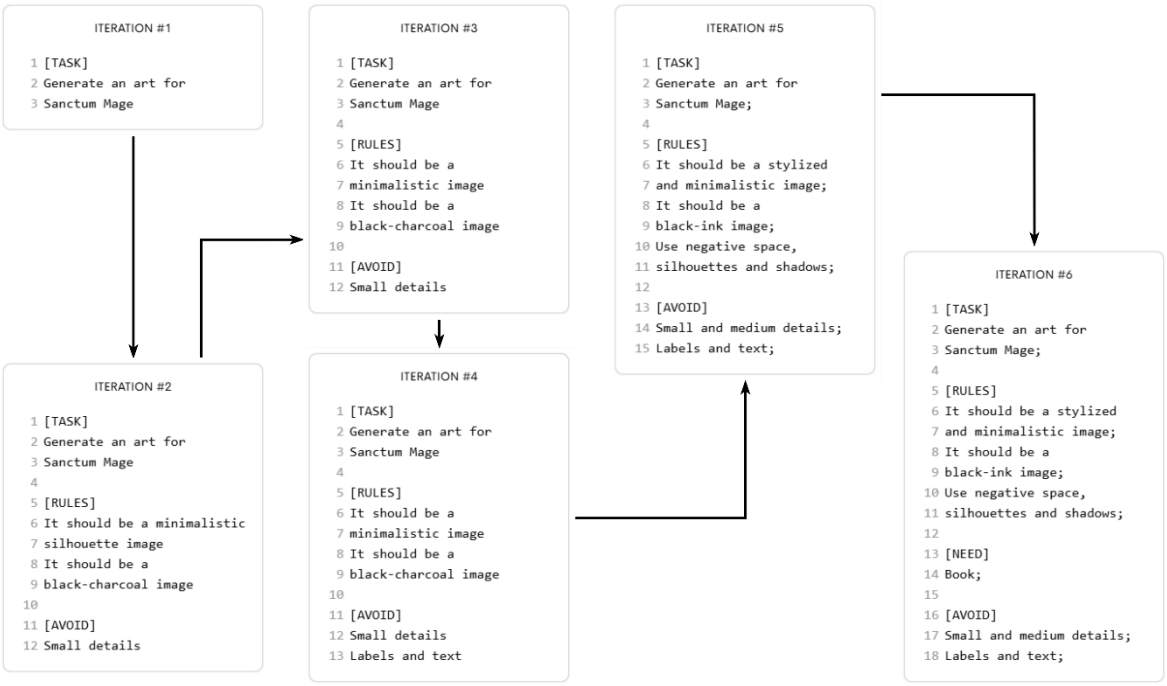


Figure 37: Evolution of the AI art style (prompt); Source: Author.

7.3 Code

7.3.1 Game Architecture

The architecture of software can be described as a collection of methods that define the fundamental structure of an application. In its core, architecture defines how individual components are organized within a project and how they interact with each other (Fundamentals of SW Architecture | GeeksforGeeks, 2022).

Composition and Inheritance

One of the main tasks during the development of both game prototypes was to determine how to represent cards, units, and abilities. Each prototype used two different methods to achieve this goal: **composition** and **inheritance**.

Inheritance is a common technique for structuring code within an application. Its main idea is to create a base class that represents a general object; this base class is then reused to define derived class, a more specialized version of the base class (Bill Wagner, 2022). Inheritance expresses an "is-a" relationship between the base class and the derived class (Shvets, 2021). For example, "Fruit" could serve as a base class, and "Apple" could be a derived class.

Composition is another common technique for structuring code; however, its main idea is to create complex objects by combining simpler objects. In Composition, objects are related via a "has-a" relationship; basically, one object has a reference to another. Another key feature of Composition is that the main object controls the life cycle of the contained object. Typically, the contained object cannot exist without the main object (Shvets, 2021). For example, an "Apple" has a "Seed" - it could be said that the "Apple" contains nutrients for the "Seed", and without the "Apple", the Seed cannot grow.

Examples of Using Composition and Inheritance

During the **manual approach**, I decided to represent card data in the game by using a **composition** technique. It allowed for a more modular and flexible game structure. As for the **artificial approach**, the AI used an **inheritance** technique for card data representation.

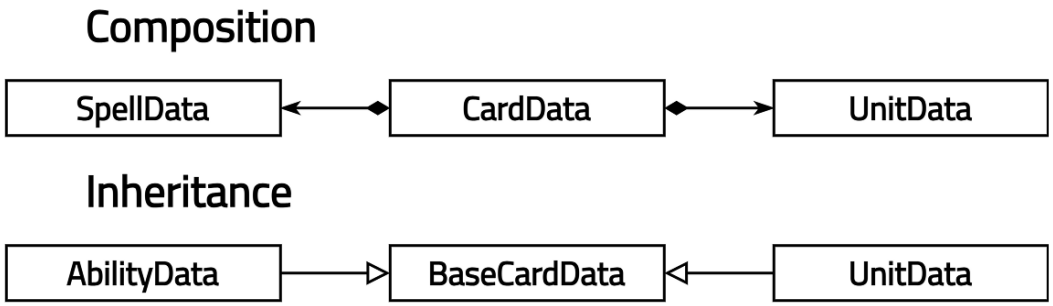


Figure 38: Examples of composition and inheritance relations in the prototypes; Source: Author.

In **prototype A**, card data is implemented by a class named "CardData". This class has an attribute named "card_type", which defines whether the card should contain a "UnitData" or a "SpellData" object. During the gameplay, one of these objects is created and referenced via an attribute called "linked_data_reference".

card_data.gd

```
"""
~ CardData ~
Represents a card's data.
"""

# Imports
const CardTypeEnum = ResourceLocator.CardTypeEnum
const SignalfulDualProperty = ResourceLocator.SignalfulDualProperty

class CardData:
    # Attributes
    var card_id: int
    var name: String
    var cost: SignalfulDualProperty
    var desc: String
    var card_type: CardTypeEnum
    # A card should be linked to another data object, which corresponds to card_type
    var linked_data_id: int
    var linked_data_reference: Variant

    # Initializes the card data object
    func _init( card_id: int,
                name: String,
                cost: SignalfulDualProperty,
                desc: String,
                card_type: int,
                linked_data_id: int
                ):
        self.card_id = card_id
        self.name = name
        self.cost = cost
        self.desc = desc
        self.card_type = card_type
        self.linked_data_id = linked_data_id
        create_linked_data(self.linked_data_id)

    # Returns a duplicate of a CardData
    func duplicate() -> CardData:
        return CardData.new(card_id, name, cost.duplicate(), desc, card_type, linked_data_id)

    # Creates and links a new UnitData or SpellData
    func create_linked_data(data_id: int):
        if card_type == CardTypeEnum.UNIT:
            linked_data_reference = DataManager.get_unit_data(data_id).duplicate()
        elif card_type == CardTypeEnum.SPELL:
            linked_data_reference = DataManager.get_spell_data(data_id).duplicate()

    # Clear reference
    func clear_linked_data():
        linked_data_reference = null
```

Figure 39: Code example from prototype A (card_data.gd); Source: Author.

It should be noted that, technically, “CardData” has a composition relationship only with “UnitData,” which represents the player's unit, while the enemy's “UnitData” has a composition relationship with the “CombatController” game node instead.

In **prototype B**, cards are implemented by a class named "BaseCardData". It defines properties that are shared between the "AbilityData" and "UnitData" classes.

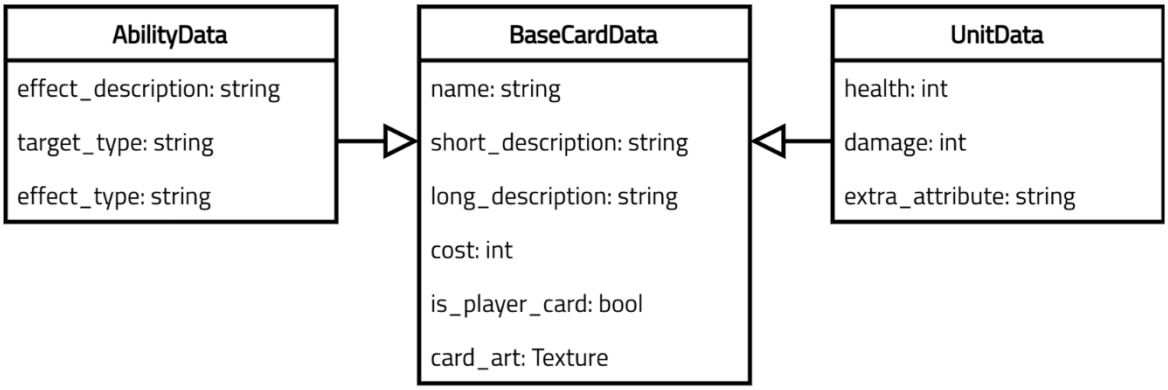


Figure 40: Relations between the classes in prototype B; Source: Author.

It is important to note that this is a quite simple implementation of inheritance. If the functionality of "BaseCardData" were to expand - for example, a new behavior was added it would be necessary to check the compatibility with the "AbilityData" and "UnitData" classes, as they are tightly connected to the base class.

Overview of Composition and Inheritance

Both composition and inheritance are fundamental techniques in programming. One should be prioritized over the other depending on the project's structure, logic, and problems which have to be solved.

- Inheritance should be used when a clear hierarchical relationship exists and when extension of base functionality is needed.
- Composition should be used when a flexible design is required, prioritizing smaller, reusable, and modular structures.

After finishing the work on **prototype B**, I realized that I could have solved a nuance with the "UnitData" class in **prototype A** by using inheritance: a better solution would be to implement the "BaseUnitData" class, then two derived classes, "PlayerUnitData" and "EnemyUnitData," and use them according to the roles.

7.3.2 Data Organization

Data organization is the process of arranging, structuring, and categorizing data in a systematic way to make it easier to access, use, and analyze. It usually involves transforming information into specific formats by different storage systems such as tables, databases, or spreadsheets (Data Organization | GeeksforGeeks, 2024).

In both prototypes, card, unit and spell information are stored separately from the gameplay logic. This allows easier code and data management. However, each prototype uses a different method for organizing external data.

Data Organization in the Manual Prototype

In the **manual prototype**, data is stored externally using tables in Google Sheets. Cards, units, and spells each have their respective tables. These tables are first exported as CSV (Comma-Separated Values) files, after which the files are read into the game at the start of the application through a CSV reader system.

It should be noted that not all units are required to have a respective card, as aforementioned enemy units exist.

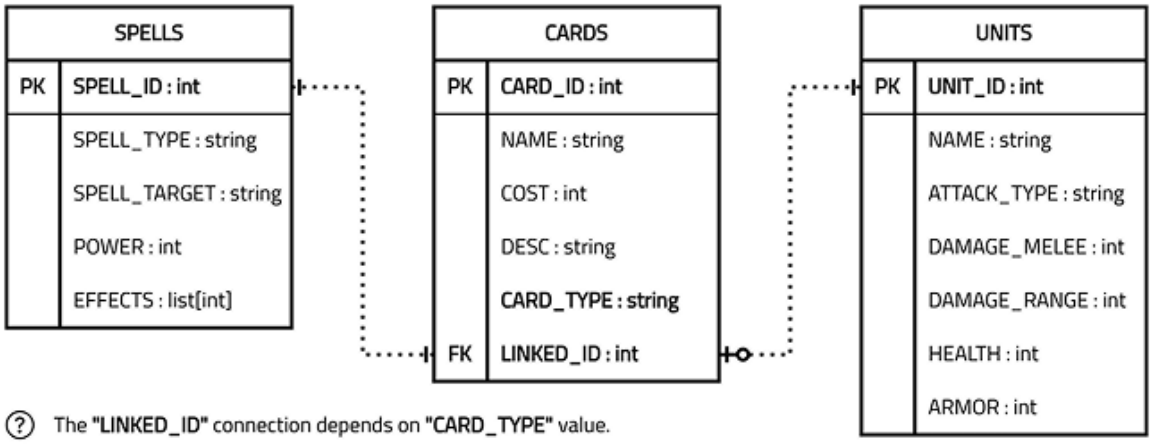


Figure 41: Relations between the data tables from prototype A; Source: Author.

Currently, there are **three** separate reading systems for units, cards, and spells. Although each system handles a different type of data, they all function similarly: the CSV text is first read as a string, then transformed into the appropriate data type. After transformation, a corresponding object - either CardData, UnitData, or SpellData - is instantiated and added to the relevant data storage. The code implementation can be found below.

card_data_manager.gd

```
"""
~ CardDataManager ~
Loads CardData objects from a CSV text file into a dictionary, keyed by card_id.
"""

# Imports
const CardTypeEnum = ResourceLocator.CardTypeEnum
const CardData =ResourceLocator.CardData
const SignalfulDualProperty = ResourceLocator.SignalfulDualProperty

class CardDataManager:
    var cards: Dictionary = {}

    func read_card_data() -> void:
        # Prepare CSV file
        var csv_file_name = "res://assets/data/db_cards.csv"
        var csv_file = FileAccess.open(csv_file_name, FileAccess.READ)

        # Check if file opened successfully
        if csv_file == null:
            push_error("CSV file read operation failed")
            return

        # Find order of card attributes in csv_line
        var header = csv_file.get_line().split(",", false)
        # Attributes
        var card_id_index: int = header.find("CARD_ID")
        var card_name_index: int = header.find("NAME")
        var card_cost_index: int = header.find("COST")
        var card_desc_index: int = header.find("DESC")
        var card_type_index: int = header.find("CARD_TYPE")
        var linked_class_id_index: int = header.find("LINKED_CLASS_ID")

        # Check if all card attributes are present
        if -1 in [card_id_index, card_name_index, card_cost_index, card_desc_index, card_type_index,
linked_class_id_index]:
            push_error("Card attribute missing")
            return

        while !csv_file.eof_reached():
            var csv_line = csv_file.get_csv_line(",")

            # Card attributes
            var card_id: int = int(csv_line[card_id_index])
            var card_name: String = csv_line[card_name_index]
            var card_cost: SignalfulDualProperty =\
                SignalfulDualProperty.new(int(csv_line[card_cost_index]))
            var card_desc: String = csv_line[card_desc_index]
            var card_type: int = -1
            var linked_class_id: int = int(csv_line[linked_class_id_index])

            # Validate CardTypeEnum
            if CardTypeEnum.has(csv_line[card_type_index]):
                card_type = CardTypeEnum[csv_line[card_type_index]]
            else:
                push_error("Invalid CardType: ", csv_line[card_type_index])
                return

            # Create and store new CardData
            self.cards[card_id] = CardData\
            .new(card_id, card_name, card_cost, card_desc, card_type, linked_class_id)
```

Figure 42: Code example from prototype A (card_data_manager.gd); Source: Author.

Data Organization in the Artificial Prototype

Meanwhile, the artificial prototype organizes card data using **TRES** (text-based resource) files. These files serve as lightweight data containers and are commonly used to store serialized instances of classes (Resources | Godot Engine Docs, n.d.).

Figure X illustrates examples of serialized data for both an ability card (AbilityData) and a unit card (UnitData), each extending from the BaseCardData class.

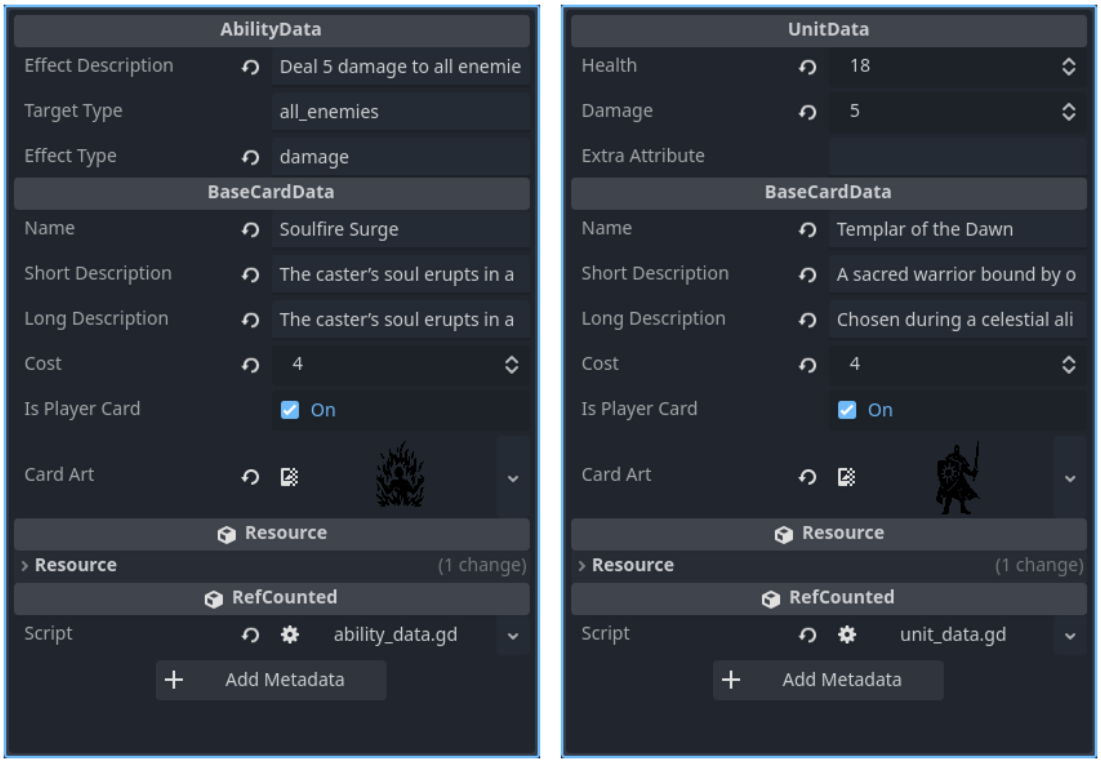


Figure 43: Examples of AbilityData and UnitData TRES files in the Godot inspector; Source: Author.

Using TRES files makes data management much easier, as Godot's built-in resource system automatically handles serialization, loading, and saving without requiring custom parsers. Another advantage is that Godot also automatically assigns and validates the correct attribute types (such as integers, strings, and booleans) when working with TRES files, thus making it safer to modify data.

Additionally, the AI decided to implement a utility script, which automates the generation of TRES files; however, it is **hard-coded**, meaning that the data and file properties must be manually defined within the script itself.

Overview of Data Organization

The following table demonstrates the key differences between the manual and approaches in terms of how data is organized within the project space.

Table 10: Overview of approaches for data organization in prototypes; Source: Author.

Aspect	Manual Approach	Artificial Approach
Storage Format	Data is stored using Google Sheets and exported as a CSV file. The CSV format is supported by a wide variety of applications.	Data is stored as native Godot TRES files.
Editing Process	Data is edited externally in Google Sheets; updates require re-exporting affected CSV files.	Data is edited directly in the Godot editor.
Bulk Editing	**Easy:** spreadsheets allow fast editing of multiple entries at once.	**Hard:** each TRES file must be edited individually or via additional tools or scripts.
Collaboration	**Easy:** supports multi-user editing via shared documents.	**Hard:** single-user editing by default unless using external tools.
Import Process	Requires a CSV reader implemented inside the game to parse and load data at runtime.	Done by Godot. Native and automatic.
Risk of Errors	**High:** wrong CSV formatting can break entire data loading.	**Low:** TRES files are structured and validated by Godot.
Performance	Initial loading needs time but has no impact during gameplay.	Faster load time, no need for manual data parsing.
Flexibility of Use	Low internal flexibility. High external flexibility.	High internal flexibility. Low external flexibility.

The Hybrid Approach

After implementing and reviewing both methods of data organization, I can state that, overall, each method excels where another method does not. Therefore, the best solution in the future would be to use the **hybrid approach**: store data as CSV tables (or any other suitable data format) in an online service, compose a smart script that automates converting CSV into TRES without needing to manually adjust data each time, and then use the generated TRES files within the Godot project.

7.4 Screenshots of the Game Prototypes

Figure 44: Main screen of the manually created prototype; Source: Author.

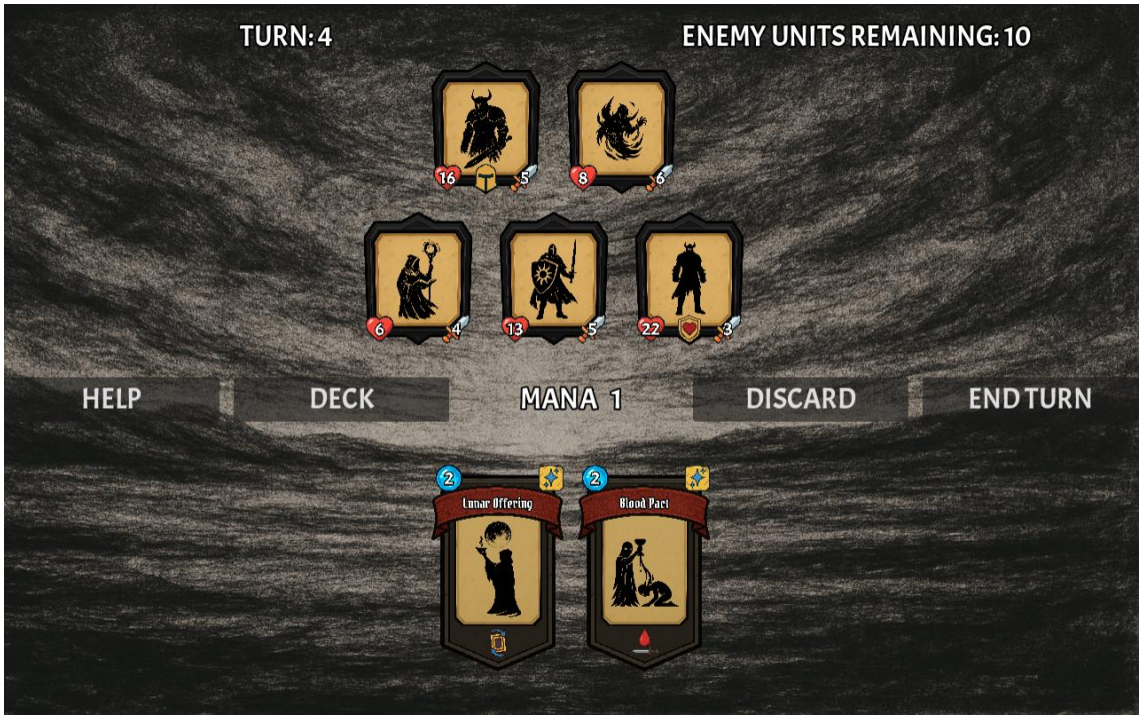


Figure 45: Main screen of the artificially created prototype; Source: Author.

8 Time and Cost Comparison

8.1 Time Comparison

8.1.1 Time and Work Overview of the Manual Approach

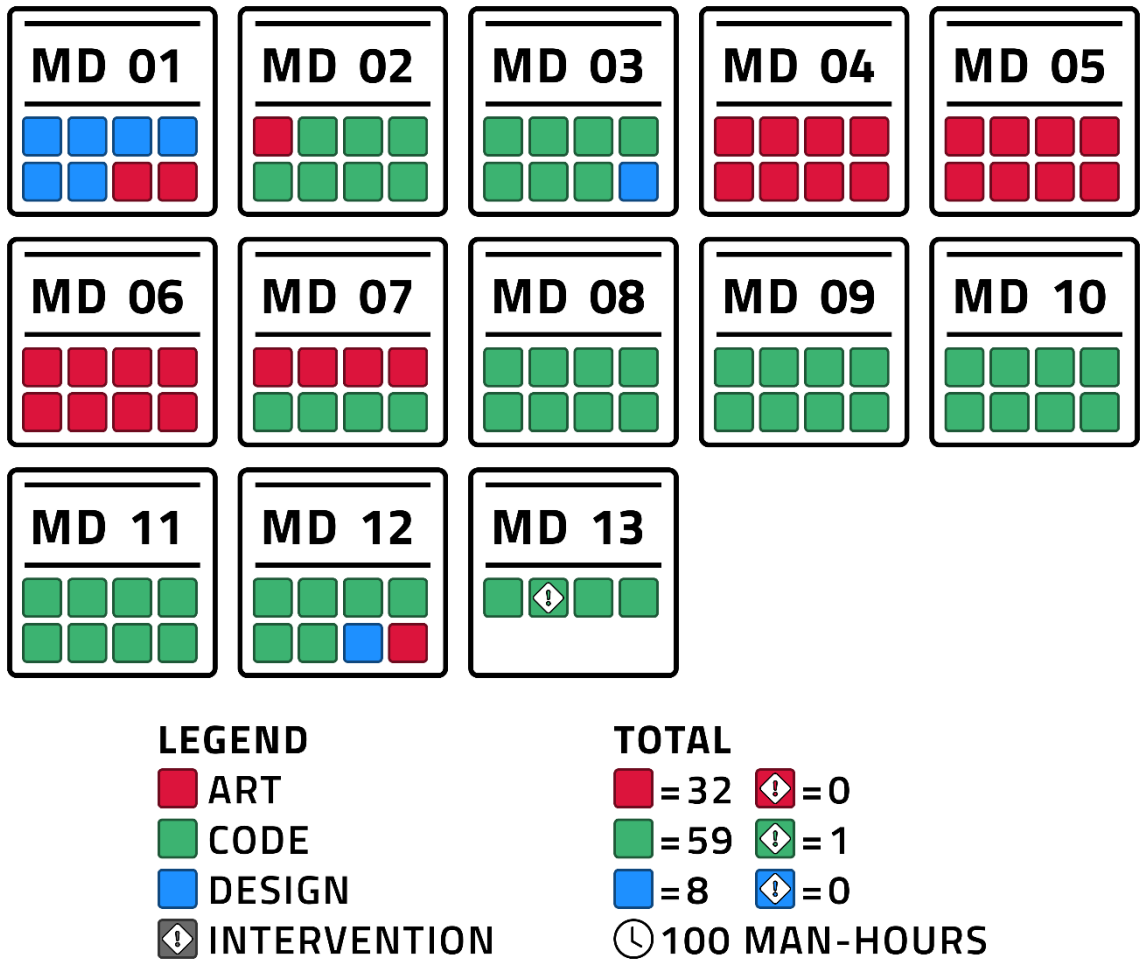


Figure 46: Time and work overview of the manual approach; Source: Author.

Prototype A was completed in approximately **twelve and a half** man-days, with a total development time of **one hundred** man-hours. During this period, multiple **design**, **art**, and **code** tasks were completed. The prototype meets its intended requirements and functions as a working game. The project was developed almost entirely manually, with only one AI intervention occurring on the final day. The workload mostly consisted of the **code-related** tasks (60.00%), with the **art** tasks occupying roughly **one third** of the project scope (32.00%), while the **design** tasks were the least time-consuming (8.00%).

The approach had the following work structure:

- On the first day (MD 01), all core design tasks were completed, including the creation of wireframes.
- From MD 02 to MD 03, an initial implementation of core game logic and mechanics began, including the creation of data tables and CSV parsers.
- Additionally, at the end of MD 03, the town and wave hints were designed as mechanics.
- During MD 04 to MD 07, the primary focus was on art production. Visual assets, such as the card's base and label, artworks for units and spells, battlefield background and icons, were created.
- From MD 08 to MD 11, development shifted fully back to coding, implementing the required game logic and mechanics, user interface, and additional utility functionality.
- Final design and art adjustments were made on the last day (MD 12), followed by a few code edits, after which the game was exported as both desktop and web versions (MD 13).

The ratio of AI tool usage for code-related tasks was 1.69% and 1.00% for the entire project. Both values fall under the category of **minimal intervention** and do not invalidate the manual development approach.

8.1.2 Time and Work Overview of the Artificial Approach

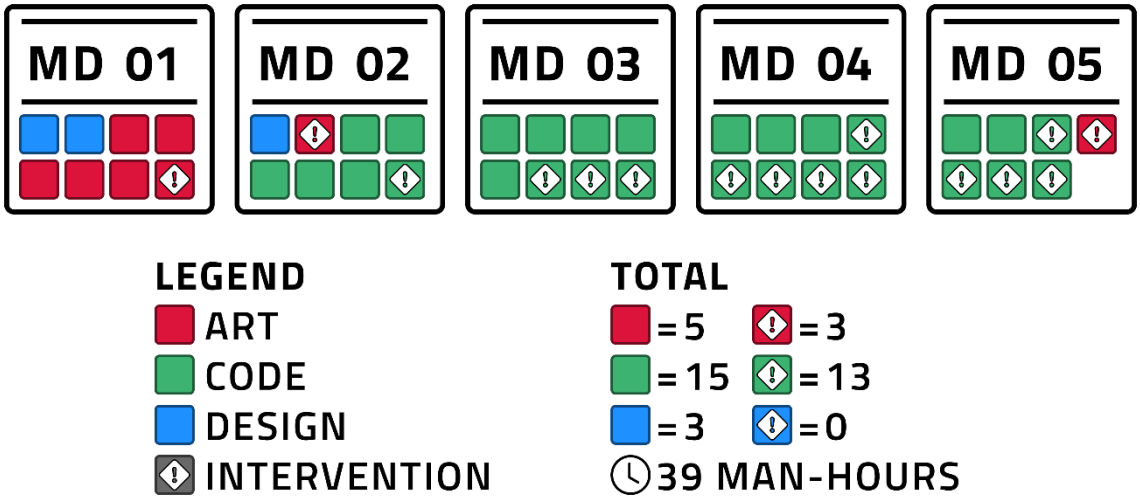


Figure 47: Time and work overview of the artificial approach; Source: Author.

Prototype B was completed much faster, in just **five** man-days, with a total development time of **thirty-nine** man-hours. The AI completed various **design**, **art**, and **code** tasks during this period; however, multiple manual interventions also occurred throughout the entire development process.

During the first day of development (MD 01 and the beginning of MD 02), the main **design** and **art** tasks were completed, which included the creation of core mechanics, cards, units and abilities from the **design side** and the generation of the wireframes, artworks and icons from the **art side**. Unfortunately, manual intervention was needed for creating the main scene wireframe and editing the artworks, as most of them had defects, such as cropping, non-transparent backgrounds and noise artefacts.

The majority of code tasks occurred from MD 02 to MD 04. As the tasks became more specific, AI generated more **hallucinations**, and more human supervision was needed. During this, all main mechanics were implemented. On the last day (MD 05), final **code** and **art** adjustments were made. The desktop and web versions had to be manually finalized and exported.

This approach received a rapid development boost with the use of AI; however, it required considerable human involvement. The ratio of manual intervention was:

- **None** (0.00%) for design-**related** tasks.
- **Minor** (37.50%) for art-**related** tasks.
- **Major** (46.43%) for the **code-related** tasks.

The **overall intervention ratio** is 41.03%, which classifies as **major** human involvement.

8.1.3 Time Comparison Summary

The data presented in the **manual** and **artificial** time and work overviews may initially appear straightforward, but it can be misleading without a deeper analysis. For example, the time difference between code-related tasks has a ratio of **60** : **28** (**manual** : **artificial**), which approximately equals **2.14**. That suggests that the artificial approach is more than **two** times faster. However, there is one factor that is omitted: **interventions**.

Because the interventions compromise the standard workflow, they have to be addressed. Therefore, I have decided to introduce a nominal and real work speed gain.

$$Nominal\ Work\ Speed\ Gain = \frac{Manual\ Task\ Time + AI\ Intervention}{Artifical\ Task\ Time + Human\ Intervention}$$

Equation 2: Nominal work speed gain; Source: Author.

$$Real\ Work\ Speed\ Gain = \frac{Manual\ Task\ Time + (AI\ Intervention * WF_{manual})}{Artifical\ Task\ Time + (Human\ Intervention * WF_{artificial})}$$

Equation 3: Real work speed gain; Source: Author.

WF - weight factor to emphasize the cost of compromised workflow. In my case I have set the **weight factor** to:

- **0** for if no **interventions** have happened.
- **1.25** for **minimal** interventions.
- **2** for **minor** interventions
- **3** for **major** interventions
- **4** for **critical** interventions,

Based on these factors, I have calculated work speed gain for each task and overall project completion time.

Table 11: Nominal and real work speed gain; Source: Author.

Task Category	*Manual Task Time (MH)*	*AI Intervention (MH)*	*Weight Factor (Manual)*	*Artificial Task Time (MH)*	*Human Intervention (MH)*	*Weight Factor (Artificial)*	*Nominal Work Speed Gain*	*Real Work Speed Gain*
Code	59	1	1.25	15	13	3	2.14	1.12
Art	32	0	0	5	3	2	4.00	2.91
Design	8	0	0	3	0	0	2.67	2.67
Overall	99	1	1.25	23	16	3	2.56	1.41

Based on the calculated real work speed gain, the **artificial approach** resulted in a **41% improvement** in overall development speed relative to the **manual approach**.

8.2 Cost Comparison

8.2.1 Data Collection

As was mentioned in the methodology chapter, the following roles of the game development team are inspected during the comparison: **artist**, **designer**, and **programmer**, which were substituted by a **prompt write**r role during the standard flow of the artificial approach.

To start analyzing development costs, information about salaries for these positions is needed. Therefore, I have decided to use data from the **Glassdoor** web resource. Glassdoor provides fresh information about the median total pay for these positions. It should be noted that median total pay is not exactly the same as median base salary – it is usually slightly higher, as it typically includes additional bonuses and benefits. However, this is a minor issue, and the median total pay is sufficient for conducting a meaningful comparison. Unfortunately, there are **two** major issues.

The first issue is that, because the **prompt writer** is a relatively new profession, currently, there is no available information about its salary. To address this lack of information, I have decided to use the salary data for the **technical writer** position as a reference.

A **technical writer** creates documents that explain how to use products or interpret complex concepts in a clear and understandable way. Their work involves gathering, organizing, and presenting information so that the target audience can easily understand it (Macari, 2025). The core responsibilities of the **technical writer** and the **prompt writer** are quite similar; therefore, the position of technical writer was used as a substitute for the **prompt writer.**

The second issue is that, due to the approach of the comparison, namely the use of man-days and man-hours as units of measurement, hourly salary data is required, whereas Glassdoor only provides monthly salary information. To resolve this, I have decided to use the salary calculator tool available on the **Calculator.net** website. To achieve more precise calculations, additional information about hours per week, workdays per week, public holidays per year, and vacation days per year was required. Given that the salary data originates from the **Czech Republic**, I utilized the following parameter values:

- Hours per week: **forty** (40).
- Days per week: **five** (5).
- Public holidays per year: **thirteen** (13).
- Vacation days per year: **twenty** (20).

It also should be noted that I have used a **"1-3 years" experience filter** when collecting data from Glassdoor.

Table 12: Information about salaries for selected roles; Source: Author.

Full Position Title	Monthly Salary (CZK)	Hourly Salary (CZK)	Data Source
Game Designer	63 000	363.46	(Salary: Game Designer | Glassdoor, 2024)
Artist	38 000	219.23	(Salary: Artist | Glassdoor, 2021)
Game Programmer	59 000	340.38	(Salary: Game Programmer | Glassdoor, 2024)
Technical Writer	53 000	305.77	(Salary: Technical Writer | Glassdoor, 2025)

Moreover, I also needed to calculate the cost of the development tools used during the project. Fortunately, most of these tools were free to use and therefore had no financial cost. However, the one exception was the ChatGPT Plus model, which had a monthly subscription fee of 20 USD (United States Dollars). To correctly account for subscription expenses, I have introduced an **effective daily cost** using the following formula:

$$Effective\ Daily\ Cost = \frac{Monthly\ Subscription\ Fee}{30}$$

Equation 4: Effective daily cost formula; Source: Author.

This approach distributes the subscription cost according to actual tool usage during development. Additionally, since the salary data was collected in CZK (Czech Korunas), the subscription cost had to be converted from USD to CZK. For this purpose, I used a fixed exchange rate of 1 USD to **22.5** CZK.

8.2.2 Overviews of Development Costs

Table 13: Development costs of the manual approach; Source: Author.

MANUAL APPROACH: DEVELOPMENT COSTS				
Personnel				
Work Category	*Position*	*Time (MH)*	*Hourly Cost (CZK)*	*Subtotal (CZK)*
Art	Artist	32	219.23	7015.36
Code	Programmer	59	340.38	20082.42
Design	Designer	8	363.46	2907.68
Intervention: Code	Programmer	1	340.38	340.38
Personnel Total (CZK)				**30345.84**
Tools				
Name	*Subscription*	*Time (MD)*	*Daily Cost (CZK)*	*Subtotal (CZK)*
ChatGPT	Plus	1	15	15
Figma	Free	3	0	0
Godot	Standard (Free)	9	0	0
Google Sheets	Free	2	0	0
Krita	Standard (Free)	1	0	0
Tools Total (CZK)				**15**
Overall Total (CZK)				**30360.84**

Table 14: Development costs of the artificial approach; Source: Author.

ARTIFICIAL APPROACH: DEVELOPMENT COSTS				
Personnel				
Work Category	*Position*	*Time (MH)*	*Hourly Cost (CZK)*	*Subtotal (CZK)*
Art	Technical Writer	6	305.77	1834.62
Code	Technical Writer	17	305.77	5198.09
Design	Technical Writer	3	305.77	917.31
Intervention: Art	Artist	2	219.23	438.46
Intervention: Code	Programmer	11	340.38	3744.18
Personnel Total (CZK)				**12132.66**
Tools				
Name	*Subscription*	*Time (MD)*	*Daily Cost (CZK)*	*Subtotal (CZK)*
ChatGPT	Plus	5	15	75
Figma	Free	2	0	0
Godot	Standard (Free)	4	0	0
Krita	Standard (Free)	1	0	0
Tools Total (CZK)				**75**
Overall Total (CZK)				**12207.66**

8.2.3 Cost Comparison Summary

The manual approach has a total development cost of **30 360.84 CZK**, meanwhile the artificial Approach required significantly less, totaling **12 207.66 CZK**. The difference is primarily due to reduced personnel costs in the artificial approach, where multiple tasks were handled by AI, resulting in both **faster work speed** and **lower total expenses.**

The **artificial approach reduced development costs by approximately 60%**, demonstrating that companies have an opportunity to greatly improve their financial situation and budgets, when integrating **generative AI** into the workflow.

9 Testing and Quality Comparison

9.1 A/B Playtesting

A/B testing is a common practice across many industries, including game development, where it plays a vital role in improving game quality. It helps developers identify which parts of the game work well, which areas need refinement, and which elements make the game engaging for players. At its core, A/B game testing can be described as a process of comparing two versions of a game to determine which one performs better. A/B testing can be done in an uncontrolled and controlled environment (Cristofolini, 2023).

Additionally, I have used a **counterbalancing technique**: counterbalancing is a method of designing an experiment used to control order effects, such as practice, fatigue, and other biases, which can impact the results. The main idea is to vary the sequence of conditions across participants in order to distribute potential order effects evenly. Counterbalancing can be obtained through different designs; one example of counterbalancing is the "ABAB" technique (Corriero, 2017).

The total of **eight** respondents (also referred to as **R1**-**R8**) participated in playtesting; during the playtesting respondents were playing the **web version of the game**.

I have split my playtesters into **two** groups: the first group played prototype A followed by prototype B, while the second group played prototype B followed by prototype A. After playing a prototype, playtesters completed the **mid survey**; at the end of playtesting, respondents were given the **end survey**. A flow of the playtest is demonstrated below.

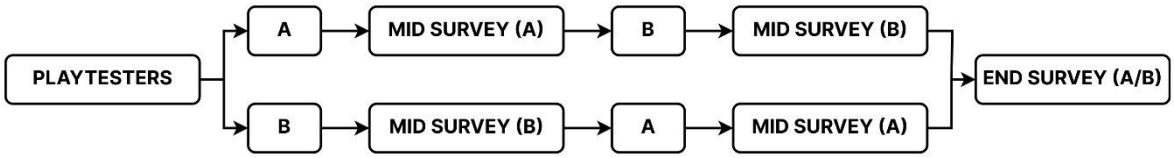


Figure 48: Structure of playtesting flow; Source: Author.

To evaluate the quality of the game prototypes, I conducted a **supervised usability A/B playtesting** session in the form of a **live gameplay demonstration.**

Before each session, respondents were informed that the goal of the test was to evaluate the game and observe their behavior reactions and thoughts. All participants volunteered **willingly**. Each playtest was carried out via a live voice conversation on the **Discord** platform, during which the playtester shared their screen while playing with the game. I observed the playing process and provided some guidance when necessary. The duration of each session ranged between **thirty** and **forty-five** minutes. To support later analysis, the audio from these sessions was recorded using the **Audacity** application.

9.2 Testing Survey Questions

9.2.1 Mid Survey

Table 15: Questions of the mid survey; Source: Author.

ID	QUESTION	KEY ASPECT	NOTE
MQ-01	*What is your first impression of the game?*	General	Asked within 1-5 minutes
MQ-02	*What is your impression of the game after a few minutes?*	General	Asked within 5-10 minutes
MQ-03	*What part of the game did you like the most?*	General	-
MQ-04	*What part of the game did you like the least?*	General	-
MQ-05	*Was the main goal of the game clear from the beginning?*	Mechanics	Yes / No
MQ-06	*Was the user interface (UI) intuitive / clear?*	Aesthetics	Yes / No
MQ-07	*Did the player reach the end of the game?*	Mechanics	Victory / Defeat / None
MQ-08	*What other game is this similar to?*	General	Response time should be recorded
MQ-09	*Which cards were the most enjoyable to play?*	Mechanics	-
MQ-10	*What impression did the game's visuals make?*	Aesthetics	-

The mid survey was conducted during each prototype playing session. Its primary purpose was to gather **feedback** from each respondent. This feedback was not used for direct comparison of **two** prototypes, but served as valuable insight for understanding the strengths and weaknesses of each version.

9.2.2 End Survey

Table 16: Questions of the end survey; Source: Author.

ID	QUESTION	KEY ASPECT	NOTE
EQ-01	*Which game had more card variety?*	General	A / B
EQ-02	*Which game had a better card design?*	Aesthetics	A / B
EQ-03	*Which game seemed more original / unique / unusual?*	Aesthetics	A / B
EQ-04	Which game had a clearer / intuitive user interface (UI)?	Aesthetics	A / B
EQ-05	*Which game had more interesting interactions between cards?*	Mechanics	A / B
EQ-06	*In which game did you have to think more before taking an action?*	Mechanics	A / B
EQ-07	*Which game had more meaningful choices that affected the gameplay?*	Mechanics	A / B
EQ-08	*Which game gave a greater sense of control over the situation?*	Mechanics	A / B
EQ-09	*Which game was more memorable?*	General	A / B
EQ-10	*Which game did you like the most?*	General	A / B
EQ-11	*Which game would you be more likely to play again if it was improved?*	General	A / B
EQ-12	*Which game, in your opinion, was created by AI?*	Other	A / B

The end survey was conducted after both prototypes had been played. The end survey's data was used for direct comparison. Participants selected the most preferable game prototype based on the question. The game with the most favorable responses in each category was identified as the higher-quality prototype for that category.

9.3 Testing Results

9.3.1 Feedback from Testers

Feedback on Game A

Game A was generally seen as more visually appealing and described as "cool", but feedback also pointed out weaker **mechanics**. Some testers suggested transferring mechanics from game B to game A. The most liked card was "**Improvise**" (selected by **four** testers). However, they also pointed out that it is too powerful and needs rework. The players liked the auto-battle feature of the game A. A major issue with game A was a bug in the battlefield that allowed the placement of friendly units on enemy positions. What was quite strange is that game A was compared to **tower defense** games, perhaps due to the battlefield layout**.**

Feedback on Game B

Game B required more **mental activity** from players as they needed to control their units and calculate the attack damage. The favorite thing to do in game B was using ability cards, especially **"Blood Pact"** and **"Soulfire Surge"** (each one was marked by **three** players as the best one in B). What players didn't like about game B was the dark and distracting **background**, which made it hard to focus on the game. **One** tester also spotted the traces of artifacts.

Additional Suggestions

Players had several suggestions for improving **game A**. The common request was to add an interactive tutorial to help orientate in the game. Several testers said they wanted the ability to move or return their units in hand du, and some even suggested being able to capture enemy positions. Others asked for more variety in units, spells, and enemies. A few testers wanted clearer health indicators. Finally, one tester suggested adding a boss to fight for more excitement.

Players also gave a few suggestions for how to improve **game B**. Many wanted an auto-battle feature added, since it was available in game A. Several testers said the game also needed an interactive tutorial to help explain how to play. Others asked for more animations and visual effects to make the game feel more responsive.

9.3.2 Quality Comparison

Table 17: Results of the end survey; Source: Author.

QUESTION	GAME		PLAYTESTERS							
	A	B	1	2	3	4	5	6	7	8
Which game had more card variety?	37.5%	62.5%	B	B	A	A	A	B	B	B
Which game had a better card design?	75.0%	25.0%	A	A	B	A	A	B	A	A
Which game seemed more original / unique / unusual?	75.0%	25.0%	A	A	B	A	B	A	A	A
Which game had a clearer / intuitive user interface (UI)?	50.0%	50.0%	B	B	A	B	A	A	B	A
Which game had more interesting interactions between cards?	50.0%	50.0%	B	B	A	A	B	B	A	A
In which game did you have to think more before taking an action?	37.5%	62.5%	B	A	B	B	A	B	B	A
Which game had more meaningful choices that affected the gameplay?	75.0%	25.0%	B	A	B	A	A	A	A	A
Which game gave a greater sense of control over the situation?	50.0%	50.0%	B	A	A	A	B	B	A	B
Which game was more memorable?	75.0%	25.0%	A	A	A	B	B	A	A	A
Which game did you like the most?	75.0%	25.0%	A	A	B	A	B	A	A	A
Which game would you be more likely to play again if it was improved?	62.5%	37.5%	B	A	B	A	A	B	A	A
Which game, in your opinion, was created by AI?	25.0%	75.0%	B	B	A	B	B	B	A	B

Summary

Game A was preferred in **six** of **eleven** final questions. It surpassed game B in **aesthetics** and **overall impressions** - it was more liked, more memorable, more original, and had better visual design.

Game B was preferred in **two** of **eleven** final questions. The AI had designed much better **mechanics**, and made the players think.

Game B was correctly identified as the **artificially created** game by **75%** of the respondents.

Both games have **fulfilled** the prototype requirements.

Conclusion

Answers on Research Questions

Q1: What are the key differences between manual and artificial development approaches?

After comparing the prototypes based on **time**, **cost** and **quality** criteria, the following insights were found:

- The artificial approach was **41%** faster than the manual approach (**adjusted for interventions**); it was completed in **five man-days** (39 man-hours), whereas the manual approach took **thirteen and a half man-days** (100 man-hours).
- Consequently, the **development costs** of the artificial approach were notably lower, by **approximately 60%.**
- The manual prototype gained the sympathy of most playtesters; it stood out in terms of **aesthetics** and **overall quality**; but the artificial approach introduced several **mechanics** that, if added into the manually made game, could significantly improve and deepen its gameplay.

However, the artificial approach required **multiple interventions** from a "professional" human developer (Author) to fulfil the requirements. The overall human involvement in the artificial development process was classified as **major**.

Additionally, there were differences in **execution** itself, for example the implementation of card component via **composition** (manual) versus via **inheritance** (artificial), the use of **external SVG files** (manual) versus **native TRES files** (manual).

The techniques and methods used by the AI can be described as **"the path of least resistance"**, whereas the manual approach focused on **long-term game structure.**

Q2: Are players able to differentiate between manual and artificially developed games?

Based on the results of the A/B playtesting, **75%** of participants **correctly answered** which game was developed by the AI and which by a human. This suggests that the majority of players **are able to differentiate** between manually and artificially developed games.

This is an important topic because people value **transparency** from the creators of AI-generated content and **fairness** toward authors of human-made creative works (as demonstrated by the "AI survey"). The less capable people are to distinguish AI-made content from human-made content, the more opportunities arise for "**bad actors"** to exploit the current environment, for example impersonating creators.

Possible Ways for Improvement

The comparative analysis can be improved by introducing **game-oriented KPIs** (key performance indicators), such as **frame rate, input latency, memory usage,** or others.

The comparative analysis can be improved by examining other development teams, for example:

- **Sound and Music** (manually versus AI-generated audio assets).
- **Marketing and Public Relations** (manually versus AI-generated marketing campaigns, market surveys).
- **Quality Assurance and Testing** (manual tests versus automated tests).

The comparative analysis can be improved by examining other fields of AI application in game development, for example manually versus AI-generated levels (AI that Creates for Player Experience).

Additionally, the **A/B testing** can be further improved by focusing on individual components of the game, such as **dialogues**, **artworks**, or even **code** (if the respondents have sufficient technical knowledge).

Personal Recommendations

The process of game development should start by outlining the **main idea** and **core components** (aesthetics, mechanics, story, and technology). I recommend drafting a **synopsis** (a one-sentence description of the game) as it keeps the creative vision clear and consistent.

When coding, using **enumerators** instead of **strings** is better, as it allows for better code management. For the same reason, I recommend using **design patterns.**

I recommend asking somebody to test your game (or other application), because You look at the game through a **creator's eyes**, whereas they look through a **consumer's eyes.**

Generative AI tools are a good source of **inspiration** or **ideas**. They also can give You a useful information or advice, however I would suggest **verifying** it, especially if it important.

AI is a good tool for **prototyping**, as it can quickly generate placeholder assets or implement basic functionality.

When working with AI use the **iterative approach**: a minor adjustment or modification of Your request may significantly impact the results. Giving AI **clear** and **well-structured** commands also helps.

List of references

Arora, K. (2023, November 17). *The Gaming Industry: A Behemoth With Unprecedented Global Reach*. Forbes. https://www.forbes.com/councils/forbesagencycouncil/2023/11/17/the-gaming-industry-a-behemoth-with-unprecedented-global-reach/

Bill Wagner. (2022, February 16). *Objected oriented programming—Inheritance—C#*. https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/inheritance

Buijsman, M. (2024, August 13). The global games market will generate $187.7 billion in 2024. *Newzoo*. https://newzoo.com/resources/blog/global-games-market-revenue-estimates-and-forecasts-in-2024

Cain, T. (Director). (2024, August 19). *Game Development Roles* [Video recording]. https://www.youtube.com/watch?v=DEnlUqYkMTk

Cardillo, A. (2025, May 2). *How Many Companies Use AI? (New 2025 Data)*. Exploding Topics. https://explodingtopics.com/blog/companies-using-ai

Cards | Inscryption Wiki. (n.d.). Inscryption Wiki. Retrieved May 5, 2025, from https://inscryption.fandom.com/wiki/Cards

Carter, J. (2024, March 29). *Indie-focused Triple-I Initiative launched by Dead Cells, Slay the Spire devs*. https://www.gamedeveloper.com/business/indie-focused-triple-i-initiative-launched-by-dead-cells-slay-the-spire-devs

ChatGPT. (2025, May 4). Wikipedia. https://en.wikipedia.org/w/index.php?title=ChatGPT&oldid=1288744394

Contel, M., Tedeev, V., Gowardun, D., Amat, M., & Sidhu, H. S. (n.d.). *What are the best A/B testing methods for game development?* Retrieved May 9, 2025, from https://www.linkedin.com/advice/3/what-best-ab-testing-methods-game-development-oca5f

Corriero, E. F. (2017). Counterbalancing. In *The SAGE Encyclopedia of Communication Research Methods* (pp. 278–281). SAGE Publications, Inc. https://doi.org/10.4135/9781483381411

Cristofolini, R. (2023, February 23). *A/B Testing for Games | LinkedIn*. https://www.linkedin.com/pulse/ab-testing-games-ricardo-cristofolini/

Culbreath, D. (2024, June 6). *Symbolic AI and Generative AI - What's the Difference? | LinkedIn*. https://www.linkedin.com/pulse/symbolic-ai-generative-whats-difference-darren-culbreath-mkvpe/

Data Organization | GeeksforGeeks. (2024, August 12). GeeksforGeeks. https://www.geeksforgeeks.org/what-is-data-organization/

Davies, I. (2024, June 21). *(9) So you want to be a video games lawyer? | LinkedIn*. https://www.linkedin.com/pulse/so-you-want-video-games-lawyer-isabel-davies-5velc/

Definition of video game. (n.d.). PCMAG. Retrieved March 4, 2025, from https://www.pcmag.com/encyclopedia/term/video-game?_gl=1%2A5b1lo8%2A_up%2AMQ..%2A_ga%2AMTU1OTgwMjEwNC4xNzQxMTE2Mjcx%2A_ga_8TEVGCYPY5%2AMTc0MTExNjI3MS4xLjAuMTc0MTExNjI3MS4wLjAuMA..

Definition of video game console. (n.d.). PCMAG. Retrieved March 24, 2025, from https://www.pcmag.com/encyclopedia/term/video-game-console?_gl=1%2A1a4bhae%2A_up%2AMQ..%2A_ga%2AMzMwOTI2NDM3LjE3NDA4NzMyMTI.%2A_ga_8TEVGCYPY5%2AMTc0MDkzMzk0Ny4yLjAuMTc0MDkzMzk1OS4wLjAuMA..

England, L. (2014, April). *"The Door Problem" – Liz England*. https://lizengland.com/blog/2014/04/the-door-problem/

Esposito, N. (2005). A Short and Simple Definition of What a Videogame Is. *ResearchGate*. https://www.researchgate.net/publication/221217421_A_Short_and_Simple_Definition_of_What_a_Videogame_Is

Figma. (2025, May 6). Wikipedia. https://en.wikipedia.org/w/index.php?title=Figma&oldid=1289103419

Fisher, M. (2023, March 19). Getting Started As A Video Game Marketer In The Gaming Industry. *Esports Tower*. https://esportstower.com/news/becoming-a-video-game-marketer/

Freed, A. (2015). *Yes, You Have To Write a Game Plot Summary; and Yes, It Has To Be Good*. https://www.gamedeveloper.com/design/yes-you-have-to-write-a-game-plot-summary-and-yes-it-has-to-be-good

Fundamentals of SW Architecture | GeeksforGeeks. (2022, December 2). https://www.geeksforgeeks.org/fundamentals-of-software-architecture/

Godot Docs – 4.4 branch. (n.d.). Godot Engine Documentation. Retrieved May 9, 2025, from https://docs.godotengine.org/en/stable/index.html

Google Sheets. (2025, March 26). Wikipedia. https://en.wikipedia.org/w/index.php?title=Google_Sheets&oldid=1282473421

Hearthstone Wiki. (n.d.). Retrieved May 12, 2025, from https://hearthstone.wiki.gg/

Heroes | Darkest Dungeon Wiki. (n.d.). Darkest Dungeon Wiki. Retrieved May 5, 2025, from https://darkestdungeon.wiki.gg/wiki/Heroes_(Darkest_Dungeon)

How do I create a good prompt for an AI model like GPT-4? | OpenAI Help Center. (n.d.). Retrieved May 9, 2025, from https://help.openai.com/en/articles/4936848-how-do-i-create-a-good-prompt-for-an-ai-model-like-gpt-4

IBM Data and AI Team. (2023, July 6). *AI vs. Machine Learning vs. Deep Learning vs. Neural Networks | IBM*. https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks

Indeed Team. (2024, April). *7 Key Roles in Video Game Development*. Indeed Career Guide. https://www.indeed.com/career-advice/finding-a-job/game-development-roles

INLINGO. (2024, July 26). Whats the difference between Indie vs AA vs AAA games. *INLINGO – Gamedev Outsourcing Studios*. https://inlingogames.com/blog/indie-vs-aaa-vs-aa-games/

Inscryption Wiki. (n.d.). Retrieved May 12, 2025, from https://inscryption.fandom.com/wiki/Inscryption_Wiki

Kovtun, D. (2024, June 5). *Key roles in a game development team—2024 edition*. Pingle Studio. https://pinglestudio.com/blog/key-roles-in-a-game-development-team-2024-edition

Layers in ANN | GeeksforGeeks. (2025, March 1). GeeksforGeeks. https://www.geeksforgeeks.org/layers-in-artificial-neural-networks-ann/

Lemne, B. (2016, January 23). *The Triple-I Revolution*. Gamereactor UK. https://www.gamereactor.eu/the-triplei-revolution/

Macari, S. (2025, March 3). *What Does a Technical Writer Do? (Plus How To Become One)*. Indeed Career Guide. https://www.indeed.com/career-advice/careers/what-does-a-technical-writer-do

Maria, T., Ramy, B., & Souha, M. (2022, May 20). Medium—Gaming Industry. *Medium*. https://medium.com/@microclub_usthb/gaming-industry-b390c5afd72b

Marr, B. (2024, April 18). *The Role Of Generative AI In Video Game Development*. Forbes. https://www.forbes.com/sites/bernardmarr/2024/04/18/the-role-of-generative-ai-in-video-game-development/

McZeal, M. (2023, September 16). *Indie Games That Cost More To Make Than You Think*. TheGamer. https://www.thegamer.com/indie-games-surprisingly-high-budgets-costs/

Minaiev, N. (2025, February 10). *Roles in game development: Building a successful game development team - iLogos Game Studios*. https://ilogos.biz/roles-in-game-development/

Mizina, A. (2025, March 18). *What Are Marketing Jobs in the Gaming Industry? A Comprehensive Overview*. https://www.trapplan.com/cms-posts/blog/what-are-marketing-jobs-in-the-gaming-industry-a-comprehensive-overview

Mobile Gaming. (n.d.). AppLovin. Retrieved March 24, 2025, from https://www.applovin.com/glossary/mobile-gaming/

MobyGames | Neon White credits. (2022). MobyGames. https://www.mobygames.com/game/186311/neon-white/credits/windows/

Monster Train Wiki. (n.d.). Retrieved May 12, 2025, from https://monster-train.fandom.com/wiki/Monster_Train_Wiki

Mullich, D. (2015, February 3). *Roles On A Game Development Team*. https://davidmullich.com/2015/02/02/roles-on-a-game-development-team/

Mullich, D. (2016, January 11). *Defining Play, Game, and Gaming | David Mullich*. https://davidmullich.com/2016/01/11/defining-play-game-and-gaming/

Newzoo's Global Games Market Report 2023. (2024, February 8). *Newzoo*. https://newzoo.com/resources/trend-reports/newzoo-global-games-market-report-2023-free-version

Newzoo's Global Games Market Report 2024. (2024, August 13). *Newzoo*. https://newzoo.com/resources/trend-reports/newzoos-global-games-market-report-2024-free-version

Nystrom, R. (2014). *Game Programming Patterns*.

(PDF) Analysis of the Video Gaming Industry. (2024, November 21). *ResearchGate*. https://doi.org/10.2991/aebmr.k.220603.185

Peñaranda, A. B. (2022, October 31). *Why are Mobile Games so Popular? | LinkedIn*. https://www.linkedin.com/pulse/why-mobile-games-so-popular-almudena-berzosa-pe%C3%B1aranda/

Porokh, A. (2023, November 1). *A Comparison Indie, AA vs AAA Games*. Kevuru Games. https://kevurugames.com/blog/indie-aa-vs-aaa-game-unraveling-the-differences/

Precedence Research. (2025, January 31). *Video Games Market Size To Hit USD 721.77 Billion By 2034*. https://www.precedenceresearch.com/video-game-market

Prompt engineering best practices for ChatGPT | OpenAI Help Center. (n.d.). Retrieved May 9, 2025, from https://help.openai.com/en/articles/10032626-prompt-engineering-best-practices-for-chatgpt

Resources | Godot Engine Docs. (n.d.). Godot Engine Documentation. Retrieved May 7, 2025, from https://docs.godotengine.org/en/stable/tutorials/scripting/tutorials/scripting/resources.html

Rocket Brush Studio. (2023, December 25). *AAA, AA, Indie Games: Different Routes in Game Development*. https://rocketbrush.com/blog/aaa-aa-indie-games-distinct-paths-in-game-development

Sakurai, M. (Director). (2022, December 16). *Jobs in Game Development [Team Management]* [Video recording]. https://www.youtube.com/watch?v=Hk2npGxEQRo

Salary: Artist in Czech Republic 2025 | Glassdoor. (2021, November 1). Glassdoor. https://www.glassdoor.com/Salaries/czech-republic-artist-salary-SRCH_IL.0,14_IN77_KO15,21.htm

Salary: Game Designer in Czech Republic 2025 | Glassdoor. (2024, July 11). Glassdoor. https://www.glassdoor.com/Salaries/game-designer-salary-SRCH_KO0,13.htm

Salary: Game Programmer in Czech Republic 2025 | Glassdoor. (2024, May 22). Glassdoor. https://www.glassdoor.com/Salaries/game-programmer-salary-SRCH_KO0,15.htm

Salary: Technical Writer in Czech Republic 2025 | Glassdoor. (2025, February 4). Glassdoor. https://www.glassdoor.com/Salaries/technical-writer-salary-SRCH_KO0,16.htm

Saleem, S. (Director). (2013, August 13). *A brief history of video games (Part I)—Safwat Saleem—YouTube* [Video recording]. https://www.youtube.com/watch?v=x24KoVNliMk

Salen, K., & Zimmerman, E. (2004). *Rules of Play: Game Design Fundamentals*.

Sasser, T. (2023, December 6). Game Development Roles. *Medium*. https://medium.com/@thaddeussasser/game-development-roles-9a7f6f1bf7ec

Schell, J. (2019). *The Art of Game Design: A Book of Lenses*. https://www.inventoridigiochi.it/wp-content/uploads/2020/07/art-of-game-design.pdf

Shroyer, J. (2023a, April 6). *(9) The Importance of Localization in Gaming: Breaking Barriers and Building Bridges | LinkedIn*. https://www.linkedin.com/pulse/importance-localization-gaming-breaking-barriers-building-shroyer/

Shroyer, J. (2023b, June 17). *The Impact of Video Games on the Economy*. Chiefcxofficer. https://chiefcxofficer.com/the-impact-of-video-games-on-the-economy

Shvets, A. (2021). *Dive into Design Patterns*. https://refactoring.guru/design-patterns/book

Slay the Spire Wiki. (n.d.). Retrieved May 12, 2025, from https://slay-the-spire.fandom.com/wiki/Slay_the_Spire_Wiki

Smith, M. (2023, August 31). *6 Indie Developers Punching With AAA Budgets*. Game Rant. https://gamerant.com/indie-developers-aaa-budgets/

Standard 52-card deck | Wikipedia. (2025, April 2). Wikipedia. https://en.wikipedia.org/w/index.php?title=Standard_52-card_deck&oldid=1283602438

Steam Publisher: Larian Studios. (2025). https://store.steampowered.com/publisher/larianstudios

SteamDB. (2025). *Steam Game Release Summary by Year*. SteamDB. https://steamdb.info/stats/releases/

Straits Research. (2024, August 7). *Books Market Size, Share & Demand, Trends Analysis Report, 2031*. https://straitsresearch.com/report/books-market

Stryker, C., & Kavlakoglu, E. (2024, August 9). *What Is Artificial Intelligence (AI)? | IBM*. https://www.ibm.com/think/topics/artificial-intelligence

Susan Jamieson. (2025, March 20). *Likert scale | Social Science Surveys & Applications | Britannica*. https://www.britannica.com/topic/Likert-Scale

Tait, C. (2024, April 24). *What is PC Gaming? The Definitive Way to Play | CyberPowerPC*. https://www.cyberpowerpc.com/blog/what-is-pc-gaming-the-definitive-way-to-play/

Thau, J. (2024, May 22). *Gaming Beyond Entertainment: How Video Games Are Shaping The Future*. Forbes. https://www.forbes.com/councils/forbestechcouncil/2024/05/22/gaming-beyond-entertainment-how-video-games-are-shaping-the-future/

The Art of Game Design: A Book of Lenses. (n.d.). Retrieved March 4, 2025, from https://dictionary.cambridge.org/dictionary/english/video-game

The Witness credits (Windows, 2016). (n.d.). MobyGames. Retrieved March 23, 2025, from https://www.mobygames.com/game/76709/the-witness/credits/windows/

Thompson, T. (n.d.). *AI and Games*. YouTube. Retrieved May 6, 2025, from https://www.youtube.com/channel/UCov_51F0betb6hJ6Gumxg3Q

Thompson, T. (2025, April 23). *How AI is Actually Used in the Video Games Industry*. https://www.aiandgames.com/p/how-ai-is-actually-used-in-the-video

Top Video Game Genres in 2025: Revenue, Statistics. (2024, January 29). https://rocketbrush.com/blog/most-popular-video-game-genres-in-2024-revenue-statistics-genres-overview

Torossian, R. (2024, December 19). *The evolution of gaming PR: Building communities and crafting digital narratives - Agility PR Solutions*. https://www.agilitypr.com/pr-news/public-relations/the-evolution-of-gaming-pr-building-communities-and-crafting-digital-narratives/

UNI Global Union. (2022, June 16). The video game industry: A resource for organizers. *UNI Global Union*. https://uniglobalunion.org/report/the-video-game-industry/

VIDEO GAME | English meaning—Cambridge Dictionary. (n.d.). Retrieved March 21, 2025, from https://dictionary.cambridge.org/dictionary/english/video-game

Video Game Consoles Dimensions & Drawings | Dimensions.com. (n.d.). Retrieved March 24, 2025, from https://www.dimensions.com/collection/video-game-consoles

Video Game Database | MobyGames. (n.d.). MobyGames. Retrieved May 12, 2025, from https://www.mobygames.com/

Video Game Industry Statistics and Facts | GameStats. (n.d.). Retrieved May 12, 2025, from https://games-stats.com/

Video Game Market Size, Share And Growth Report, 2030. (n.d.). Retrieved March 21, 2025, from https://www.grandviewresearch.com/industry-analysis/video-game-market

Walfisz, M., Zackariasson, P., & Wilson, T. L. (2006, February). *(PDF) Real-Time Strategy: Evolutionary Game Development*. https://www.researchgate.net/publication/4885334_Real-Time_Strategy_Evolutionary_Game_Development

Webber, J. E. (2016, February 9). The Witness: How Jonathan Blow rejected game design rules to make a masterpiece. *The Guardian*. https://www.theguardian.com/technology/2016/feb/09/the-witness-how-jonathan-blow-rejected-game-design-rules-to-make-a-masterpiece

Welcome to the Krita 5.2 Manual! —Krita Manual 5.2.0 documentation. (n.d.). Retrieved May 9, 2025, from https://docs.krita.org/en/en/index.html

What is Artificial Intelligence? | NASA. (2024, May 13). https://www.nasa.gov/what-is-artificial-intelligence/

What is Symbolic AI? | GeeksforGeeks. (2024, September 5). GeeksforGeeks. https://www.geeksforgeeks.org/what-is-symbolic-ai/

What is Wireframing? The Complete Guide [Free Checklist]. (n.d.). Figma. Retrieved May 7, 2025, from https://www.figma.com/resource-library/what-is-wireframing/

Whitehead, R. (2024, May 23). *The Role of Data Science in the Gaming Industry*. IoA - Institute of Analytics. https://ioaglobal.org/blog/role-of-data-science-in-gaming-industry/

Wijman, T. (2024, February 8). Newzoo's games market revenue estimates and forecasts by region and segment for 2023. *Newzoo*. https://newzoo.com/resources/blog/games-market-estimates-and-forecasts-2023

Zack Daniels. (2023, December 6). *Indie Games Need a Better Definition | by Zack Daniels | Medium*. https://medium.com/%40ZackkWrites/indie-games-need-a-better-definition-19ae944a3cdb

Zolotarenko, J. (2024, March 11). Indie, AA, and AAA Games: The Ultimate Guide. *Medium*. https://hitberrygames.medium.com/indie-aa-and-aaa-games-the-ultimate-guide-4147c393ca72

Appendices

Appendix A: Manually Created Game Prototype

This appendix contains the desktop version of manually created game. The game is also available from (https://i-antiva-i.github.io/game-approach-a/).

Appendix B: Artificially Created Game Prototype

This appendix contains the desktop version of artificially created game. The game is also available from (https://i-antiva-i.github.io/game-approach-b/).

Appendix C: Google Forms Survey Questionnaire

This appendix contains questions from "AI: Usage, Impact, and Opinions" questionnaire. The questions are presented in English.

Appendix D: Google Forms Survey Data

This appendix includes the collected results from "AI: Usage, Impact, and Opinions" survey.

Appendix E: Conversations with ChatGPT

This appendix contains several conversations with ChatGPT, what occurred during the artificial development process. The conversations demonstrate how prompts were used to co-work with the AI.

Appendix F: Playtesting Data

This appendix contains review and feedback data from the playtesting sessions.

Appendix G: Other

This appendix contains various data (texts, images, and other assets) that I was unable to include or describe in the main body of the work due to time or other constraints.